
Bit statt Float? Eine vergleichende Untersuchung binärer und kontinuierlicher Embeddings für semantisches Retrieval

Projektarbeit
zum Modul *Deep Learning, Machine Learning und Künstliche Intelligenz*
im Studiengang Informatik
an der Fakultät für Informatik und Ingenieurwissenschaften
der Technischen Hochschule Köln

vorgelegt von: Nick Betker
Nils Morczinietz
Jesko Förster

eingereicht bei: Prof. Dr. Daniel Gaida

Gummersbach, TT.MM.JJJJ

Kurzfassung

Embedding-basierte Vektorsuchsysteme bilden das Rückgrat moderner Informationsrückgewinnung und agentenbasierter KI-Architekturen. Mit wachsenden Dokumentkorpora steigen Speicherbedarf und Betriebskosten hochdimensionaler Float32-Vektoren jedoch drastisch an. Die nachträgliche Quantisierung auf wenige Bits pro Dimension verspricht erhebliche Speichervorteile, führt jedoch unweigerlich zu Informationsverlusten.

In dieser Arbeit wird der Einfluss von Quantisierung und Dimensionsreduktion auf die Retrieval-Qualität und den Ressourcenbedarf systematisch und kausal untersucht. Anhand eines kontrollierten Sechs-Phasen-Experiments auf Basis des Embedding-Modells `bge-large-en-v1.5` und dreier heterogener BEIR-Datensätze (SciFact, FiQA, TREC-COVID) wird die Erhaltung geometrischer Raumeigenschaften, Distanzkorrelationen, Nachbarschaften und Retrieval-Metriken (NDCG@10, Recall@100) über verschiedene Bittiefen (1 bis 8 Bit) sowie PCA-Zielstufen evaluiert. Verglichen werden naive Quantisierungsansätze mit dem rotationsbasierten Verfahren TurboQuant.

Die Ergebnisse zeigen, dass eine Kompression bis 4-Bit praktisch verlustfrei bleibt und über 99,5 % der Float32-Retrieval-Qualität bei einer Speicherreduktion um das 7,8-Fache wahrt. Für kostenoptimierte Cloud-Systeme bietet 2-Bit bei voller Dimension (1024d) den besten Kompromiss mit 98,1 % erhaltener Qualität und 15-facher Speichersparnis, was unter den gewählten Annahmen einer modellierten AWS-Infrastruktur 93,7 % der monatlichen Instanzkosten einspart. Selbst 1-Bit-Konfigurationen behalten eine hohe Trustworthiness und eignen sich für zweistufige Filterstrukturen sowie lokale On-Device-Szenarien mit 100 000 bis 500 000 Vektoren unterhalb von 50 MiB Arbeitsspeicher. Die geometrische Analyse belegt zudem, dass TurboQuant durch orthogonale Rotation das bei nativer Binarisierung auftretende Verdünnungsparadox unter Dimensionsreduktion erfolgreich verhindert.

Schlagwörter: Vektor-Embeddings, Quantisierung, TurboQuant, Semantische Suche, Information Retrieval, Retrieval-Augmented Generation (RAG), Matryoshka-Reduktion, Embedding-Kompression, Skalierbarkeit

Inhaltsverzeichnis

Tabellenverzeichnis	V
Abbildungsverzeichnis	VI
Glossar	VII
Abkürzungsverzeichnis	VIII
Symbolverzeichnis	IX
1 Einleitung	1
1.1 Problemstellung	1
1.2 Quantisierung als Lösungsansatz	1
1.3 Stand der Forschung	2
1.4 Forschungsfrage und Zielsetzung	3
1.5 Methodischer Ansatz	3
1.6 Aufbau der Arbeit	4
2 Grundlagen	5
2.1 Embeddings und vektorbasierte Suche	5
2.1.1 Ähnlichkeitsmaße	5
2.1.2 Speicher- und Rechenkomplexität	6
2.2 Quantisierung von Vektorrepräsentationen	6
2.2.1 Quantisierung und Dequantisierung	7
2.2.2 Qualitätskriterien	7
2.3 Binäre Embeddings und Hamming-Distanz	8
2.3.1 Hamming-Distanz	8
2.3.2 Informationsverlust der Binarisierung	8
2.4 TurboQuant	9
2.4.1 Varianzumverteilung als Schlüsselmechanismus	10
2.4.2 Einordnung	10
2.5 Dimensionsreduktion durch Hauptkomponentenanalyse	10
2.5.1 Abgrenzung zu Matryoshka Representation Learning	11
2.6 Intrinsische Dimensionalität	11

2.7	Evaluationsmetriken für Information Retrieval	12
2.7.1	Normalized Discounted Cumulative Gain	12
2.7.2	Recall	13
2.7.3	Mean Reciprocal Rank	13
2.7.4	Zusammenspiel der Metriken	14
2.8	Anwendungskontext: Praktischer Einsatz vektorbasierter Suche	14
3	Methodik	15
3.1	Experimenteller Rahmen	15
3.2	Datensatz	16
3.3	Embedding-Modell	16
3.4	Erzeugung der Repräsentationsräume	17
3.4.1	Binäre Repräsentationen	18
3.4.2	TurboQuant-Repräsentation	18
3.4.3	Dimensionsreduktion als zusätzlicher Kompressionsparameter	20
3.5	Phase 1: Charakterisierung des Float-Raums	21
3.5.1	Normverteilung	21
3.5.2	Dimensionsstatistiken	22
3.5.3	Intrinsische Dimensionalität	22
3.6	Phase 2: Distanz- und Fehleranalyse	23
3.6.1	Paarweise Distanzkorrelation	23
3.6.2	Absolute Verzerrung (Distortion)	24
3.7	Phase 3: Nachbarschaftserhaltung	25
3.7.1	Ablauf	25
3.7.2	Distanzmetriken in den komprimierten Räumen	26
3.7.3	Neighborhood Overlap	26
3.7.4	Trustworthiness	27
3.8	Phase 4: Retrieval-Evaluation	28
3.8.1	Retrieval-Protokoll	28
3.8.2	Metriken	29
3.8.3	Statistische Signifikanz	29
3.9	Phase 5: Praktische Evaluation	29
3.9.1	Speicheranalyse	30
3.9.2	Laufzeitanalyse	30
3.9.3	Qualitäts-Speicher-Trade-off	31
3.10	Phase 6: Praktische Anwendungsfall- und Skalierungsanalyse	31
3.10.1	Ziel und Abgrenzung	31
3.10.2	Szenarioauswahl	32
3.10.3	Skalierungsrechnung	33
3.10.4	Kostenmodell	33
3.11	Synthese und Interpretation	34

4	Ergebnisse und Diskussion	35
4.1	Phase 1: Charakterisierung des Float-Raums	35
4.1.1	Normverteilung	35
4.1.2	Dimensionsstatistiken	36
4.1.3	Intrinsische Dimensionalität	38
4.1.4	Zusammenfassung und Vorhersagen	40
4.2	Phase 2: Distanz- und Fehleranalyse	41
4.2.1	Distanzerhaltung bei voller Dimensionszahl	41
4.2.2	Einfluss der Dimensionsreduktion	42
4.2.3	Gesamtmatrix und absolute Verzerrung	45
4.2.4	Pareto-Analyse: Qualität vs. Speicherbedarf	46
4.2.5	Diskussion und Bezug zu Phase 1	47
4.3	Phase 3: Nachbarschaftserhaltung	48
4.3.1	Nachbarschaftserhaltung bei voller Dimensionszahl	49
4.3.2	Einfluss der Dimensionsreduktion	50
4.3.3	Pareto-Analyse: Nachbarschaft vs. Speicherbedarf	53
4.3.4	Diskussion und Bezug zu den vorherigen Phasen	53
4.4	Phase 4: Retrieval-Evaluation	54
4.4.1	Retrieval-Qualität bei voller Dimensionszahl	54
4.4.2	Einfluss der Dimensionsreduktion	56
4.4.3	Recall für zweistufige Systeme	57
4.4.4	Pareto-Analyse: Qualität vs. Speicherbedarf	57
4.4.5	Diskussion und Bezug zu den vorherigen Phasen	58
4.5	Phase 5: Speicher- und Laufzeiteffizienz	60
4.5.1	Speicherbedarf	60
4.5.2	Laufzeit	62
4.5.3	Qualitäts-Speicher-Trade-off	65
4.5.4	Diskussion und Bezug zu den vorherigen Phasen	66
4.6	Phase 6: Praktische Anwendungsfall- und Skalierungsanalyse	67
4.6.1	Szenarioüberblick	67
4.6.2	Mobile Skalierung unterhalb von 10 Millionen Vektoren	68
4.6.3	Cloud-Kosten und betriebliche Relevanz	70
4.6.4	Einordnung	73
4.7	Synthese: Beantwortung der Forschungsfrage	73
	Literatur	76
	Anhang	82

Tabellenverzeichnis

4.1	L2-Normverteilung der Float32-Embeddings	35
4.2	Dimensionsstatistiken über alle 1024 Dimensionen	36
4.3	Intrinsische Dimensionalität nach TwoNN und PCA	38
4.4	Distanzkorrelation bei voller Dimensionszahl	42
4.5	Neighborhood Overlap und Trustworthiness bei voller Dimensionszahl	49
4.6	Retrieval-Metriken bei voller Dimensionszahl	55
4.7	Speicherbedarf pro Vektor bei voller Dimensionszahl	60
4.8	Laufzeit im NumPy-Pfad bei voller Dimensionszahl	63
4.9	Praktische TurboQuant-Szenarien in Phase 6	67
4.10	Dimensionsvergleich für <code>tq_1bit</code> im mobilen Szenario	69
4.11	AWS-Kostenrechnung für einen 10-Millionen-Vektorindex	72

Abbildungsverzeichnis

4.1	Verteilung von Schiefe und Kurtosis über alle Dimensionen	37
4.2	PCA-Varianzspektrum	39
4.3	Kumulative erklärte Varianz	40
4.4	Pearson r als Funktion der PCA-Dimension	43
4.5	Spearman ρ als Funktion der PCA-Dimension	44
4.6	Heatmap der Pearson-Korrelation (SciFact)	46
4.7	Pareto-Front der Distanzerhaltung	47
4.8	Neighborhood Overlap als Funktion der PCA-Dimension	51
4.9	Trustworthiness als Funktion der PCA-Dimension	52
4.10	Neighborhood Overlap als Funktion der Nachbarschaftsgröße	52
4.11	Pareto-Front der Nachbarschaftserhaltung	53
4.12	NDCG@10 als Funktion der PCA-Dimension	56
4.13	Recall@100 als Funktion der PCA-Dimension	58
4.14	Pareto-Front der Retrieval-Qualität	59
4.15	Theoretischer und gemessener Speicherbedarf pro Repräsentation	61
4.16	Kompressionsfaktor als Funktion der PCA-Dimension	62
4.17	Laufzeit der paarweisen Distanzberechnung	63
4.18	Laufzeit der Brute-Force-Top- k -Suche	64
4.19	Compression-Quality-Score CQ_1 als Funktion der PCA-Dimension	65
4.20	Speicherbedarf der Phase-6-Szenarien bei 10 Millionen Vektoren	68
4.21	Qualitäts-Speicher-Trade-off mit Anwendungsszenarien	69
4.22	Mobile Speicherskalierung unterhalb von 10 Millionen Vektoren	71
4.23	Geschätzte monatliche AWS-Kosten für einen 10-Millionen-Vektorindex	72

Glossar

Binarisierung Extremfall der Quantisierung, bei dem jede Dimension anhand ihres Vorzeichens auf ein einzelnes Bit reduziert wird.

Distortion Maß für die Verzerrung, die durch die Kompression zwischen dem Originalraum und dem rekonstruierten Raum entsteht.

Embedding Abbildung eines Datenobjekts wie Text oder Bild auf einen hochdimensionalen Vektor, dessen geometrische Nähe semantische Ähnlichkeit widerspiegelt.

Pareto-Front Menge der nicht dominierten Konfigurationen, für die keine andere Konfiguration zugleich einen geringeren Speicherbedarf und eine höhere Qualität erreicht.

Quantisierung Reduktion der Darstellungsgenauigkeit eines Vektors, indem kontinuierliche Werte auf eine endliche Anzahl diskreter Stufen mit weniger Bits pro Dimension abgebildet werden.

Abkürzungsverzeichnis

ANN Approximate Nearest Neighbor.

AWS Amazon Web Services.

BEIR Benchmarking Information Retrieval.

BLAS Basic Linear Algebra Subprograms.

DCG Discounted Cumulative Gain.

EC2 Elastic Compute Cloud.

MRL Matryoshka Representation Learning.

MRR Mean Reciprocal Rank.

MTEB Massive Text Embedding Benchmark.

NDCG Normalized Discounted Cumulative Gain.

PCA Principal Component Analysis (Hauptkomponentenanalyse).

PQ Product Quantization.

RAG Retrieval-Augmented Generation.

TwoNN Two Nearest Neighbors.

Symbolverzeichnis

CQ_1 Compression-Quality-Score, kombiniert Qualitätserhaltung und Speicherersparnis zu einer skalaren Trade-off-Kennzahl.

$M(N)$ Speicherbedarf eines Index mit N Vektoren.

N Anzahl der Vektoren im Index beziehungsweise Korpus.

$\mathbf{b}$ Binärer Vektor aus $\{0, 1\}^d$, Ergebnis der Binarisierung.

ρ Spearman-Rangkorrelationskoeffizient, misst die Rangerhaltung von Distanzen.

d_{int} Intrinsische Dimensionalität, effektive Anzahl unabhängiger Freiheitsgrade des Datenraums.

d Dimensionszahl der Vektorrepräsentation.

k Größe der betrachteten Nachbarschaft oder Position des Rang-Cutoffs bei Retrieval-Metriken.

r Pearson-Korrelationskoeffizient, misst den linearen Zusammenhang zwischen Distanzen.

1 Einleitung

1.1 Problemstellung

Embedding-basierte Suchsysteme bilden einen zentralen Baustein moderner KI-Anwendungen. Sie überführen Texte, Dokumente oder andere Datenobjekte in hochdimensionale Vektorrepräsentationen, deren geometrische Nähe semantische Ähnlichkeit abbildet. Diese Technik kommt unter anderem bei der semantischen Suche [42, 29], bei Retrieval-Augmented Generation [32] und bei Ähnlichkeitsanalysen zum Einsatz.

Der Speicherbedarf solcher Systeme skaliert linear mit der Korpusgröße und der Vektordimension. Ein Korpus von einer Million Dokumenten mit 1024-dimensionalen Float32-Vektoren belegt bereits rund 4 GB Arbeitsspeicher allein für die Vektordaten. In Szenarien mit zehn oder hundert Millionen Dokumenten übersteigt der Ressourcenbedarf schnell die Kapazitäten typischer Produktivsysteme. Hinzu kommt, dass jede Suchanfrage im Brute-Force-Fall eine vollständige Distanzberechnung gegen alle gespeicherten Vektoren erfordert, was die Antwortlatenz linear mit der Korpusgröße ansteigen lässt [26].

Besonders deutlich wird dieses Spannungsfeld im Kontext agentenbasierter KI-Systeme. Solche Systeme ermöglichen es Sprachmodellen, zur Laufzeit auf externe Werkzeuge zuzugreifen und dabei Retrieval-Operationen auszuführen [44, 56]. Die Echtzeitanforderungen solcher Architekturen verschärfen den Bedarf an kompakten, schnell durchsuchbaren Repräsentationen. Dies ist jedoch nur ein Beispiel für ein breiteres Praxisfeld, das von lokaler Suche auf Endgeräten bis zu kostenoptimierten Retrieval-Diensten in der Cloud reicht und in dem Speicher- und Kostenbudgets die einsetzbare Repräsentation maßgeblich bestimmen.

1.2 Quantisierung als Lösungsansatz

Quantisierung adressiert dieses Problem, indem sie die Genauigkeit der Vektordarstellung gezielt reduziert [19]. Anstelle von 32 Bit pro Dimension werden nur noch 4, 2 oder im Extremfall 1 Bit verwendet. Der theoretische Speichervorteil ist erheblich: Eine Reduktion von Float32 auf 1 Bit entspricht einem Kompressionsfaktor von 32

bei gleicher Dimensionszahl; kombiniert mit einer Dimensionsreduktion sind Faktoren von über 300 erreichbar.

Zugleich ist der Informationsverlust bei Quantisierung unvermeidlich. Werden kontinuierliche Werte auf eine endliche Stufenzahl abgebildet, gehen Unterscheidungen verloren. Die entscheidende Frage ist daher nicht, *ob* Information verloren geht, sondern *welche* Informationen verloren gehen und ob die für das Retrieval relevante Struktur des Vektorraums erhalten bleibt.

1.3 Stand der Forschung

Die Kompression von Vektorrepräsentationen für die Ähnlichkeitssuche ist seit Jahrzehnten Gegenstand der Forschung. Die informationstheoretische Grundlage bildet die Rate-Distortion-Theorie [46]; eine umfassende Übersicht über skalare und Vektorquantisierung geben Gray und Neuhoff [19]. Eine erste Verfahrenslinie bilden Hashing-Ansätze, die Vektoren auf kurze Binärcodes abbilden: Charikar zeigt, dass die Vorzeichen zufälliger Projektionen den Winkel zwischen zwei Vektoren unverzerrt schätzen [11], und Iterative Quantization lernt eine Rotation, die den Fehler der anschließenden Binarisierung minimiert [18]. Eine zweite Linie bildet die Product Quantization [24] mit ihrer Erweiterung Optimized Product Quantization [17]; sie zerlegt Vektoren in Teilräume und kodiert jeden Teilraum über ein trainiertes Codebuch. Aktuelle Suchbibliotheken wie ScaNN [20] und Faiss [26, 14] kombinieren solche Kompressionsverfahren mit approximativen Indexstrukturen, etwa dem graphbasierten HNSW [33]; ANN-Benchmarks [8] und Übersichtsarbeiten zu Vektordatenbanken [40] dokumentieren den Stand dieser Systeme.

Mit der Verbreitung dichter Retrieval-Modelle [42, 29] hat die Kompression von Text-Embeddings zusätzliche praktische Bedeutung erlangt. Yamada et al. zeigen, dass binäre Passage-Repräsentationen in Kombination mit einem nachgeschalteten Reranking nahezu die Qualität des unkomprimierten Retrievals erreichen [55], und aktuelle Praxisberichte belegen, dass nachträgliche Binär- und Int8-Quantisierung moderner Embedding-Modelle einen Großteil der Retrieval-Qualität bei drastisch reduziertem Speicherbedarf erhält [45]. Zandieh et al. beschreiben mit TurboQuant eine Methode, die Vektoren mittels orthogonaler Rotation und uniformer skalarer Quantisierung auf wenige Bits pro Dimension komprimiert und dabei sowohl Distortion als auch Retrieval-Qualität günstig beeinflusst [57]. Das Open-Source-Projekt turbovec setzt diesen Ansatz als Vektorindex mit Python-Anbindung um [12].

Einen komplementären Weg verfolgt Matryoshka Representation Learning [30], bei dem Embedding-Modelle so trainiert werden, dass auch verkürzte Vektorpräfixe noch

leistungsfähig bleiben. Nomic Embed v1.5 [39] erweitert diesen Ansatz um eine explizite Optimierung auf binäre Ausgabe.

Trotz dieser Fortschritte fehlt eine systematische empirische Analyse, die den Einfluss von Quantisierung auf die Suchqualität kausal erklärt. Bestehende Benchmarks messen zwar Retrieval-Metriken bei verschiedenen Kompressionsraten, führen aber den beobachteten Qualitätsverlust selten auf die geometrischen Eigenschaften des Ausgangsraums zurück. Damit bleibt unklar, *warum* bestimmte Verfahren auf bestimmten Daten besser oder schlechter funktionieren.

1.4 Forschungsfrage und Zielsetzung

Die vorliegende Arbeit untersucht die zentrale Frage: *Wie beeinflusst die Quantisierung von Embedding-Vektoren die Retrieval-Qualität und den Speicherverbrauch eines vektorbasierten Suchsystems, und welche semantischen Eigenschaften des Float-Raums bleiben unter Kompression erhalten beziehungsweise gehen verloren?*

Der Fokus liegt damit auf dem Trade-off zwischen Kompressionsgrad und Suchqualität. Die Arbeit zielt nicht auf die Entwicklung neuer Quantisierungsverfahren oder das Training spezialisierter Embedding-Modelle ab, sondern auf ein empirisch fundiertes Verständnis des Informationsverlusts bei nachträglicher Quantisierung. Konkret werden folgende Teilfragen adressiert:

1. Welche statistischen und geometrischen Eigenschaften des Float-Embedding-Raums determinieren die Quantisierbarkeit?
2. In welchem Maß bleiben paarweise Distanzen und Nachbarschaftsbeziehungen unter verschiedenen Bittiefen und Dimensionsreduktionen erhalten?
3. Ab welchem Kompressionsgrad treten signifikante Verluste in der Retrieval-Qualität auf, gemessen an etablierten Information-Retrieval-Metriken?
4. Welches Verhältnis von Speicherersparnis zu Qualitätsverlust ergibt sich für verschiedene Kombinationen aus Bittiefe und Dimensionszahl?

1.5 Methodischer Ansatz

Zur Beantwortung dieser Fragen wird ein kontrolliertes Experiment in sechs aufeinander aufbauenden Phasen durchgeführt (vgl. Kapitel 3). Den Ausgangspunkt bildet ein Float32-Referenzraum, der mit dem Modell `bge-large-en-v1.5` [54] auf drei heterogenen BEIR-Datensätzen [48] erzeugt wird. Dieser Referenzraum wird nachträglich mit

verschiedenen Verfahren komprimiert: naive Binarisierung (1 Bit), rotationsbasierte Quantisierung nach dem TurboQuant-Prinzip (1, 2, 4 und 8 Bit) sowie PCA-basierte Dimensionsreduktion.

Die Analyse folgt einer Kausalkette von der Raungeometrie bis zur praktischen Anwendung: Zunächst wird der Float-Raum charakterisiert, dann wird die Erhaltung von Distanzen und Nachbarschaften gemessen, darauf aufbauend die Retrieval-Qualität quantifiziert, und abschließend werden Effizienz und Praxistauglichkeit evaluiert. Dieser schrittweise Aufbau ist methodisch notwendig, da er es ermöglicht, Qualitätsverluste kausal auf geometrische Eigenschaften des Ausgangsraums zurückzuführen.

1.6 Aufbau der Arbeit

Kapitel 2 legt die theoretischen Grundlagen zu Embeddings, Quantisierungsverfahren und dem praktischen Anwendungskontext vektorbasierter Suche. Die Methodik in Kapitel 3 beschreibt den experimentellen Aufbau, die Datensätze und die Evaluationsmetriken. Die Ergebnisse werden im Ergebniskapitel entlang der sechs Phasen dargestellt und diskutiert. Die Arbeit schließt mit einer Zusammenfassung der zentralen Erkenntnisse und einem Ausblick auf weiterführende Fragestellungen.

2 Grundlagen

Dieses Kapitel führt die theoretischen Grundlagen ein, auf denen die empirische Untersuchung aufbaut. Die Darstellung folgt einer logischen Abhängigkeitsstruktur: Zunächst wird das Konzept der Embedding-basierten Suche als Gegenstandsbereich der Arbeit eingeführt. Darauf aufbauend werden die Kompressionsverfahren erläutert, die den Untersuchungsgegenstand bilden. Anschließend werden die mathematischen Konzepte dargestellt, die für das Verständnis der experimentellen Ergebnisse notwendig sind. Den Abschluss bilden die Evaluationsmetriken und der Anwendungskontext.

2.1 Embeddings und vektorbasierte Suche

Embedding-Verfahren überführen diskrete Objekte wie Wörter, Sätze oder Dokumente in Vektoren eines kontinuierlichen Raums $\mathbb{R}^d$ [35, 42]. Die zentrale Eigenschaft dieser Abbildung ist, dass semantische Ähnlichkeit als geometrische Nähe kodiert wird: Objekte mit verwandter Bedeutung liegen im Vektorraum nahe beieinander, unabhängig von ihrer lexikalischen Oberflächenform.

Vektorbasierte Suchsysteme nutzen diese Eigenschaft, um zu einer gegebenen Anfrage die semantisch ähnlichsten Dokumente aus einem Korpus zu identifizieren. Die Anfrage wird ebenfalls als Vektor kodiert und anschließend mit allen Dokumentenvektoren verglichen. Das Ergebnis ist eine nach Ähnlichkeit sortierte Rangliste. Das Prinzip geht auf das Vektorraummodell des Information Retrieval zurück [43, 34]; an die Stelle gewichteter Termvektoren treten dabei gelernte dichte Repräsentationen [29].

2.1.1 Ähnlichkeitsmaße

Das in der Embedding-Literatur gebräuchlichste Ähnlichkeitsmaß ist die Cosine-Similarity [34, 42]:

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|_2 \cdot \|\mathbf{y}\|_2} \quad (2.1)$$

Sie misst den Kosinus des Winkels zwischen zwei Vektoren und ist damit invariant gegenüber deren Betrag. Werte nahe 1 zeigen hohe semantische Ähnlichkeit an, Werte nahe 0 Orthogonalität, also semantische Unabhängigkeit.

Für normierte Vektoren ($\|\mathbf{x}\|_2 = 1$) vereinfacht sich Gleichung 2.1 zum Skalarprodukt $\mathbf{x} \cdot \mathbf{y}$, was die Berechnung auf eine einzelne Matrixmultiplikation reduziert. Moderne Embedding-Modelle erzeugen in der Regel bereits normierte Ausgaben, sodass dieser Spezialfall in der Praxis der Regelfall ist [42, 54].

Für normierte Vektoren besteht zudem ein direkter Zusammenhang zwischen euklidischer Distanz und Cosine-Similarity: Es gilt $\|\mathbf{x} - \mathbf{y}\|_2^2 = 2 - 2(\mathbf{x} \cdot \mathbf{y})$. Beide Maße induzieren damit dieselbe Rangordnung der Nachbarn, was den späteren Vergleich von Repräsentationen erlaubt, die unterschiedliche native Distanzmetriken verwenden.

2.1.2 Speicher- und Rechenkomplexität

Der Ressourcenbedarf eines Brute-Force-Suchsystems ist durch zwei Größen dominiert. Der Speicherbedarf beträgt $N \cdot d \cdot b$ Bit, wobei N die Korpusgröße, d die Vektordimension und b die Bitbreite pro Wert bezeichnet. Für Float32-Darstellungen ($b = 32$) ergibt sich bei $N = 10^7$ Dokumenten und $d = 1024$ ein Speicherbedarf von $10^7 \times 1024 \times 32 \approx 41$ GB.

Die Suchkomplexität einer einzelnen Anfrage skaliert mit $\mathcal{O}(N \cdot d)$ Multiplikationen. Bei Brute-Force-Suche wächst die Antwortlatenz linear mit der Korpusgröße, was für Echtzeit-Anwendungen mit großen Korpora unpraktikabel wird. Diese Skalierungseigenschaften motivieren sowohl approximative Indexstrukturen [33, 26] als auch die im Folgenden beschriebenen Kompressionsverfahren; die vorliegende Arbeit beschränkt sich bewusst auf Letztere (vgl. Abschnitt 3.8).

2.2 Quantisierung von Vektorrepräsentationen

Quantisierung bezeichnet die Abbildung eines kontinuierlichen Wertebereichs auf eine endliche Menge diskreter Stufen [19]. Im informationstheoretischen Rahmen formalisiert Shannons Rate-Distortion-Theorie [46] den fundamentalen Trade-off zwischen der Kompressionsrate (Bits pro Symbol) und der unvermeidlichen Verzerrung (Distortion) der rekonstruierten Werte.

Im Kontext von Embedding-Vektoren bedeutet Quantisierung, dass jede Dimension eines Vektors statt mit 32 Bit (Float32) mit einer reduzierten Anzahl von Bits dargestellt wird. Der resultierende Informationsverlust ist unvermeidlich: Werden 2^{32} mögliche Werte auf 2^b Stufen mit $b < 32$ abgebildet, gehen Unterscheidungen verloren. Die zentrale Frage ist nicht, *ob* Information verloren geht, sondern *welche* Informationsanteile betroffen sind und ob die für eine gegebene Anwendung relevante Struktur des Raums erhalten bleibt.

2.2.1 Quantisierung und Dequantisierung

Die in dieser Arbeit betrachtete Multi-Bit-Quantisierung arbeitet pro Dimension mit einer Min-Max-Skalierung. Für jede Dimension werden das Minimum und das Maximum über den Datensatz bestimmt und der Wertebereich linear auf 2^b gleichmäßig verteilte Stufen abgebildet; dies entspricht einem uniformen skalaren Quantisierer [19]. Um aus einem quantisierten Code wieder einen näherungsweisen Float-Wert zu gewinnen, ist der umgekehrte Schritt nötig, die *Dequantisierung*: Der Code wird mithilfe der gespeicherten Grenzwerte zurück auf den ursprünglichen Wertebereich skaliert.

Die Grenzwerte pro Dimension müssen daher zusätzlich zu den eigentlichen Codes gespeichert werden. Sie bilden einen festen Metadaten-Aufwand, der unabhängig von der Anzahl der Vektoren ist und den praktischen Speicherbedarf gegenüber der reinen Bitpackung leicht erhöht. Distanzberechnungen auf quantisierten Multi-Bit-Vektoren erfolgen in dieser Arbeit auf den dequantisierten, also rekonstruierten Werten; die 1-Bit-Variante benötigt keine solche Rekonstruktion, da hier direkt die Hamming-Distanz auf den Bitcodes verwendet wird.

2.2.2 Qualitätskriterien

Zwei komplementäre Eigenschaften bestimmen die Eignung einer Quantisierung für Retrieval-Anwendungen:

- **Distortion** bezeichnet die mittlere Verzerrung der paarweisen Distanzen zwischen Vektoren. Niedrige Distortion bedeutet, dass die geometrische Struktur des Raums – also die relativen Abstände der Punkte zueinander – weitgehend erhalten bleibt.
- **Rangerhaltung** bezeichnet die Fähigkeit, die relative Reihenfolge der Ähnlichkeiten korrekt wiederzugeben. Für Retrieval-Systeme, deren Ausgabe eine Rangliste ist, ist diese Eigenschaft wichtiger als absolute Distanztreue. Solange das korrekte Dokument den ersten Rang einnimmt, ist die absolute Distanz irrelevant.

Diese Unterscheidung ist für die Interpretation der experimentellen Ergebnisse zentral. Ein Verfahren kann hohe Distortion aufweisen, aber dennoch für Retrieval geeignet sein, sofern die Rangordnung erhalten bleibt. Umgekehrt kann niedrige Distortion mit gestörter Rangordnung einhergehen, wenn systematische Verzerrungen die Reihenfolge naher Punkte invertieren.

Da Kompression stets einen Kompromiss zwischen Qualität und Speicherbedarf darstellt, werden die Ergebnisse häufig als Pareto-Front dargestellt. Eine Konfiguration liegt auf der Pareto-Front, wenn keine andere Konfiguration bei gleichem oder geringerem Speicherbedarf eine bessere Qualität erreicht; solche Konfigurationen sind nicht dominiert und bilden die jeweils bestmögliche Wahl für ein gegebenes Speicherbudget.

2.3 Binäre Embeddings und Hamming-Distanz

Die extremste Form der skalaren Quantisierung reduziert jede Dimension auf ein einzelnes Bit [11, 18, 45]. Die naive Binarisierung nutzt das Vorzeichen als Schwellwert:

$$b_i = \begin{cases} 1 & \text{falls } x_i > 0 \\ 0 & \text{sonst} \end{cases} \quad (2.2)$$

Der resultierende Bitvektor $\mathbf{b} \in \{0, 1\}^d$ benötigt pro Dimension $\frac{1}{32}$ des Speichers eines Float32-Vektors, entsprechend einem Kompressionsfaktor von 32.

2.3.1 Hamming-Distanz

Die Ähnlichkeit zweier binärer Vektoren wird über die Hamming-Distanz [21] bestimmt:

$$d_H(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^d \mathbf{1}[a_i \neq b_i] \quad (2.3)$$

Der Ausdruck $\mathbf{1}[a_i \neq b_i]$ ist die Indikatorfunktion; sie ist 1, wenn sich die beiden Bits an Position i unterscheiden, und sonst 0. Die Hamming-Distanz zählt also schlicht die Anzahl der Positionen, an denen sich zwei Bitvektoren unterscheiden. Sie lässt sich auf moderner Hardware sehr effizient berechnen: Ein bitweises Exklusiv-Oder (XOR) markiert die abweichenden Positionen mit einer 1, und ein Population-Count (das Zählen der gesetzten Einsen) liefert das Ergebnis in wenigen Taktzyklen [38]. Gegenüber der Float-Cosine-Similarity, die d Multiplikationen und $d - 1$ Additionen erfordert, ermöglicht dies Geschwindigkeitsvorteile um Größenordnungen.

2.3.2 Informationsverlust der Binarisierung

Die Vorzeichenfunktion in Gleichung 2.2 bildet jede Komponente x_i allein anhand ihres Vorzeichens auf ein Bit ab. Zwei Informationsanteile gehen dabei verloren, weil

sie in den Bitvektor nicht eingehen: der Betrag $|x_i|$ jeder einzelnen Komponente und die L2-Norm $\|\mathbf{x}\|_2$ des Gesamtvektors, die sich aus diesen Beträgen zusammensetzt. Erhalten bleibt allein die Information, in welchem Orthanten des Raums der Vektor liegt, also seine grobe Richtung. Ob dieser Verlust für eine gegebene Aufgabe tolerierbar ist, hängt davon ab, wie die semantische Information im Vektorraum verteilt ist. Liegt die relevante Struktur überwiegend in der Richtung (den Vorzeichen) der Vektoren und nicht in ihren Beträgen, ist der Verlust gering; auf dieser Beobachtung beruhen binäre Hashing-Verfahren, deren Hamming-Distanz den Winkel zwischen Vektoren approximiert [11, 18]. Liegt umgekehrt substantielle Information in den Magnitudenunterschieden, wird die Binarisierung diese zerstören.

2.4 TurboQuant

TurboQuant [57] adressiert eine grundlegende Schwäche der naiven skalaren Quantisierung: die ungleiche Informationsverteilung über die Dimensionen. In typischen Embedding-Räumen konzentrieren wenige Dimensionen einen Großteil der Varianz, während die meisten Dimensionen fast konstant sind. Ein uniformer Quantisierer, der jede Dimension mit derselben Stufenzahl behandelt, verschwendet Bits auf varianzarme Dimensionen und bietet zu wenig Auflösung für varianzreiche.

TurboQuant löst dieses Problem in zwei Schritten, die keine Trainingsphase auf den Zieldaten erfordern [57]:

1. **Orthogonale Rotation.** Die Eingabevektoren werden mit einer zufälligen orthogonalen Matrix $\mathbf{R} \in \text{SO}(d)$ (der speziellen orthogonalen Gruppe, also den Rotationen im $\mathbb{R}^d$) multipliziert: $\mathbf{x}' = \mathbf{R}\mathbf{x}$. Da orthogonale Transformationen winkel- und abstandserhaltend sind, bleibt die semantische Struktur des Raums exakt erhalten. Die Idee, durch eine zufällige Rotation die Energie eines Vektors gleichmäßig auf alle Koordinaten zu verteilen, geht auf den Fast Johnson-Lindenstrauss Transform zurück [1] und wird auch in Hashing-Verfahren für die Winkeldistanz genutzt [7]. Die Rotation dekorreliert jedoch die Dimensionen und verteilt die Varianz gleichmäßig über alle Achsen. Das ist kein Widerspruch: Die Abstände zwischen den Punkten bleiben identisch, nur die Ausrichtung der Achsen relativ zur Punktwolke ändert sich, sodass keine einzelne Achse mehr die Varianz dominiert.
2. **Uniforme skalare Quantisierung.** Jede Dimension des rotierten Vektors wird unabhängig auf 2^b gleichmäßig verteilte Stufen quantisiert [19], wobei b die gewählte Bittiefe bezeichnet. Nach der Rotation ist jede Dimension gleich informationsreich, sodass ein uniformer Quantisierer seine verfügbaren Stufen optimal nutzen kann.

2.4.1 Varianzumverteilung als Schlüsselmechanismus

Der Beitrag der Rotation liegt formal in der Umverteilung der Varianz [57]. Seien $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$ die Eigenwerte der Kovarianzmatrix des Embedding-Raums; jeder Eigenwert λ_i gibt die Varianz entlang einer der Hauptrichtungen an. Vor der Rotation ist die Varianz asymmetrisch verteilt; einzelne Hauptkomponenten dominieren. Über zufällige Rotationen gemittelt beträgt die erwartete Varianz pro Dimension $\frac{1}{d} \sum_{i=1}^d \lambda_i$ und ist damit für alle Dimensionen gleich. Ein uniformer Quantisierer mit 2^b Stufen verursacht dann auf jeder Dimension näherungsweise denselben Quantisierungsfehler, was die Gesamt-Distortion minimiert.

Die Methode setzt voraus, dass eine gleichmäßige Varianzverteilung die Quantisierung verbessert. Der relevante Aspekt ist dabei die Verteilung der Varianz über die Achsen (das Eigenwertspektrum), nicht die Form der Randverteilung innerhalb einer einzelnen Dimension. TurboQuant ist folglich besonders dann wirksam, wenn die Varianz stark ungleich über die Achsen verteilt ist, wenige Richtungen also den Großteil der Gesamtvarianz tragen. Ob dies für einen konkreten Embedding-Raum zutrifft, ist eine empirische Frage; für den in dieser Arbeit untersuchten Raum wird sie in Phase 1 (Abschnitt 4.1.3) geprüft, mit unmittelbaren Konsequenzen für die zu erwartende Wirksamkeit der Rotation.

2.4.2 Einordnung

TurboQuant übertrifft in Nearest-Neighbor-Aufgaben bestehende Product-Quantization-Verfahren [24] hinsichtlich des Recall bei gleichzeitig deutlich geringerem Indexierungsaufwand, da weder Codebook-Training noch eine Zuordnung zu Subquantisierern erforderlich ist [57].

2.5 Dimensionsreduktion durch Hauptkomponentenanalyse

Neben der Reduktion der Bittiefe pro Dimension bildet die Reduktion der Anzahl der Dimensionen die zweite Achse der Kompression. Die Hauptkomponentenanalyse (PCA) [41, 22] projiziert die Vektoren auf die Richtungen maximaler Varianz im Datenraum; eine umfassende Darstellung gibt Jolliffe [27].

Formal wird die Kovarianzmatrix $\mathbf{C} = \frac{1}{N} \mathbf{X}^\top \mathbf{X}$ des zentrierten Datensatzes $\mathbf{X} \in \mathbb{R}^{N \times d}$ (jede Dimension um ihren Mittelwert verschoben) in ihre Eigenwerte und Eigenvektoren zerlegt. Die Eigenvektoren geben die Hauptrichtungen der Daten an, die zugehörigen Eigenwerte λ_i die Varianz entlang dieser Richtungen. Die Projektion auf die d' Eigenvektoren mit den größten Eigenwerten ergibt eine Repräsentation in

$\mathbb{R}^{d'}$, die den maximalen Anteil der Gesamtvarianz erhält. Die kumulative erklärte Varianz

$$V(d') = \frac{\sum_{i=1}^{d'} \lambda_i}{\sum_{i=1}^d \lambda_i} \quad (2.4)$$

quantifiziert, welcher Anteil der Gesamtvarianz nach der Reduktion erhalten bleibt [27, 28].

Für die vorliegende Arbeit ist PCA in doppelter Hinsicht relevant. Erstens dient sie als Kompressionsverfahren, das orthogonal zur Quantisierung angewendet wird: Ein Vektor kann zunächst in seiner Dimension reduziert und anschließend quantisiert werden, wobei beide Effekte separierbar bleiben. Zweitens liefert die PCA-Analyse des Varianzspektrums eine diagnostische Aussage über die Redundanz des Ausgangsraums. Ein Raum, dessen kumulative Varianz bereits bei wenigen Komponenten nahe 1 liegt, enthält hohe lineare Redundanz und ist daher gut komprimierbar.

2.5.1 Abgrenzung zu Matryoshka Representation Learning

Ein konzeptionell verwandter Ansatz ist Matryoshka Representation Learning (MRL) [30], bei dem ein Embedding-Modell so trainiert wird, dass die ersten d' Originaldimensionen eines d -dimensionalen Vektors bereits eine nutzbare Repräsentation bilden. Im Gegensatz zu PCA erfordert MRL keine nachträgliche Projektion, setzt aber ein speziell trainiertes Modell voraus.

Die vorliegende Arbeit verwendet bewusst kein MRL-Modell, sondern nachträgliche PCA auf einem Standard-Embedding-Raum. Dies ermöglicht die Untersuchung, wie viel Redundanz ein nicht auf Kompression optimiertes Modell über seine Dimensionen verteilt, und isoliert den Effekt der nachträglichen Dimensionsreduktion vom Trainingsverfahren.

2.6 Intrinsische Dimensionalität

Obwohl Embedding-Modelle Vektoren mit einer festen nominalen Dimensionszahl d erzeugen, ist die tatsächliche Anzahl der Freiheitsgrade der Daten oft deutlich geringer. Die Punkte liegen dann auf einer niedrigdimensionalen Mannigfaltigkeit innerhalb des umgebenden hochdimensionalen Raums [10].

Die intrinsische Dimensionalität d_{int} quantifiziert diese effektive Anzahl unabhängiger Freiheitsgrade. Ist $d_{\text{int}} \ll d$, enthält der Raum substantielle Redundanz: Viele Dimensionen tragen keine unabhängige Information bei, sondern sind durch die übrigen vorhersagbar.

Für die Quantisierung hat dies direkte Konsequenzen. Redundante Dimensionen können mit geringem Informationsverlust auf wenige Bits reduziert oder durch Dimensionsreduktion entfernt werden, da ihre Information bereits in anderen Dimensionen kodiert ist. Ein Raum mit niedriger intrinsischer Dimensionalität ist daher grundsätzlich besser für Kompression geeignet als ein Raum, der seine volle nominale Kapazität nutzt.

Zur Schätzung der intrinsischen Dimensionalität existieren verschiedene Verfahren; eine Übersicht geben Campadelli et al. [10]. Der TwoNN-Schätzer [15] nutzt die Verhältnisse der Abstände zum ersten und zweiten nächsten Nachbarn und erfasst damit die lokale Mannigfaltigkeitsstruktur. Er zählt wie der Schätzer von Levina und Bickel [31] zu den nachbarschaftsbasierten Verfahren, die die Dimensionalität aus den Abständen zu den nächsten Nachbarn ableiten. Die PCA-basierte Schätzung über die Anzahl der Komponenten, die einen definierten Varianzanteil (typischerweise 95%) erklären, erfasst hingegen die globale lineare Dimensionalität; sie gehört zu den projektionsbasierten Schätzern, deren Grundidee auf Fukunaga und Olsen zurückgeht [16, 10]. Beide Perspektiven ergänzen einander: Stimmen die Schätzer überein, ist die Aussage robust; divergieren sie, deutet dies auf nichtlineare Strukturen hin, die eine rein lineare Analyse nicht erfasst [10].

2.7 Evaluationsmetriken für Information Retrieval

Die Bewertung der Suchqualität erfordert Metriken, die quantifizieren, wie gut eine Rangliste die tatsächlich relevanten Dokumente identifiziert. Im Folgenden werden die in dieser Arbeit verwendeten Standardmetriken der Information-Retrieval-Evaluation eingeführt [34].

2.7.1 Normalized Discounted Cumulative Gain

NDCG@ k [23] bewertet eine Rangliste unter Berücksichtigung sowohl der Relevanzgrade als auch der Rankingpositionen. Die Grundlage bildet der Discounted Cumulative Gain:

$$\text{DCG}@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)} \quad (2.5)$$

wobei rel_i die Relevanzstufe des Dokuments auf Position i bezeichnet. Der Zähler $2^{rel_i} - 1$ gewichtet höhere Relevanzstufen überproportional, sodass hochrelevante Treffer stärker zählen als mehrere gering relevante. Der Logarithmus im Nenner bewirkt eine

positionsabhängige Diskontierung: Relevante Dokumente auf späteren Positionen tragen weniger zum Gesamtwert bei als solche auf frühen Positionen.

Die Normalisierung erfolgt durch Division mit dem idealen DCG (IDCG), der sich ergibt, wenn die Dokumente optimal nach ihrer Relevanz sortiert wären:

$$\text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k} \quad (2.6)$$

$\text{NDCG}@k$ liegt im Intervall $[0, 1]$, wobei 1 eine perfekte Rangliste anzeigt. Die Metrik eignet sich insbesondere für Datensätze mit mehrstufigen Relevanzannotationen, da sie feinere Relevanzunterschiede abbildet.

2.7.2 Recall

$\text{Recall}@k$ [34] misst den Anteil der relevanten Dokumente, die in den ersten k Positionen der Rangliste enthalten sind:

$$\text{Recall}@k = \frac{|\{d \in \text{Top-}k \mid d \text{ relevant}\}|}{|\{d \in \text{Korpus} \mid d \text{ relevant}\}|} \quad (2.7)$$

Die Metrik quantifiziert die Vollständigkeit der Suche. $\text{Recall}@10$ ist für Endnutzer-Systeme relevant, in denen nur wenige Ergebnisse angezeigt werden. $\text{Recall}@100$ ist für zweistufige Systeme relevant, in denen eine schnelle erste Stufe eine Kandidatenmenge erzeugt, die von einer präziseren zweiten Stufe nachgerankt wird [37].

2.7.3 Mean Reciprocal Rank

MRR [13] bewertet, an welcher Position das erste relevante Dokument in der Rangliste erscheint:

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q} \quad (2.8)$$

wobei rank_q die Position des ersten relevanten Dokuments für Anfrage q bezeichnet. MRR ist besonders aussagekräftig für Anwendungen, in denen der Nutzer primär das erste relevante Ergebnis sucht, etwa bei Frage-Antwort-Systemen [13, 51].

2.7.4 Zusammenspiel der Metriken

Die drei Metriken erfassen komplementäre Aspekte der Suchqualität. $\text{NDCG}@k$ bewertet die Gesamtqualität der Rangliste unter Berücksichtigung von Relevanzgraden und Positionen. $\text{Recall}@k$ misst Vollständigkeit unabhängig von der internen Reihenfolge der relevanten Treffer. MRR fokussiert auf den Einzelfall des ersten relevanten Treffers.

Für die vorliegende Arbeit dient $\text{NDCG}@10$ als primäre Metrik, da sie als Standardmaß im BEIR-Benchmark [48] etabliert ist und mehrstufige Relevanzurteile korrekt verarbeitet. Recall und MRR ergänzen die Analyse, indem sie spezifische Fehlermodi sichtbar machen: Ein System mit hohem NDCG, aber niedrigem Recall bei kleinem k ist für Endnutzer möglicherweise ungeeignet, obwohl die Gesamtrangliste gut ist.

2.8 Anwendungskontext: Praktischer Einsatz vektorbasierter Suche

Der praktische Anwendungskontext dieser Arbeit ist der produktive Einsatz vektorbasierter Suche unter realen Speicher-, Qualitäts- und Kostenrestriktionen. Ein typisches Beispiel sind KI-Agenten, die zur Laufzeit über eine einheitliche Schnittstelle auf externe Retrieval-Werkzeuge zugreifen [44, 56, 32]; ebenso relevant sind lokale Suche auf Endgeräten und kostenoptimierte Retrieval-Dienste in der Cloud. Für die vorliegende Arbeit ist dieser Kontext in zweierlei Hinsicht relevant. Erstens beruht die produktive Nutzung auf einer sauberen Trennung zwischen der anfragenden Komponente und dem Retrieval-System, das Ergebnisse liefert. Dadurch lassen sich verschiedene Retrieval-Backends, ob Float, quantisiert oder binär, austauschen, ohne die aufrufende Anwendung zu modifizieren, was Qualitätsunterschiede eindeutig auf das Backend zurückführbar macht. Genau diese Entkopplung bildet das methodische Fundament der Arbeit, da sie den Vergleich der Kompressionsverfahren bei sonst identischen Bedingungen erlaubt. Zweitens stellt der reale Einsatz konkrete Speicher- und Kostenanforderungen, die sich je nach Umgebung stark unterscheiden: von lokalen Beständen auf Endgeräten mit engem Speicherbudget über kostenempfindliche Cloud-Dienste bis hin zu hochpräzisen Enterprise-Systemen. Diese Bandbreite motiviert die untersuchte Kompression unmittelbar und wird in Phase 6 (Abschnitt 3.10) zu drei konkreten Einsatzszenarien verdichtet. Die konkrete Einbettung in eine Agenten- oder Anwendungsschnittstelle ist dabei selbst nicht Gegenstand der Experimente.

3 Methodik

3.1 Experimenteller Rahmen

Zur Beantwortung der in Abschnitt 1.4 formulierten Forschungsfrage wird ein kontrolliertes Experiment durchgeführt. Aus dem Float-Raum werden zunächst komprimierte Repräsentationen abgeleitet, variiert entlang zweier Achsen (Bittiefe und Dimensionszahl); die Erzeugung dieser Repräsentationen ist in Abschnitt 3.4 beschrieben. Die eigentliche Untersuchung gliedert sich anschließend in sechs Phasen:

1. **Charakterisierung:** Der Float-Raum wird statistisch und geometrisch analysiert, um Erwartungen an den Quantisierungsverlust zu formulieren.
2. **Distanz- und Fehleranalyse:** Die Erhaltung der globalen paarweisen Distanzstruktur unter Quantisierung wird gemessen.
3. **Nachbarschaftserhaltung:** Die Erhaltung der lokalen Nachbarschaftsstruktur wird über Neighborhood Overlap und Trustworthiness bewertet.
4. **Retrieval-Evaluation:** Die tatsächliche Auswirkung auf die Suchqualität wird mit etablierten Information-Retrieval-Metriken quantifiziert.
5. **Praktische Evaluation:** Speicherbedarf und Suchgeschwindigkeit werden gemessen, um Qualität und Effizienz gegeneinander abzuwägen.
6. **Anwendungs- und Skalierungsanalyse:** Die gemessenen Qualitäts- und Effizienzwerte werden auf konkrete Einsatzszenarien, Indexgrößen und Betriebskosten übertragen.

Diese Reihenfolge ist methodisch notwendig. Würde direkt mit der Retrieval-Evaluation begonnen, ließe sich zwar feststellen, *dass* ein Verfahren schlechter abschneidet, aber nicht *warum*. Die vorgeschaltete Raumanalyse ermöglicht es, Qualitätsverluste kausal auf geometrische Eigenschaften des Ausgangsraums zurückzuführen. Der vollständige Quellcode des Experiments, einschließlich aller Analyseskripte, Quantisierungsverfahren und Evaluationspipelines, ist öffentlich verfügbar.¹

¹<https://github.com/JeskoFoerster/BitEmb>

3.2 Datensatz

Als Evaluationsgrundlage dient der BEIR-Benchmark [48]. BEIR stellt annotierte Relevanzurteile zwischen Anfragen und Dokumenten bereit und deckt heterogene Domänen ab. Die Wahl fällt auf BEIR aus zwei Gründen. Erstens liegen manuell geprüfte Relevanzannotationen vor, sodass Retrieval-Qualität objektiv messbar ist und nicht auf synthetischen Ähnlichkeitsmaßen beruht. Zweitens erlaubt die Domänenvielfalt eine Aussage über die Generalisierbarkeit der Ergebnisse. Innerhalb von BEIR werden drei Subsets verwendet:

- **SciFact**, ein Datensatz zur wissenschaftlichen Faktenverifikation mit kurzen Dokumenten und binären Relevanzurteilen.
- **FiQA**, ein Frage-Antwort-Datensatz aus dem Finanzbereich mit längeren, argumentativen Texten.
- **TREC-COVID**, ein biomedizinischer Suchdatensatz mit umfangreichen Dokumenten und mehrstufigen Relevanzannotationen.

Die Auswahl folgt dem Prinzip maximaler Variation entlang dreier Achsen, nämlich Dokumentlänge (kurz, mittel, lang), Fachsprache (Naturwissenschaft, Finanz, Biomedizin) und Annotationsschema (binär vs. mehrstufig). Damit kann systematisch geprüft werden, ob Quantisierungseffekte domänenspezifisch auftreten oder sich über verschiedene Texttypen hinweg konsistent verhalten. Würde nur ein einzelnes Subset verwendet, wäre unklar, ob die Ergebnisse an der Methode oder an den Daten liegen.

3.3 Embedding-Modell

Für die Erzeugung aller Vektorrepräsentationen wird ausschließlich das Modell **bge-large-en-v1.5** [54] verwendet. Die Beschränkung auf ein einziges Modell ist methodisch notwendig, um Konfundierung zu vermeiden. Unterschiedliche Modelle erzeugen Vektorräume mit unterschiedlicher Geometrie, sodass ein Vergleich der Quantisierungsergebnisse nicht mehr eindeutig auf die Kompressionsverfahren zurückführbar wäre. Die Wahl fällt auf **bge-large-en-v1.5** aus folgenden Gründen:

- Das Modell erreicht zum Zeitpunkt der Modellauswahl konsistent hohe Werte auf dem MTEB-Leaderboard [36] und gilt als etablierter Referenzpunkt in der Embedding-Literatur.
- Es erzeugt Vektoren mit 1024 Dimensionen. Damit ist der Ausgangsraum hochdimensional genug für eine aussagekräftige Quantisierungsanalyse, bleibt aber

für exakte Brute-Force-Messungen auf kontrollierten BEIR-Ausschnitten praktikabel.

- Als Open-Source-Modell ermöglicht es vollständige Reproduzierbarkeit.

Eine naheliegende Alternative wäre `nomic-embed-text-v1.5` [39], das durch Matryoshka-Training bereits auf variable Dimensionsgrößen und binäre Ausgabe optimiert ist. Genau diese Optimierung macht es für die vorliegende Untersuchung jedoch weniger geeignet. Da das Ziel ist, den Informationsverlust bei nachträglicher Quantisierung zu messen, würde ein bereits auf Kompression trainiertes Modell den Effekt der Quantisierungsverfahren selbst verschleiern. Die Ergebnisse wären nicht mehr auf das Kompressionsverfahren, sondern auf die Trainingsoptimierung des Modells zurückführbar. Ein Vergleich mit speziell auf Binarisierung trainierten Modellen bleibt weiterführenden Arbeiten vorbehalten. Die Beschränkung auf ein einzelnes Modell impliziert, dass die Ergebnisse zunächst modellspezifisch sind. Andere Architekturen erzeugen Vektorräume mit abweichender Varianzstruktur und Dimensionskorrelation, sodass Quantisierungseffekte dort quantitativ anders ausfallen können. Die qualitative Analyse (welche Raumeigenschaften Quantisierung begünstigen oder erschweren) ist davon weniger betroffen, da sie auf allgemeinen geometrischen Prinzipien beruht.

3.4 Erzeugung der Repräsentationsräume

Ausgangspunkt ist der vollständige Float-Raum. Für jedes Dokument im Datensatz wird ein Embedding-Vektor $\mathbf{x} \in \mathbb{R}^{1024}$ berechnet. Die Berechnung erfolgt über die Bibliothek Sentence-Transformers [42] mit aktivierter L2-Normalisierung (Option `normalize_embeddings`), sodass jeder Ausgabevektor auf Einheitslänge skaliert wird; Phase 1 verifiziert diese Eigenschaft nachträglich für jeden einzelnen Vektor (vgl. Abschnitt 3.5). Die resultierende Matrix $\mathbf{X} \in \mathbb{R}^{N \times 1024}$ bildet die Ground Truth, gegen die alle komprimierten Repräsentationen evaluiert werden. Aus diesem Float-Raum werden mehrere komprimierte Varianten *nachträglich* abgeleitet. Dieser Ansatz, also die nachträgliche Kompression eines bestehenden Float-Raums, ist eine bewusste methodische Entscheidung gegenüber der Alternative, ein Modell zu verwenden, das direkt binäre oder quantisierte Ausgaben erzeugt (etwa durch Matryoshka-Training oder spezialisierte Binary-Embedding-Modelle). Die nachträgliche Kompression ist für die Forschungsfrage aus drei Gründen notwendig:

1. Sie ermöglicht einen kontrollierten Vorher-Nachher-Vergleich. Ohne einen Float-Referenzraum kann der Informationsverlust nicht gemessen werden, weil kein unkomprimierter Zustand existiert, gegen den evaluiert werden könnte.

2. Sie isoliert genau eine unabhängige Variable, nämlich das Kompressionsverfahren. Bei end-to-end trainierten Modellen wären Unterschiede in der Retrieval-Qualität nicht eindeutig auf die Kompression zurückführbar, sondern könnten ebenso an Architektur, Trainingsdaten oder Verlustfunktion liegen.
3. Sie bildet das praxisrelevante Szenario ab, in dem bestehende Float-Embeddings nachträglich komprimiert werden sollen, ohne das zugrunde liegende Modell austauschen zu müssen.

Die Wahl der komprimierten Varianten bildet verschiedene Punkte eines Kompressions-Qualität-Spektrums ab. Die naive Binarisierung steht für maximale Kompression bei minimaler Methodik. TurboQuant 1-Bit und 2-Bit repräsentieren sehr starke Kompression mit vorgeschalteter Rotation, während TurboQuant 4-Bit und 8-Bit moderatere Kompressionsstufen mit höherer erwarteter Qualitätserhaltung abdecken. Zusätzlich werden naive 8-Bit-, 4-Bit-, 2-Bit- und 1-Bit-Varianten ohne Rotation als Ablationen verwendet, um den Effekt der Rotation vom Effekt der reinen Bitreduktion zu trennen. Float16 dient als niedrigpräzise Float-Referenz zwischen Float32 und 8-Bit-Quantisierung.

3.4.1 Binäre Repräsentationen

Die naive Binarisierung wendet den in Kapitel 2 definierten Vorzeichenschwellwert an (Gleichung 2.2). Jede Dimension wird auf ein einzelnes Bit reduziert, was pro Dimension einem Kompressionsfaktor von 32 gegenüber Float32 entspricht. Die Ähnlichkeit zweier binärer Vektoren wird über die Hamming-Distanz bestimmt. Dieses Verfahren dient als untere Basislinie. Es ist das einfachste denkbare Quantisierungsverfahren, das keine Kenntnis über die Datenverteilung nutzt. Jede informiertere Methode sollte besser abschneiden. Falls die naive Binarisierung bereits gute Ergebnisse liefert, deutet dies darauf hin, dass der Float-Raum Eigenschaften besitzt, die Quantisierung inhärent begünstigen, etwa eine Konzentration der Information im Vorzeichen. Zusätzlich wird eine rotierte 1-Bit-Variante betrachtet. Sie entspricht der TurboQuant-Idee im Extremfall: Der Float-Raum wird zunächst orthogonal rotiert und anschließend über das Vorzeichen binarisiert. Dadurch lässt sich messen, ob Rotation auch bei maximaler Kompression noch Retrieval-Qualität erhält.

3.4.2 TurboQuant-Repräsentation

Die TurboQuant-Varianten implementieren das Kernprinzip nach [57]: eine datenunabhängige orthogonale Rotation mit anschließender uniformer skalarer Quantisierung. Die konkrete Umsetzung verwendet eine zufällige Rotationsmatrix aus $SO(d)$ anstelle

der im Paper beschriebenen Randomized Hadamard Transform und quantisiert anschließend jede Dimension per Min-Max-Normalisierung auf 2^b gleichverteilte Stufen. Diese vereinfachte Implementierung erhält die konzeptionell entscheidende Eigenschaft – die Varianzumverteilung durch Rotation vor der uniformen Quantisierung – verzichtet aber auf die laufzeitoptimierte Hadamard-basierte Rotation und die theoretisch optimierten Quantisierungsgrenzen des vollständigen Verfahrens. Für die Forschungsfrage ist diese Vereinfachung unproblematisch, da nicht die Laufzeit der Rotationsphase, sondern die resultierende Retrieval-Qualität der komprimierten Repräsentationen untersucht wird. Eine eigene Implementierung wird aus zwei Gründen verwendet. Erstens erlaubt sie die volle Kontrolle über die Verarbeitungskette, insbesondere das Zusammenspiel von Rotation, Dimensionsreduktion und einer über alle Repräsentationen einheitlichen Distanzberechnung, was für den fairen Vergleich der Verfahren notwendig ist. Zweitens ist die verwendete $SO(d)$ -Rotation keine Verschlechterung gegenüber dem Paper: Die Randomized Hadamard Transform ist eine schnelle Approximation ($\mathcal{O}(d \log d)$) einer gleichverteilten zufälligen Rotation, während $SO(d)$ genau diese Rotation exakt realisiert, allerdings zum Preis einer höheren Rechenzeit, die hier nicht relevant ist. Die funktionale Gleichwertigkeit der beiden Rotationen wurde in einem separaten Verifikationsexperiment bestätigt. Da beide Rotationen zufällige orthogonale Transformationen derselben (Haar-)Verteilung sind, folgt ihre Gleichwertigkeit bereits theoretisch und damit datenunabhängig; das Experiment dient der empirischen Bestätigung. Es verwendet synthetische Embeddings mit ungleich über die Achsen verteilter Varianz, da nur in diesem Regime die Rotation überhaupt wirkt und ein aussagekräftiger Vergleich möglich ist. Dort erhält die $SO(d)$ -Rotation Normen und paarweise Distanzen exakt (Fehler in der Größenordnung der Maschinengenauigkeit), gleicht die Varianz über die Dimensionen ebenso aus wie die Hadamard-Variante und erreicht nach uniformer Quantisierung dieselbe Distanzverzerrung und dieselbe Nachbarschaftserhaltung; beide rotierten Verfahren übertreffen die unrotierte naive Quantisierung deutlich. Die Gleichwertigkeit betrifft damit die Rotationskomponente bei sonst identischer uniformer Quantisierung; auf die im Paper zusätzlich optimierten Quantisierungsgrenzen wird bewusst verzichtet. Für die untersuchte Fragestellung ist die eigene Implementierung damit als hinreichend gleichwertig anzusehen. Das Verfahren wird in mehreren Bitstiefen evaluiert:

- **TurboQuant 8-Bit** (256 Quantisierungsstufen pro Dimension, Kompressionsfaktor 4 gegenüber Float32).
- **TurboQuant 4-Bit** (16 Quantisierungsstufen pro Dimension, Kompressionsfaktor 8 gegenüber Float32).
- **TurboQuant 2-Bit** (4 Quantisierungsstufen pro Dimension, Kompressionsfaktor 16 gegenüber Float32).

- **TurboQuant 1-Bit** (Vorzeichenquantisierung nach Rotation, Kompressionsfaktor 32 gegenüber Float32).

Die TurboQuant-8-Bit-Variante und die Float16-Referenz dienen dazu, den Übergang von nahezu verlustfreier niedriger Präzision zu aggressiver Quantisierung sichtbar zu machen. Die 4-Bit-Variante schließt die Lücke zwischen 8-Bit und 2-Bit. Sie ermöglicht die Unterscheidung, ob beobachtete Qualitätsverluste primär durch die geringe Stufenzahl oder durch die Bitreduktion an sich entstehen. Der Vergleich zwischen naiven und rotierten Varianten gleicher Bitstiefe isoliert den Effekt der TurboQuant-Rotation. TurboQuant ist aus mehreren Gründen als Vergleichsmethode geeignet:

- Es repräsentiert den aktuellen Stand der Forschung für datenunabhängige Vektorquantisierung.
- Im Gegensatz zur naiven Binarisierung nutzt es eine vorgeschaltete Rotation, die eine ungleiche Varianzverteilung über die Achsen ausgleicht (vgl. Kapitel 2).
- Der Kompressionsfaktor liegt zwischen Float und Binär, was eine Einordnung entlang des Kompression-Qualität-Spektrums ermöglicht.

Der methodische Mehrwert des Vergleichs liegt darin, dass sich mehrere Effekte trennen lassen. Der Effekt der Bitreduktion wird entlang der Stufen 16-Bit, 8-Bit, 4-Bit, 2-Bit und 1-Bit sichtbar. Der Effekt der Rotation wird jeweils durch den Vergleich zwischen naiver Quantisierung und TurboQuant bei gleicher Bitstiefe isoliert. Der Effekt extremer Kompression zeigt sich besonders im Vergleich von 2-Bit- und 1-Bit-Varianten.

3.4.3 Dimensionsreduktion als zusätzlicher Kompressionsparameter

Die bisherigen Repräsentationen variieren zunächst die Bittiefe pro Dimension bei voller Modell-Dimensionalität ($d = 1024$). Dies bildet jedoch nur eine Achse des Kompressionsraums ab. Die zweite Achse, die Anzahl der verwendeten Dimensionen, wird ergänzend untersucht, um den vollständigen Trade-off zwischen Kompression und Qualität zu erfassen. Konkret werden alle Quantisierungsverfahren zusätzlich auf dimensionsreduzierte Vektoren angewandt. Die Reduktion erfolgt über eine PCA-Projektion auf die Hauptkomponenten mit der höchsten erklärten Varianz. Evaluiert werden die PCA-Dimensionen $d \in \{64, 128, 256, 384, 512, 768, 1024\}$. Daraus ergibt sich eine zweidimensionale Versuchsmatrix:

- Repräsentationen in den erweiterten Phasen: Float32, Float16, naive und rotierte 8-Bit-, 4-Bit-, 2-Bit- und 1-Bit-Varianten

- Dimensionszahl: 64, 128, 256, 384, 512, 768, 1024

Die konkrete Metrik hängt von der jeweiligen Phase ab: Phase 2 misst Distanzkorrelation und Verzerrung, Phase 3 Neighborhood Overlap und Trustworthiness, Phase 4 Retrieval-Metriken wie NDCG@10 und Phase 5 Speicher- sowie Laufzeitmetriken. Die Ergebnisse werden als Pareto-Front dargestellt, die den optimalen Trade-off zwischen Gesamtkompression (Bits pro Vektor) und Retrieval-Qualität zeigt. Der Gesamtspeicherbedarf einer Kombination beträgt $d \times b$ Bit pro Vektor, wobei b die Bittiefe ist. Damit reicht das Spektrum von $1024 \times 32 = 32\,768$ Bit (Float32, volle Dimension) bis $64 \times 1 = 64$ Bit (1-Bit, minimale Dimension), ein Kompressionsfaktor von bis zu 512. Die Wahl von PCA als Reduktionsverfahren ist methodisch konsistent mit dem Gesamtansatz der Arbeit. Sie ist aufgabenunabhängig in dem Sinne, dass keine überwachte oder aufgabenspezifische Optimierung stattfindet; die Hauptkomponenten werden zwar aus den Daten geschätzt, aber ohne Kenntnis der Retrieval-Aufgabe oder Relevanzurteile. Sie ist orthogonal zur anschließenden Quantisierung, sodass beide Effekte weiterhin separierbar bleiben. Darüber hinaus liefert die in Phase 1 ohnehin durchgeführte PCA-Analyse der erklärten Varianz direkt eine Erwartung darüber, ab welcher Präfixlänge substantielle Information verloren geht.

3.5 Phase 1: Charakterisierung des Float-Raums

Bevor Quantisierung stattfindet, wird der Float-Raum analysiert. Diese Phase dient nicht der Bewertung der Kompressionsverfahren, sondern der Formulierung von Erwartungen. Aus den Eigenschaften des Raums lässt sich vorhersagen, wo Quantisierung gut funktionieren wird und wo nicht. Diese Vorhersagen werden in den späteren Phasen empirisch überprüft.

3.5.1 Normverteilung

Für jeden einzelnen Vektor der vollständigen Korpora (5 183 Dokumente bei SciFact, 57 638 bei FiQA und 171 332 bei TREC-COVID) wird die L2-Norm $\|\mathbf{x}\|_2$ berechnet und deskriptiv zusammengefasst (Mittelwert, Standardabweichung, Variationskoeffizient, Bereich); eine Stichprobenziehung findet in dieser Phase nicht statt. Die Normverteilung beantwortet eine konkrete Frage, nämlich ob die Vektoren annähernd auf einer Hypersphäre liegen oder ob signifikante Normunterschiede existieren. Da die Embeddings bereits bei der Erzeugung L2-normiert werden (vgl. Abschnitt 3.4), dient die Analyse der Verifikation dieser Eigenschaft und nicht einer nachträglichen Normalisierung. Diese Analyse ist für die Binarisierung direkt relevant. Die Vorzeichenfunktion kodiert jede Komponente allein anhand ihres Vorzeichens, sodass die L2-Norm nicht in

den Bitvektor eingeht (vgl. Abschnitt 2.3). Wenn die Normen stark variieren, geht bei der Binarisierung Information verloren, die nicht im Vorzeichen kodiert ist. Wenn die Normen hingegen nahezu konstant sind, ist der Informationsgehalt der Norm gering und ihr Verlust unproblematisch.

3.5.2 Dimensionsstatistiken

Für jede der 1024 Dimensionen werden vier Kennzahlen berechnet:

- **Mittelwert und Standardabweichung** beschreiben Lage und Streuung.
- **Schiefe (Skewness)** zeigt, ob die Verteilung symmetrisch um Null liegt. Bei starker Schiefe liegt der Vorzeichenschwellwert $x_i = 0$ der Binarisierung nicht am informationstheoretisch optimalen Punkt, da eine Seite der Verteilung mehr Masse trägt und das resultierende Bit damit weniger Information kodiert.
- **Kurtosis** misst die Schwere der Verteilungsränder. Hohe Kurtosis deutet auf Ausreißer hin, die bei Quantisierung auf wenige Stufen besonders stark verzerrt werden, weil die extremen Werte auf dieselben Quantisierungsgrenzen abgebildet werden wie die häufigen.

Diese Kennzahlen sind nicht Selbstzweck, sondern liefern spezifische Vorhersagen. Schiefe und Kurtosis werden als standardisierte dritte und vierte Momente berechnet, wobei die Kurtosis als Exzess-Kurtosis nach Fisher angegeben wird, sodass die Normalverteilung in beiden Kennzahlen den Wert 0 annimmt [25]. Als Schwellwerte für problematische Dimensionen werden $|\text{skew}| > 1,0$ und $\text{kurtosis} > 3,0$ festgelegt. Der Schiefe-Schwellwert folgt der verbreiteten Faustregel, nach der Verteilungen mit $|\text{skew}| > 1$ als stark schief gelten [9]; der Kurtosis-Schwellwert markiert Verteilungen, deren Ränder deutlich schwerer sind als die der Normalverteilung, da die Kurtosis vor allem das Gewicht der Verteilungsränder misst [52]. Dimensionen mit hoher Schiefe sollten bei der Binarisierung mehr Fehler verursachen als symmetrische Dimensionen. Dimensionen mit hoher Kurtosis sollten bei TurboQuant (2 Bit, also 4 Stufen) mehr Quantisierungsfehler zeigen. Ob diese Vorhersagen zutreffen, wird in Phase 2 überprüft.

3.5.3 Intrinsische Dimensionalität

Die intrinsische Dimensionalität (vgl. Abschnitt 2.6) wird über die beiden dort beschriebenen komplementären Verfahren geschätzt: TwoNN [15] für die lokale Mannigfaltigkeitsstruktur und die PCA-basierte Schätzung über die Anzahl der Hauptkomponenten für 95% erklärte Varianz für die globale lineare Dimensionalität. Das Ergebnis liefert

eine konkrete Vorhersage für die Dimensionsreduktion in den späteren Phasen. Liegt die intrinsische Dimensionalität deutlich unter der nominalen ($d_{\text{int}} \ll 1024$), sollte eine PCA-Reduktion auf $d \geq d_{\text{int}}$ nur geringen Qualitätsverlust verursachen, während eine Reduktion auf $d < d_{\text{int}}$ substantielle Information zerstört. Diese Schwelle wird in Phase 2 und Phase 3 empirisch überprüft. Die Kombination beider Schätzer erlaubt eine Aussage über die Robustheit des Ergebnisses. Stimmen beide überein, ist die geschätzte Redundanz belastbar. Divergieren sie, deutet dies auf nichtlineare Struktur hin, die PCA nicht erkennt.

3.6 Phase 2: Distanz- und Fehleranalyse

Diese Phase beantwortet die Kernfrage, ob die geometrischen Beziehungen zwischen Dokumenten unter Quantisierung erhalten bleiben.

3.6.1 Paarweise Distanzkorrelation

Aus dem Dokumentenkörper werden 10 000 Paare gleichverteilt zufällig gezogen. Für jedes Paar werden Distanzwerte im Float-Raum und in allen neun komprimierten Repräsentationen berechnet (Float16, naive und rotierte 8-Bit-, 4-Bit-, 2-Bit- und 1-Bit-Varianten). Die Pairstichprobe ist für alle Repräsentationen identisch, sodass Unterschiede ausschließlich auf das Kompressionsverfahren zurückführbar sind. Da die Repräsentationen unterschiedliche native Distanzmetriken verwenden – Cosine-Distanz für Float32 und Float16, euklidische Distanz auf dequantisierten Vektoren für die Multi-Bit-Varianten und Hamming-Distanz für die 1-Bit-Varianten – werden alle Distanzvektoren vor dem Vergleich per Min-Max-Normalisierung auf das Intervall $[0, 1]$ abgebildet. Für normierte Eingabevektoren sind Cosine-Distanz und euklidische Distanz ohnehin linear proportional, sodass die Normalisierung keinen Informationsverlust verursacht. Die Stichprobengröße von 10 000 Paaren ist so gewählt, dass das 95%-Konfidenzintervall für den Pearson-Korrelationskoeffizienten bei $r = 0,9$ eine Breite von weniger als $\pm 0,005$ besitzt. Damit sind selbst kleine Unterschiede zwischen den Verfahren statistisch auflösbar. Die Distanzerhaltung wird über vier Metriken quantifiziert, die zwei komplementäre Perspektiven abdecken: relative Strukturhaltung (Pearson, Spearman) und absolute Verzerrung (MAE, RMSE).

Pearson r – linearer Zusammenhang. Für jedes der 10 000 Paare liegen zwei Distanzwerte vor: die Float-Distanz x_i und die komprimierte Distanz y_i . Der Pearson-

Korrelationskoeffizient berechnet sich als:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \cdot \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (3.1)$$

Dabei ist $\bar{x}$ der Mittelwert der Float-Distanzen und $\bar{y}$ der Mittelwert der komprimierten Distanzen über alle $n = 10\,000$ Paare. Der Wert liegt zwischen -1 und $+1$. Ein $r = 0,975$ bedeutet: Wenn man die Float-Distanz kennt, kann man die komprimierte Distanz bis auf geringe Streuung durch eine Gerade vorhersagen. Quantitativ erklärt $r^2 = 0,95$ der komprimierten Distanzvarianz durch die Float-Distanz, 5 % sind Quantisierungsrauschen. Geometrisch entspricht $r = 1$ einer perfekten Geraden in einem Streudiagramm (Float-Distanz auf der x-Achse, komprimierte Distanz auf der y-Achse); mit sinkendem r wird die Punktwolke breiter.

Spearman ρ – Rangerhaltung. Spearman ρ berechnet sich identisch zu Pearson r , operiert aber nicht auf den Distanzwerten selbst, sondern auf deren *Rängen*. Zuerst werden die 10 000 Float-Distanzen aufsteigend sortiert und durch ihre Rangposition ersetzt ($1, 2, \dots, n$); dasselbe geschieht mit den komprimierten Distanzen. Anschließend wird der Pearson-Koeffizient auf diesen Rängen berechnet. Ein $\rho = 0,97$ bedeutet: Wenn Paar A im Float-Raum näher ist als Paar B, dann ist es mit hoher Wahrscheinlichkeit auch im komprimierten Raum näher. Die genauen Abstandswerte können dabei beliebig verzerrt sein – solange die *Reihenfolge* stimmt, bleibt ρ hoch. Für Retrieval, das Ergebnisse nach Ähnlichkeit sortiert, ist Spearman ρ daher die relevantere Metrik (vgl. Abschnitt 2.2, Rangerhaltung). Die Kombination beider Maße erlaubt eine differenzierte Aussage. Sind beide hoch, ist die Quantisierung strukturtreu. Ist nur Spearman hoch, eignet sich die Repräsentation für Ranking, aber nicht für distanzbasierte Schwellwertentscheidungen. Sind beide niedrig, ist die semantische Struktur substantiell beschädigt.

3.6.2 Absolute Verzerrung (Distortion)

Zusätzlich zur relativen Struktur wird die absolute Verzerrung gemessen. Für jedes Paar i wird der Fehler $e_i = |x_i - y_i|$ auf den bereits auf $[0, 1]$ normierten Distanzen berechnet. Daraus ergeben sich zwei Aggregatmaße:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |x_i - y_i| \quad (3.2)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - y_i)^2} \quad (3.3)$$

MAE gibt die durchschnittliche absolute Abweichung an. Ein MAE von 0,05 bedeutet: Die komprimierte Distanz weicht im Mittel um 5 Prozentpunkte auf der normierten $[0, 1]$ -Skala vom Float-Wert ab. RMSE gewichtet große Fehler stärker, weil die Quadrierung extreme Abweichungen überproportional bestraft. Wenn $\text{RMSE} \approx \text{MAE}$, sind die Fehler gleichmäßig verteilt. Wenn $\text{RMSE} \gg \text{MAE}$, gibt es einzelne Paare mit besonders hoher Verzerrung. Während Pearson und Spearman die *relative* Struktur bewerten, quantifizieren MAE und RMSE die *absolute* Verzerrung. Beide Perspektiven sind notwendig. Ein Verfahren mit hoher Korrelation, aber hohem MAE transformiert die Distanzen zwar monoton korrekt, verschiebt aber das gesamte Distanzniveau. Für Schwellwert-basierte Systeme (z. B. “zeige nur Ergebnisse mit Ähnlichkeit $> 0,8$ ”) wäre dies problematisch.

3.7 Phase 3: Nachbarschaftserhaltung

Die Distanzkorrelation aus Phase 2 betrachtet globale Paare. Für Retrieval ist jedoch die *lokale* Struktur entscheidend. Es kommt nicht darauf an, ob weit entfernte Dokumente korrekt als weit entfernt erkannt werden, sondern ob die nächsten Nachbarn eines Dokuments dieselben bleiben. Phase 3 operiert daher auf derselben zweidimensionalen Versuchsmatrix (neun komprimierte Repräsentationen $\times$ sieben PCA-Dimensionen), bewertet aber nicht paarweise Distanzerhaltung, sondern die Erhaltung lokaler Nachbarschaftsstrukturen.

3.7.1 Ablauf

Für jede Kombination aus PCA-Dimension und Repräsentation wird der folgende Ablauf durchgeführt:

1. **PCA-Projektion.** Die Float32-Embeddings werden auf die gewählte Dimensionszahl d projiziert (entfällt für $d = 1024$).
2. **k-NN-Bestimmung im Float-Raum (Ground Truth).** Für jedes Dokument werden die k nächsten Nachbarn über Cosine-Similarity berechnet. Da die Embeddings L2-normiert sind, reduziert sich dies auf maximales Skalarprodukt. Die resultierende Nachbarschaftsmenge $N_k^{\text{float}}(i)$ bildet die Referenz.
3. **k-NN-Bestimmung in jedem komprimierten Raum.** Die Nachbarn werden mit der jeweiligen nativen Distanzmetrik der Repräsentation bestimmt:
 - **Float16:** Cosine-Similarity auf Float16-gecasteten Vektoren.

- **Naive und TurboQuant 2-Bit, 4-Bit, 8-Bit:** Euklidische Distanz auf dequantisierten Vektoren. Die Quantisierung und anschließende Dequantisierung simuliert den Informationsverlust; die Nachbarsuche erfolgt dann auf den rekonstruierten Werten.
- **Naive und TurboQuant 1-Bit:** Hamming-Distanz auf bitgepackten Binärvektoren.

4. **Berechnung von Overlap und Trustworthiness.** Für jedes $k \in \{5, 10, 20\}$ werden die Metriken berechnet.

Die Suche erfolgt in allen Räumen exakt (Brute-Force), konsistent mit dem Vorgehen in Phase 2 und 4. Da die k-NN-Berechnung für jedes Dokument Distanzen zu allen anderen erfordert, skaliert der Aufwand quadratisch mit der Korpusgröße ($\mathcal{O}(N^2 \cdot d)$). Im Gegensatz zu Phase 2, die unabhängig von N nur 10 000 zufällige Paare zieht, wird das Korpus daher auf maximal 5 000 Dokumente begrenzt. Die Auswahl erfolgt als gleichverteilte Zufallsstichprobe mit festem Seed, sodass die Ergebnisse reproduzierbar bleiben. Diese Stichprobengröße ist ausreichend, um stabile Mittelwerte für Overlap und Trustworthiness zu erhalten, da beide Metriken über alle N Dokumente gemittelt werden und die Varianz des Mittelwerts mit $1/N$ abnimmt.

3.7.2 Distanzmetriken in den komprimierten Räumen

Die Wahl der Distanzmetrik pro Repräsentation spiegelt das wider, was ein reales Retrieval-System zur Suchzeit verwenden würde. Float-Vektoren werden über Cosine-Similarity verglichen, Multi-Bit-Quantisierungen über euklidische Distanz auf den dequantisierten Werten, und binäre Repräsentationen über Hamming-Distanz. Die euklidische Distanz auf dequantisierten Vektoren ist konsistent mit der asymmetrischen Distanzberechnung der späteren Retrieval-Phase. Für normierte Eingabevektoren gilt $\|a - b\|^2 = 2 - 2 \cos(a, b)$, sodass die euklidische Distanz nach Dequantisierung dasselbe Ranking erzeugt wie Cosine-Distanz, sofern die Dequantisierung die Normierung nicht substantiell verzerrt. Damit sind die Ergebnisse zwischen Phase 3 und Phase 4 methodisch vergleichbar.

3.7.3 Neighborhood Overlap

Der Overlap misst den Anteil der Nachbarn, die im Float-Raum und im komprimierten Raum übereinstimmen:

$$\text{Overlap}(i) = \frac{|N_k^{\text{float}}(i) \cap N_k^{\text{quant}}(i)|}{k} \quad (3.4)$$

Dieser Wert wird über alle Dokumente gemittelt und als Funktion von k berichtet. Die Variation von k zeigt, ob Quantisierung vor allem die engste Nachbarschaft ($k = 5$) oder auch breitere lokale Strukturen ($k = 20$) verändert. Für Retrieval-Systeme, die typischerweise die Top-10 anzeigen, ist $k = 10$ der praxisrelevanteste Wert. Als Referenzwert dient der erwartete Overlap bei zufälligem Ranking: $\mathbb{E}[\text{Overlap}] = k/N$, wobei N die Korpusgröße ist. Für die verwendeten Datensätze liegt dieser Zufalls-Overlap unter 0,02, sodass bereits moderate Overlap-Werte eine deutliche Strukturierung anzeigen.

3.7.4 Trustworthiness

Neighborhood Overlap ist symmetrisch und unterscheidet nicht zwischen zwei verschiedenen Fehlertypen. Trustworthiness [50, 49] differenziert und misst, ob im komprimierten Raum *falsche* Nachbarn eingeführt werden, also Dokumente, die im Float-Raum weit entfernt sind, im komprimierten Raum aber nahe erscheinen. Formal:

$$T(k) = 1 - \frac{2}{Nk(2N - 3k - 1)} \sum_{i=1}^N \sum_{j \in U_k(i)} (r(i, j) - k) \quad (3.5)$$

wobei $U_k(i)$ die Menge der Punkte bezeichnet, die im komprimierten Raum unter den k Nachbarn von i sind, aber im Float-Raum nicht, und $r(i, j)$ der Rang von j bezüglich i im Float-Raum ist. Der Rang $r(i, j)$ wird dabei über die Cosine-Similarity im Float-Raum bestimmt. Der Strafterm $(r(i, j) - k)$ gewichtet Fehler proportional zur Schwere: Ein Dokument, das im Float-Raum auf Rang 50 steht ($r = 50$), aber im komprimierten Raum unter die Top-5 gelangt, wird stärker bestraft ($50 - 5 = 45$) als eines von Rang 8 ($8 - 5 = 3$). Ein Wert von $T = 1$ bedeutet, dass keine falschen Nachbarn eingeführt werden. Dieser Fehlertyp – das Einführen falscher Nachbarn – ist für Retrieval besonders schädlich, da er irrelevante Suchergebnisse bedeutet, die relevante Treffer aus den Top- k verdrängen. Die Wahl, Trustworthiness statt des Gegenstücks Continuity zu berichten, ist bewusst. Continuity misst, ob echte Nachbarn verloren gehen (Recall-Perspektive). Trustworthiness quantifiziert hingegen direkt die Precision der Nachbarschaft, die im Retrieval-Kontext die nutzerrelevantere Größe ist. Für ein Retrieval-System ist es gravierender, irrelevante Dokumente anzuzeigen, als einzelne relevante Dokumente zu übersehen, solange genügend relevante Treffer verbleiben. Ob Letzteres der Fall ist, überprüft der Recall in Phase 4 direkt.

3.8 Phase 4: Retrieval-Evaluation

Die bisherigen Phasen messen geometrische Strukturhaltung. Phase 4 prüft, ob sich diese geometrischen Beobachtungen in tatsächlicher Suchqualität niederschlagen.

3.8.1 Retrieval-Protokoll

Für jede Query im BEIR-Testset wird eine exakte Suche in allen aktuell untersuchten Repräsentationsräumen durchgeführt. Die Repräsentationsmatrix umfasst:

- **Float32**: Ranking nach Cosine-Similarity. Da die Embeddings normalisiert sind, entspricht dies dem Skalarprodukt.
- **Float16**: Ranking nach Cosine-Similarity auf in Float16 gecasteten Vektoren.
- **Naive 8-Bit, 4-Bit und 2-Bit**: Ranking nach asymmetrischer quantisierter L2-Distanz ohne vorgeschaltete Rotation.
- **TurboQuant 8-Bit, 4-Bit und 2-Bit**: Ranking nach asymmetrischer quantisierter L2-Distanz nach orthogonaler Rotation.
- **Naive 1-Bit**: Ranking nach Hamming-Distanz auf direkt binarisierten Sign-Bits.
- **TurboQuant 1-Bit**: Ranking nach Hamming-Distanz auf rotierten Sign-Bits.

Damit werden zehn Repräsentationsräume verglichen: `float32`, `16bit`, `naive_8bit`, `tq_8bit`, `naive_4bit`, `tq_4bit`, `naive_2bit`, `tq_2bit`, `naive_1bit` und `tq_1bit`. Die Evaluation wird für die PCA-Dimensionen $d \in \{64, 128, 256, 384, 512, 768, 1024\}$ durchgeführt. Die Suche erfolgt exakt (Brute-Force), nicht approximativ. Der Grund ist, dass approximative Verfahren (z. B. HNSW [33]) eigene Fehler einführen, die den Quantisierungseffekt überlagern würden. Das Experiment soll ausschließlich den Effekt der Kompression isolieren, nicht den einer approximativen Suche. Die Konsequenz dieser Entscheidung ist, dass die gemessene Retrieval-Qualität methodisch sauber vergleichbar ist, die Laufzeiten jedoch nicht direkt auf Produktivsysteme übertragbar sind. In produktiven Systemen mit größeren Korpora wären approximative Indizes (z. B. HNSW) häufig notwendig. Die Interaktion zwischen Quantisierung und approximativer Indexierung bleibt eine offene Frage, die über den Rahmen dieser Arbeit hinausgeht.

3.8.2 Metriken

Die Retrieval-Qualität wird über vier Standardmetriken bewertet:

- **NDCG@10**, die primäre Metrik, da sie sowohl Relevanzgrade als auch Rankingpositionen berücksichtigt und in der BEIR-Literatur als Hauptmetrik etabliert ist.
- **Recall@10** und **Recall@100** messen, welcher Anteil der relevanten Dokumente in den Top- k enthalten ist. Recall@100 ist besonders relevant für zweistufige Systeme, in denen die erste Stufe eine Kandidatenmenge erzeugt.
- **MRR** (Mean Reciprocal Rank) misst, an welcher Position das erste relevante Dokument erscheint.

3.8.3 Statistische Signifikanz

Retrieval-Metriken variieren stark zwischen Queries. Ein Unterschied in der mittleren NDCG@10 zwischen zwei Verfahren ist nur dann aussagekräftig, wenn er nicht durch zufällige Query-Schwankungen erklärbar ist. Für jeden paarweisen Vergleich wird daher ein Wilcoxon-Signed-Rank-Test [53] auf den per-Query-Metriken durchgeführt; dieser Test ist für die Evaluation von Retrieval-Systemen etabliert [47]. Die Wahl fällt auf Wilcoxon statt eines gepaarten t-Tests, weil Retrieval-Metriken typischerweise nicht normalverteilt sind und Wilcoxon keine Verteilungsannahme erfordert. Das Signifikanzniveau wird auf $\alpha = 0,05$ festgelegt. Bei zehn Repräsentationen entstehen pro Metrik und Dimension $\binom{10}{2} = 45$ paarweise Vergleiche. Daher wird eine Bonferroni-Korrektur angewandt ($\alpha_{\text{kor}} = 0,05/45 \approx 0,0011$), um das multiple Testproblem zu berücksichtigen.

3.9 Phase 5: Praktische Evaluation

Die bisherigen Phasen bewerten die Qualität der Kompression. Phase 5 ergänzt die Perspektive der praktischen Anwendbarkeit. Sie misst Speicherbedarf und Laufzeit im tatsächlich implementierten NumPy-Pfad und verbindet diese Werte anschließend mit den Retrieval-Ergebnissen aus Phase 4.

3.9.1 Speicheranalyse

Für jede Repräsentation wird der theoretische Speicherbedarf pro Vektor berechnet:

- Float32: $1024 \times 32 = 32\,768$ Bit = 4 096 Byte
- Float16: $1024 \times 16 = 16\,384$ Bit = 2 048 Byte
- 8-Bit: $1024 \times 8 = 8\,192$ Bit = 1 024 Byte
- 4-Bit: $1024 \times 4 = 4\,096$ Bit = 512 Byte
- 2-Bit: $1024 \times 2 = 2\,048$ Bit = 256 Byte
- 1-Bit: $1024 \times 1 = 1\,024$ Bit = 128 Byte

Diese Werte sind ideale Untergrenzen bei vollständiger Bitpackung. Zusätzlich wird der tatsächliche Speicherverbrauch des NumPy-Layouts gemessen. Dabei werden neben den gepackten Codes auch Metadaten wie `col_min` und `col_max` berücksichtigt, die für 8-Bit-, 4-Bit- und 2-Bit-Quantisierung zur Dequantisierung benötigt werden. Für 1-Bit entstehen in der aktuellen Implementierung keine zusätzlichen Quantisierungsmetadaten; der Speicher entspricht daher bei durch acht teilbaren Dimensionen exakt der bitgepackten Größe. Der Kompressionsfaktor wird jeweils gegen die volle Float32-Baseline mit 1024 Dimensionen berechnet.

3.9.2 Laufzeitanalyse

Die Suchgeschwindigkeit wird im aktuell implementierten NumPy-Pfad gemessen. Dies ist keine optimierte native C- oder AVX-Implementierung, sondern ein reproduzierbarer Referenzpfad, dessen rechenintensive Teile über NumPy und BLAS ausgeführt werden. Gemessen werden:

- **Pairwise-Distanzlaufzeit**, also die Zeit für eine feste Stichprobe von 10 000 zufälligen Dokumentpaaren.
- **Brute-Force-Top- k -Laufzeit**, also die Zeit, für alle Vektoren im betrachteten Korpusausschnitt die $k = 10$ nächsten Nachbarn zu bestimmen.
- **Throughput**, gemessen als Paare pro Sekunde für Pairwise-Distanzen beziehungsweise Queries pro Sekunde für Top- k .

Die Hauptmessung wird für SciFact, FiQA und TREC-COVID auf kontrollierten Korpusausschnitten mit 1 000 Dokumenten durchgeführt. Zusätzlich werden Skalierungsläufe mit 250, 500 und 1 000 Dokumenten erzeugt, um das Verhalten bei wachsender Vektoranzahl sichtbar zu machen. Die Dimensionsmatrix entspricht der üblichen Evaluation und umfasst $d \in \{64, 128, 256, 384, 512, 768, 1024\}$. Der Indexaufbau, das Embedding-Encoding, Dataset-Download, JSON-Schreiben und Plotting zählen nicht zur Suchlaufzeit. Die gemessenen Laufzeiten beschreiben daher die Suche auf bereits erzeugten Repräsentationen. Die Resultate sind als Implementierungsmessung des NumPy-Pfads zu interpretieren: Float32 profitiert von optimierter Matrixmultiplikation, während 1-Bit-Top- k in der aktuellen Version über Broadcast-XOR und Lookup-Table-Popcount läuft und daher keine Aussage über einen optimalen nativen Binärindex mit Hardware-Popcount darstellt.

3.9.3 Qualitäts-Speicher-Trade-off

Ergänzend zu Speicher und Laufzeit werden Trade-off-Metriken berechnet, die Retrieval-Qualität und Speicherersparnis gemeinsam bewerten. Dazu gehören die Relative Quality-to-Space Ratio (RQSR), der Compression-Quality-Score CQ_1 und die Quality Elasticity of Storage (QES). Diese Metriken dienen nicht als Ersatz für NDCG@10, sondern als zusätzliche Entscheidungshilfe, um Konfigurationen mit ähnlicher Qualität nach ihrem Speicherbedarf zu vergleichen.

3.10 Phase 6: Praktische Anwendungsfall- und Skalierungsanalyse

Die Phasen 1 bis 5 evaluieren die Quantisierungsverfahren unter kontrollierten Bedingungen: Distanzverzerrung, Nachbarschaftserhaltung, Retrieval-Qualität, Speicherbedarf und Laufzeit werden isoliert betrachtet. Phase 6 übersetzt diese Laborergebnisse in konkrete Anwendungsszenarien. Im Zentrum steht nicht mehr die Frage, ob eine Repräsentation geometrisch oder retrievalbezogen funktioniert, sondern unter welchen praktischen Speicher- und Kostenbudgets sie sinnvoll eingesetzt werden kann.

3.10.1 Ziel und Abgrenzung

Phase 6 verfolgt drei Ziele. Erstens wird der Speicherbedarf ausgewählter TurboQuant-Konfigurationen auf realistische Indexgrößen hochgerechnet. Zweitens werden diese Konfigurationen Anwendungsszenarien zugeordnet, die unterschiedliche Anforderungen an Speicher, Qualität und Betriebskosten stellen. Drittens wird für das mobile

Szenario eine feinere Skalierung unterhalb von 10 Millionen Vektoren betrachtet, da dort Speicherbudgets im Bereich weniger MiB bis einiger hundert MiB praktisch entscheidend sind. Die Analyse ist bewusst keine neue Retrieval-Evaluation. Die Qualitätswerte werden aus Phase 4 übernommen, insbesondere NDCG@10 auf SciFact relativ zur Float32-Referenz bei $d = 1024$. Der Speicherbedarf wird aus Phase 5 übernommen und linear auf größere Indexgrößen übertragen. Die Extrapolation bezieht sich ausschließlich auf die komprimierte Vektorrepräsentation. Metadaten, Textchunks, ANN-Zusatzstrukturen, Betriebssystem-Overhead und Laufzeitpuffer werden nicht eingerechnet. Damit beschreibt Phase 6 eine untere, aber kontrolliert vergleichbare Speichergrenze. Naive Quantisierung wird in Phase 6 nicht als eigene Anwendungslinie weitergeführt. Bei gleicher Bittiefe und gleicher Dimensionszahl ist ihr Speicherbedarf identisch zu TurboQuant, sodass die Speicher- und Kostenrechnung für beide Verfahren gilt. Da sich Phase 6 auf Speicher- und Kostenskalierung konzentriert, wird die Empfehlungslinie zur einheitlichen Darstellung auf TurboQuant beschränkt; die zugehörige Qualitätsbewertung erfolgt anhand der in Phase 4 gemessenen Werte.

3.10.2 Szenarioauswahl

Für die Übertragung in die Praxis werden drei Einsatzszenarien definiert, die sich in ihren Anforderungen an Speicher, Qualität und Betriebskosten unterscheiden. Jedem Szenario wird eine TurboQuant-Konfiguration als Arbeitspunkt zugeordnet:

- **Edge/Mobile:** Ziel ist minimale Speichernutzung für lokale Suche auf Smartphones, Laptops oder im Browser. Als aggressive Konfiguration wird `tq_1bit` bei $d = 768$ verwendet.
- **Business Sweet Spot:** Ziel ist eine kostenoptimierte Cloud-Suche, die einen ausgewogenen Kompromiss zwischen Speicherbedarf und Qualität sucht. Hier wird `tq_2bit` bei $d = 1024$ betrachtet.
- **Enterprise Precision:** Ziel ist ein möglichst geringer Qualitätsverlust bei weiterhin deutlicher Speicherreduktion. Hier wird `tq_4bit` bei $d = 1024$ betrachtet.

Diese Szenarien sind keine neuen Algorithmen, sondern feste Arbeitspunkte auf der Qualitäts-Speicher-Kurve, deren Qualitäts- und Speicherwerte aus den Phasen 4 und 5 stammen. Die Float32-Repräsentation bei $d = 1024$ dient als Referenz.

3.10.3 Skalierungsrechnung

Für jede Konfiguration wird zunächst der Speicherbedarf pro Vektor bestimmt. Für eine Indexgröße N ergibt sich der reine Vektorspeicher zu

$$M(N) = \frac{N \cdot b}{1024^3}, \quad (3.6)$$

wobei b die Anzahl der Bytes pro Vektor bezeichnet und $M(N)$ in GiB angegeben wird. Für kleinere mobile Korpora wird analog in MiB berichtet. Die Hauptszenarien werden auf $N = 100\,000$, $N = 1\,000\,000$ und $N = 10\,000\,000$ Vektoren skaliert. Für das mobile Szenario wird zusätzlich die feinere Folge

$$N \in \{10\,000, 50\,000, 100\,000, 250\,000, 500\,000, 1\,000\,000, 5\,000\,000\}$$

betrachtet. Diese Größenordnung deckt typische lokale Bestände ab: App-Hilfen, persönliche Dokumente, kuratierte Wissenssammlungen oder Teilindizes größerer Systeme.

3.10.4 Kostenmodell

Für Cloud-Szenarien wird der Speicherbedarf auf grobe AWS-EC2-Instanzklassen abgebildet. Die Kostenrechnung ist als betriebswirtschaftliche Größenordnung zu verstehen, nicht als exakter Beschaffungsplan. Die konkrete Preisquelle ist die regionale Amazon-EC2-Preisliste für `eu-central-1` [2]; die AWS-Dokumentation zur Price List Bulk API beschreibt den Zugriff auf solche Preislistendateien [4]. Aus der CSV werden die Zeilen für On-Demand, Linux, Shared Tenancy, Stundenabrechnung, ohne vorinstallierte Zusatzsoftware und die Region `eu-central-1` gefiltert. Die Monatskosten ergeben sich explizit aus Stundenpreis $\times 730$, wobei die 730 Stunden der Annahme des AWS Pricing Calculator entsprechen [3]. Die Zuordnung lautet: Float32 bei 10 Millionen Vektoren benötigt eine speicherstärkere Instanzklasse mit 64 GiB RAM, `tq_4bit` passt in eine 8-GiB-Instanz, und `tq_2bit` passt in eine 4-GiB-Instanz. Die RAM-Ausstattung der gewählten Instanztypen wird den AWS-Instanzspezifikationen entnommen: `r6i.2xlarge` aus den memory-optimized Instanzen, `m6g.large` und `t4g.medium` aus den general-purpose Instanzen [6, 5]. Konkret wird die Float32-Referenz einer speicheroptimierten `r6i`-Instanz zugeordnet, während die komprimierten Arbeitspunkte in günstigere Allzweck- beziehungsweise Graviton-Instanzen (`t4g`, `m6g`) fallen. Die Ersparnis speist sich damit aus zwei Quellen: dem geringeren Speicherbedarf und dem dadurch möglichen Wechsel der Instanzfamilie. Die Zuordnung der Float32-Referenz zur speicherstarken `r6i`-Instanz ist eine bewusst konservative Annahme und markiert das obere Ende der darstellbaren Ersparnis; die berichteten Prozentwerte sind entsprechend als

Best-Case-Größenordnung und nicht als garantierter Mindestwert zu verstehen. Da AWS-Listenpreise regions- und zeitabhängig sind, gilt die Rechnung nur für den angegebenen Abrufzeitpunkt, die verwendete Preislistenversion und die gewählte Region. Das Edge/Mobile-Szenario wird nicht als Cloud-Kostenfall modelliert, weil der Index lokal betrieben wird.

3.11 Synthese und Interpretation

Die abschließende Analyse verknüpft die Ergebnisse aller Phasen zu einer kohärenten Beantwortung der Forschungsfrage. Konkret werden vier Leitfragen beantwortet:

1. **Was bleibt erhalten?** Welche Aspekte der semantischen Struktur (globale Distanzen, lokale Nachbarschaften und Retrieval-Rangordnungen) überleben die Quantisierung intakt?
2. **Was geht verloren?** Wo treten systematische Verzerrungen auf, und lassen sich diese aus den in Phase 1 identifizierten Raumeigenschaften erklären?
3. **Was bedeutet das für die Praxis?** Ab welchem Qualitätsverlust wird der Speicher- und Geschwindigkeitsvorteil durch unbrauchbare Suchergebnisse zunichte gemacht?
4. **Praktische Konfiguration:** Welche TurboQuant-Arbeitspunkte ergeben sich für mobile, kostenoptimierte und hochpräzise Einsatzszenarien?

Diese Struktur stellt sicher, dass die Arbeit nicht bei der Beobachtung stehen bleibt, sondern eine wissenschaftlich fundierte Empfehlung abgeben kann, unter welchen Bedingungen welche Kompressionstiefe gerechtfertigt ist, sowohl im kontrollierten Benchmark als auch unter praktischen Speicher- und Kostenrestriktionen.

4 Ergebnisse und Diskussion

Dieses Kapitel präsentiert die experimentellen Ergebnisse und interpretiert sie im Kontext der Forschungsfrage. Die Darstellung folgt der Phasenstruktur des Experiments: Für jede Phase werden zunächst die Ergebnisse berichtet, anschließend im Hinblick auf die in der Methodik formulierten Erwartungen diskutiert und schließlich Implikationen für die nachfolgenden Phasen abgeleitet.

4.1 Phase 1: Charakterisierung des Float-Raums

Phase 1 analysiert den unkomprimierten Float-Raum, um Erwartungen an das Quantisierungsverhalten zu formulieren. Die Ergebnisse zeigen, dass der von **bge-large-en-v1.5** erzeugte Embedding-Raum Eigenschaften besitzt, die eine gute Quantisierbarkeit erwarten lassen.

4.1.1 Normverteilung

Tabelle 4.1 fasst die Normstatistiken über alle drei Datensätze zusammen. Alle

Tabelle 4.1: L2-Normverteilung der Float32-Embeddings. CV bezeichnet den Variationskoeffizienten σ/μ .

Datensatz	μ	σ	CV
SciFact	1,0	$3,05 \times 10^{-8}$	$3,05 \times 10^{-8}$
FiQA	1,0	$2,99 \times 10^{-8}$	$2,99 \times 10^{-8}$
TREC-COVID	1,0	$3,04 \times 10^{-8}$	$3,04 \times 10^{-8}$

Normen liegen bis auf Float32-Maschinengenauigkeit bei 1,0 (gemessener Bereich [0,9999999, 1,0000001]). Die Variationskoeffizienten von CV $\approx 3 \times 10^{-8}$ liegen in der Größenordnung der Float32-Maschinengenauigkeit ($\epsilon_{\text{float32}} \approx 1,2 \times 10^{-7}$) und stellen damit numerisches Rauschen dar, keine inhaltliche Variation. Die Vektoren liegen damit auf der Einheitshypersphäre. Die Statistik umfasst jeden einzelnen Vektor

der drei vollständigen Korpora (5 183, 57 638 und 171 332 Dokumente); eine Stichprobenziehung findet in Phase 1 nicht statt. Die Einheitslänge ist dabei nicht das Ergebnis einer Normalisierung innerhalb der Analyse, sondern eine Eigenschaft der Embedding-Erzeugung: Der Encoder skaliert jeden Ausgabevektor auf L2-Norm 1 (vgl. Abschnitt 3.4). Die Messung bestätigt, dass diese Normierung für alle Vektoren bis auf Rundungsfehler exakt gilt.

Interpretation. Die Binarisierung nach Gleichung 2.2 kodiert jede Komponente x_i allein anhand ihres Vorzeichens. Die L2-Norm eines Vektors geht in diese Kodierung nicht ein: Zwei Vektoren mit identischem Vorzeichenmuster, aber unterschiedlicher Norm erhalten denselben Bitvektor (vgl. Abschnitt 2.3). Ein Informationsverlust entsteht dadurch nur, wenn sich die Dokumente in ihrer Norm unterscheiden, die Norm also zur Unterscheidung von Dokumenten beiträgt. Da alle Normen hier bis auf Rundungsfehler gleich 1 sind, enthält die Norm keine dokumentspezifische Information; ihr Wegfall ist somit verlustfrei. Der verbleibende Informationsverlust der Binarisierung betrifft ausschließlich die Beträge $|x_i|$ der einzelnen Komponenten: Wie weit ein Wert von Null entfernt liegt, geht verloren, das Vorzeichen bleibt erhalten. Diese günstige Voraussetzung ist keine Selbstverständlichkeit; bei Embedding-Modellen ohne explizite L2-Normierung würde die Binarisierung zusätzlich die in der Norm kodierte Information verlieren.

4.1.2 Dimensionsstatistiken

Tabelle 4.2 zeigt die aggregierten Dimensionsstatistiken. Die zugehörige Verteilung über alle 1024 Dimensionen ist in Abbildung 4.1 dargestellt. Die Ergebnisse sind über alle

Tabelle 4.2: Dimensionsstatistiken über alle 1024 Dimensionen. Die Schwellwerte $|\text{skew}| > 1,0$ und $\text{kurtosis} > 3,0$ (Fisher-Kurtosis, Normalverteilung = 0) entsprechen den in Abschnitt 3.5.2 festgelegten Kriterien für quantisierungskritische Dimensionen; $n_{\text{probl.}}$ zählt die Dimensionen, die mindestens einen Schwellwert überschreiten.

Datensatz	$ \overline{\text{skew}} $	$\max \text{skew} $	$\overline{\text{kurt}}$	$\max(\text{kurt})$	$n_{\text{probl.}}$
SciFact	0,074	0,377	0,003	0,662	0 / 1024
FiQA	0,081	0,471	0,044	0,665	0 / 1024
TREC-COVID	0,085	0,366	0,022	0,539	0 / 1024

drei Datensätze konsistent: Keine einzige der 3072 untersuchten Dimensionen (3×1024) überschreitet die Schwellwerte. Die maximale Schiefe liegt bei 0,47 (FiQA) und

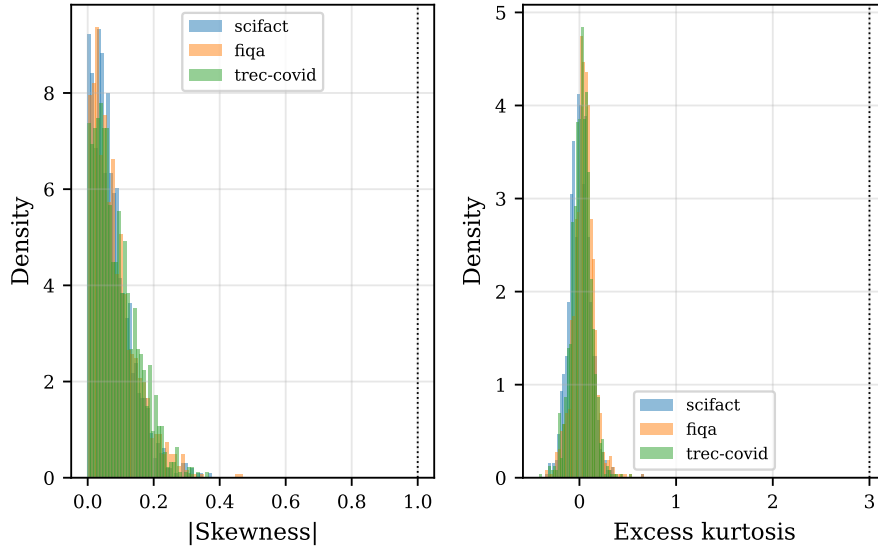


Abbildung 4.1: Verteilung der Schiefe und Kurtosis über alle 1024 Dimensionen für die drei Datensätze. Die gestrichelten Linien markieren die Schwellwerte $|\text{skew}| = 1,0$ und $\text{kurtosis} = 3,0$, ab denen Quantisierungsprobleme zu erwarten wären (vgl. Abschnitt 3.5.2).

damit bei weniger als der Hälfte des Grenzwerts. Die maximale Kurtosis beträgt 0,67 (FiQA), was einer annähernd normalverteilten Dimension entspricht und weit von problematischem Ausreißerverhalten entfernt liegt.

Interpretation. Die nahezu symmetrischen Verteilungen um Null haben zwei konkrete Implikationen für die Quantisierung. Erstens: Bei der Binarisierung wird der Schwellwert bei $x_i = 0$ gesetzt, positiv wird zu 1, negativ zu 0. Bei einer symmetrischen Verteilung teilt dieser Schwellwert die Werte in zwei annähernd gleich große Hälften. Damit trägt jedes einzelne Bit nahezu maximale Information (marginale Entropie nahe 1). Wären Dimensionen stark schief, würde z. B. eine 80/20-Aufteilung entstehen, und das Bit wäre weniger informativ. Diese Aussage betrifft die Randverteilung jeder Dimension; ob die Bits untereinander unkorreliert sind und damit auch gemeinsam maximale Information tragen, wird in Phase 1 nicht untersucht. Da hier keine problematische Schiefe vorliegt, arbeitet die Binarisierung zumindest bezüglich der Schwellenwahl unter günstigen Bedingungen. Zweitens: Die niedrige Kurtosis bedeutet, dass keine Dimension Ausreißer aufweist, die bei uniformer Quantisierung mit wenigen Stufen (z. B. 4 Stufen bei 2-Bit) zu großen Fehlern führen würden. Bei hoher Kurtosis würden einzelne extreme Werte den Wertebereich aufblähen und die wenigen Stufen

fast leer bleiben. Das ist hier nicht der Fall. Die gleichmäßige Stufenverteilung von TurboQuant ist für diesen Raum gut geeignet. Die Konsistenz über drei unterschiedliche Domänen (Naturwissenschaft, Finanz, Biomedizin) deutet darauf hin, dass diese Eigenschaften vom Modell und nicht vom Datensatz bestimmt werden.

4.1.3 Intrinsische Dimensionalität

Tabelle 4.3 stellt die Schätzungen der intrinsischen Dimensionalität gegenüber. Das zugehörige PCA-Varianzspektrum und die kumulative erklärte Varianz sind in den Abbildungen 4.2 und 4.3 dargestellt.

Tabelle 4.3: Intrinsische Dimensionalität: TwoNN (lokale Mannigfaltigkeitsstruktur) und PCA (globale lineare Dimensionalität, 95% erklärte Varianz). Ergänzend die kumulative erklärte Varianz $V(d)$ an den evaluierten Reduktionszielen.

Datensatz	TwoNN	PCA _{95%}	$V(64)$	$V(256)$	$V(384)$
SciFact	13,2	390	55,1 %	87,4 %	94,8 %
FiQA	22,1	395	57,6 %	87,8 %	94,6 %
TREC-COVID	11,4	417	54,6 %	85,4 %	93,7 %

Diskrepanz zwischen lokaler und globaler Schätzung. Die auffälligste Beobachtung ist die starke Divergenz zwischen den beiden Schätzern. TwoNN schätzt die lokale Mannigfaltigkeitsdimension auf 11–22, während PCA 390–417 Komponenten für 95 % Varianz benötigt, eine Differenz von mehr als einer Größenordnung. TwoNN misst die lokalen Freiheitsgrade: Wie viele unabhängige Richtungen existieren in der unmittelbaren Umgebung eines Punktes? Der niedrige Wert zeigt, dass semantisch ähnliche Dokumente sich nur entlang weniger Achsen unterscheiden; die Daten liegen lokal auf einer niedrigdimensionalen Mannigfaltigkeit. PCA misst hingegen die globale lineare Dimensionalität: Wie viele orthogonale Richtungen sind nötig, um die Gesamtstreuung aller Punkte zu erfassen? Der hohe Wert zeigt, dass die Mannigfaltigkeit zwar lokal flach ist, aber global viele verschiedene Orientierungen einnimmt. Die naheliegende Deutung ist die einer gekrümmten Mannigfaltigkeit, die in einen hochdimensionalen Raum eingebettet ist. Da TwoNN als lokaler Maximum-Likelihood-Schätzer eigene Verteilungsannahmen trifft und bei hochdimensionalen, verrauschten Daten zur Unterschätzung neigt, ist der absolute Wert mit Vorsicht zu lesen; für die hier gezogene Schlussfolgerung genügt jedoch die klare Größenordnungsdifferenz zwischen lokaler und globaler Dimensionalität.

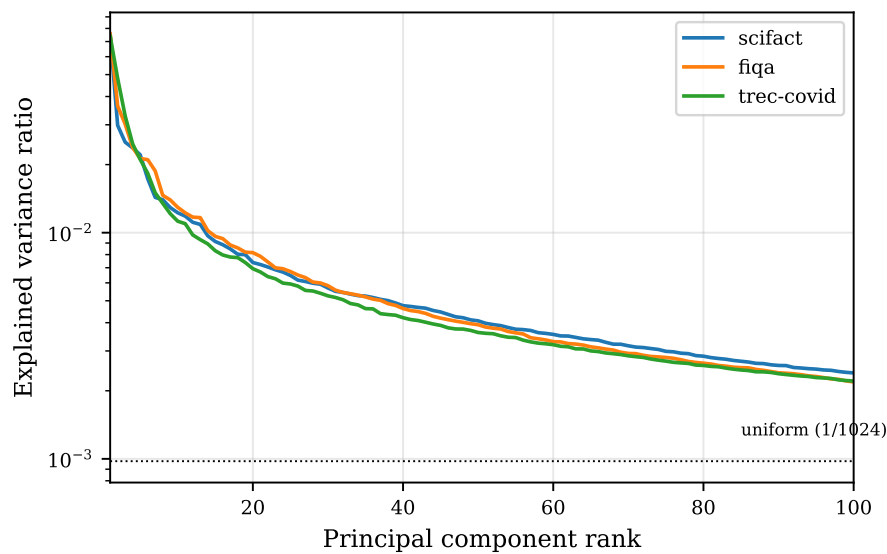


Abbildung 4.2: PCA-Varianzspektrum: Erklärte Varianz pro Hauptkomponente (absteigende Reihenfolge). Der schnelle Abfall der ersten Komponenten gefolgt von einem langen, flachen Schwanz zeigt die typische Varianzkonzentration in hochdimensionalen Embedding-Räumen.

Implikationen für die Dimensionsreduktion. Aus den PCA-Ergebnissen lassen sich quantitative Vorhersagen für die nachfolgenden Phasen ableiten. Tabelle 4.3 berichtet die kumulative erklärte Varianz an den evaluierten Reduktionszielen:

- $d = 384$: Mit 93,7–94,8 % erklärter Varianz bleibt fast die gesamte Streuung erhalten. Nur eine marginale Qualitätsreduktion ist zu erwarten.
- $d = 256$: Bei 256 Komponenten verbleiben 85,4–87,8 % der Varianz. Hier beginnt der Bereich, in dem spürbarer Informationsverlust einsetzt.
- $d = 64$: Mit nur 54,6–57,6 % erklärter Varianz geht fast die Hälfte der Gesamtstreuung verloren, sodass ein deutlicher Qualitätsverlust zu erwarten ist.

Implikationen für die Quantisierung. Die niedrige lokale Dimensionalität (TwoNN ≈ 11 –22) hat eine weitere Konsequenz. Die lokale Nachbarschaftsstruktur, die für Retrieval entscheidend ist, wird von relativ wenigen effektiven Dimensionen bestimmt. Quantisierung behandelt jede Dimension unabhängig und verwendet damit einen substantiellen Teil der Bitbreite auf redundante Dimensionen, ohne die relevante lokale Struktur zu beschädigen. Das legt nahe, dass selbst aggressive Quantisierung

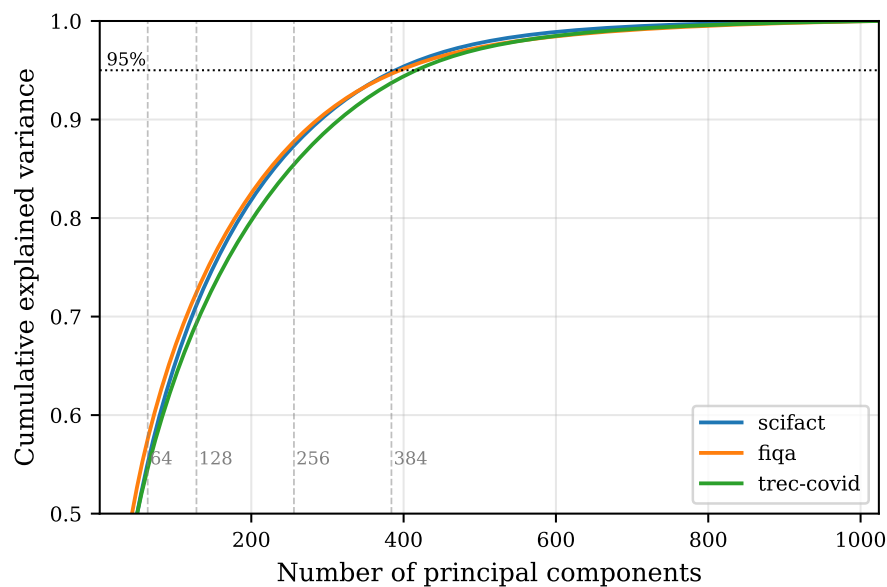


Abbildung 4.3: Kumulative erklärte Varianz als Funktion der Komponentenzahl. Die horizontale Linie markiert 95 %. Die vertikalen Markierungen zeigen die evaluierten PCA-Dimensionen $d \in \{64, 128, 256, 384, 512, 768, 1024\}$.

(2-Bit, 1-Bit) die *lokale* Nachbarschaftsstruktur besser erhalten könnte als die *globale* Distanzstruktur. Die Begründung: Erstere hängt von weniger effektiven Dimensionen ab als letztere. Diese Hypothese wird in Phase 3 empirisch geprüft.

4.1.4 Zusammenfassung und Vorhersagen

Die Charakterisierung ergibt ein konsistentes Bild: Der Embedding-Raum von **bge-large-en-v1.5** besitzt drei Eigenschaften, die aus theoretischer Sicht eine gute Quantisierbarkeit erwarten lassen:

1. **Perfekte Normierung:** Die Binarisierung verliert keine Norminformation.
2. **Symmetrische, nahezu normalverteilte Dimensionen ohne schwere Ränder:** Der Vorzeichenschwellwert liegt optimal, und uniforme Quantisierung verursacht keine Ausreißer-Fehler.
3. **Hohe Redundanz:** Die intrinsische Dimensionalität liegt weit unter der nominalen, sodass Kompression über beide Achsen (Bittiefe und Dimensionszahl) möglich ist.

Daraus ergeben sich testbare Vorhersagen für die nachfolgenden Phasen:

- Selbst die naive Binarisierung sollte aufgrund der günstigen Raumeigenschaften eine nicht-triviale Distanz- und Nachbarschaftserhaltung zeigen.
- Der Vorteil von TurboQuant gegenüber naiver Quantisierung sollte moderat ausfallen, da die einzelnen Dimensionen bereits symmetrisch und ausreißerarm verteilt sind und die Rotation an dieser günstigen Ausgangslage pro Dimension wenig verbessern kann. Diese Erwartung betrifft die volle Dimensionszahl; unter Dimensionsreduktion kann die Rotation stärker wirken, da PCA die Varianz ungleich über die Komponenten verteilt.
- PCA-Reduktion auf $d \geq 384$ sollte bei allen Quantisierungsstufen nur marginale Qualitätsverluste verursachen.
- Die stärksten Qualitätseinbrüche werden bei $d \leq 128$ erwartet, wo rund 30 % ($d = 128$) bis über 40 % ($d = 64$) der Varianz verloren gehen.

Ob diese Vorhersagen zutreffen, wird in den Phasen 2 bis 4 empirisch überprüft.

4.2 Phase 2: Distanz- und Fehleranalyse

Phase 2 prüft, ob die geometrischen Beziehungen zwischen Dokumenten unter Quantisierung erhalten bleiben. Dazu werden für 10 000 zufällig gezogene Paare die Distanzen im Float-Raum mit den Distanzen in allen komprimierten Repräsentationen verglichen. Die Ergebnisse werden entlang dreier Achsen präsentiert: zunächst das Verhalten bei voller Dimensionszahl ($d = 1024$, ohne PCA), dann der Einfluss der Dimensionsreduktion, und schließlich die Pareto-optimalen Konfigurationen.

4.2.1 Distanzerhaltung bei voller Dimensionszahl

Tabelle 4.4 zeigt die Pearson- und Spearman-Korrelationen bei $d = 1024$ (keine PCA-Reduktion). Die Werte sind über alle drei Datensätze gemittelt; die datensatzspezifischen Werte weichen um weniger als $\pm 0,02$ vom Mittel ab.

Hierarchie der Bittiefen. Bei voller Dimensionszahl ergibt sich eine klare Stufung: Float16 und 8-Bit sind von Float32 praktisch ununterscheidbar ($r > 0,9999$). 4-Bit verursacht minimale Verluste ($r \approx 0,998$), die für Retrieval-Anwendungen irrelevant sind. 2-Bit und 1-Bit erreichen Korrelationen um $r \approx 0,92$, was bedeutet, dass die Distanzstruktur noch erkennbar, aber nicht mehr verzerrungsfrei ist.

Tabelle 4.4: Distanzkorrelation bei $d = 1024$ (keine Dimensionsreduktion). Mittelwerte über drei Datensätze.

Repräsentation	Pearson r	Spearman ρ	MAE	RMSE
Float16	0,9999977	0,9999973	0,0003	0,0004
Naive 8-Bit	0,9999933	0,9999919	0,0005	0,0006
TQ 8-Bit	0,9999934	0,9999921	0,0005	0,0006
Naive 4-Bit	0,998	0,998	0,007	0,009
TQ 4-Bit	0,998	0,998	0,006	0,008
Naive 2-Bit	0,927	0,922	0,086	0,098
TQ 2-Bit	0,925	0,920	0,087	0,099
Naive 1-Bit	0,925	0,918	0,051	0,062
TQ 1-Bit	0,921	0,914	0,069	0,080

Konvergenz von TurboQuant und naiver Quantisierung. Ein bemerkenswertes Ergebnis ist, dass TurboQuant bei voller Dimensionszahl *keinen* Vorteil gegenüber naiver Quantisierung bietet. Bei 8-Bit und 4-Bit sind die Werte nahezu identisch; bei 2-Bit und 1-Bit ist naive Quantisierung sogar geringfügig besser. Dies erklärt sich aus der in Phase 1 beobachteten Raumstruktur: Die Dimensionen des Modells sind bereits annähernd symmetrisch und gleichmäßig verteilt. Die orthogonale Rotation von TurboQuant, die darauf abzielt, ungleichmäßige Varianz zu verteilen, findet hier kaum Optimierungspotenzial. Bei 1-Bit liegt naive Quantisierung mit $r = 0,93$ gegenüber 0,92 sogar minimal vorn. Dieser vernachlässigbare, aber systematische Unterschied entsteht, weil die Rotation bei ohnehin binärer Quantisierung zusätzliche Korrelationen zwischen den Bits einführen kann, ohne die Vorzeichenstruktur zu verbessern. Dies bestätigt die in Abschnitt 4.1.4 formulierte Vorhersage, dass der Vorteil von TurboQuant moderat ausfallen sollte. Bei voller Dimensionszahl fällt er sogar gänzlich weg; die Rotation zählt sich erst unter Dimensionsreduktion aus (siehe nächster Abschnitt).

4.2.2 Einfluss der Dimensionsreduktion

Abbildung 4.4 zeigt den Pearson-Korrelationskoeffizienten als Funktion der PCA-Dimensionszahl für alle Quantisierungsstufen. Die folgende Darstellung stützt sich auf Pearson r ; die für Retrieval eigentlich relevantere Rangkorrelation Spearman ρ (Abbildung 4.5) verläuft nahezu deckungsgleich, sodass die Wahl der Metrik für die Interpretation unkritisch ist. Die zentrale Beobachtung ist, dass die verschiedenen Bittiefen sehr unterschiedlich auf Dimensionsreduktion reagieren.

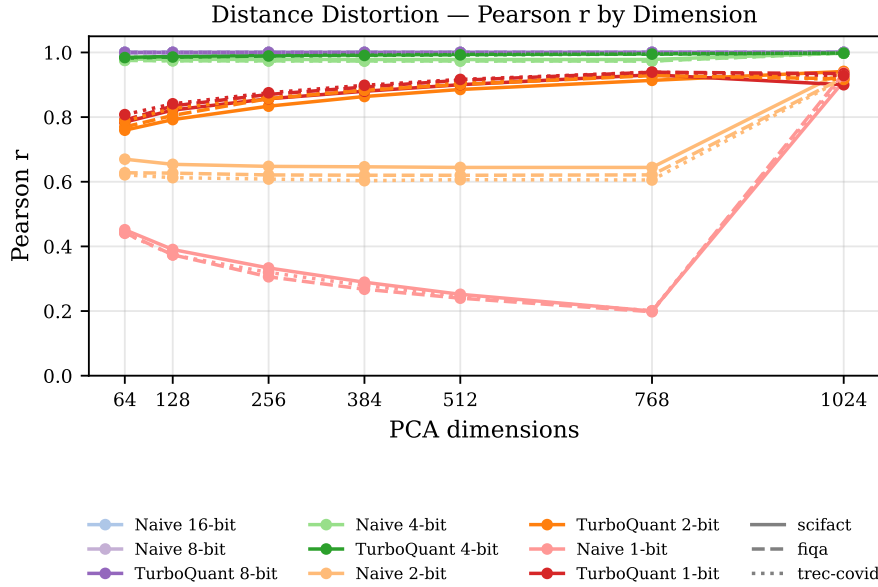


Abbildung 4.4: Pearson r als Funktion der PCA-Dimension d für alle Quantisierungsmethoden. Mittelwert über drei Datensätze. Die Kurven zeigen, dass 8-Bit und 4-Bit robust gegenüber Dimensionsreduktion sind, während 1-Bit ohne Rotation bei mittleren Dimensionen kollabiert.

Robuste Methoden (8-Bit, 4-Bit, Float16). Float16, 8-Bit und 4-Bit (sowohl naiv als auch TQ) zeigen nahezu keine Abhängigkeit von der Dimensionszahl. Selbst bei $d = 64$ liegt Pearson r für 8-Bit über 0,999 und für 4-Bit über 0,975. Das bedeutet: Bei ausreichender Bittiefe ist Dimensionsreduktion auf $d \geq 64$ mit minimalem zusätzlichem Fehler möglich. Dies bestätigt die PCA-Analyse aus Phase 1, die bei 50 Komponenten bereits 50 % der Varianz konserviert; für hochbitige Repräsentationen reicht selbst diese reduzierte Information aus, um die Distanzstruktur zu erhalten.

Naive 2-Bit: Stagnation. Naive 2-Bit-Quantisierung zeigt ein unerwartetes Verhalten: Der Pearson-Koeffizient verharrt über den gesamten Bereich von $d = 64$ bis $d = 768$ bei $r \approx 0,62$ – $0,64$ und springt erst bei $d = 1024$ auf $r \approx 0,93$. Die Dimensionsreduktion scheint weder zu helfen noch wesentlich zu schaden; das dominante Problem ist die grobe Quantisierung selbst. Bei nur 4 Stufen werden viele unterschiedliche Float-Werte auf denselben quantisierten Wert abgebildet; diese Kollisionen dominieren den Fehler unabhängig davon, ob die Eingabe 64 oder 768 Dimensionen hat. TQ 2-Bit hingegen steigt kontinuierlich von $r \approx 0,77$ bei $d = 64$ auf $r \approx 0,92$ bei $d = 768$ und konvergiert bei $d = 1024$ mit der naiven Variante.

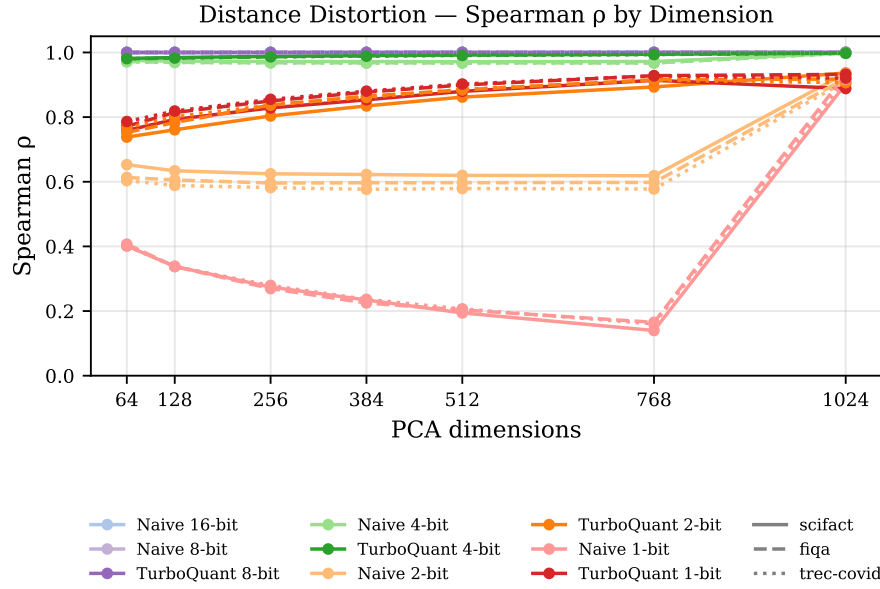


Abbildung 4.5: Spearman ρ als Funktion der PCA-Dimension d . Das Bild bestätigt die Pearson-Ergebnisse: Die Rangordnung bleibt bei 8-Bit und 4-Bit über alle Dimensionen stabil, bricht aber bei naiver 1-Bit-Quantisierung unter Dimensionsreduktion zusammen.

Naive 1-Bit: Das Verdünnungsparadox. Das auffälligste Ergebnis der gesamten Phase 2 betrifft die naive 1-Bit-Quantisierung. Entgegen der naiven Erwartung, dass mehr Dimensionen stets mehr Information bedeuten, *sinkt* die Korrelation mit steigender PCA-Dimension: von $r \approx 0,45$ bei $d = 64$ auf $r \approx 0,20$ bei $d = 768$. Erst bei $d = 1024$ (ohne PCA) springt der Wert auf $r \approx 0,93$. Dieses Verhalten lässt sich plausibel wie folgt erklären. PCA-Projektion auf $d < 1024$ komprimiert die Information in weniger Dimensionen, erhält aber deren Signalstärke; die Werte pro Dimension haben eine ausreichende Varianz, um nach Binarisierung noch unterscheidbar zu sein. Bei $d = 1024$ (originaler Raum ohne PCA) liegen die Werte in einem gut verteilten, vom Modell gelernten Raum. Im Zwischenbereich ($d = 128\text{--}768$) entsteht jedoch ein problematischer Zustand: Die PCA-Projektion verteilt die verbleibende Varianz auf mehr Dimensionen als nötig, wobei die hinteren Komponenten immer weniger Signal tragen. Die Binarisierung dieser signalschwachen Dimensionen erzeugt quasi-zufällige Bits, die die Hamming-Distanz mit Rauschen überfluten. Bei $d = 768$ tragen bereits hunderte Dimensionen kaum noch Information, ihre Bits sind aber alle gleichgewichtet in der Hamming-Distanz. Dieses Verdünnungsparadox tritt nur bei 1-Bit auf, weil höhere Bittiefen die Magnitudenbeibehaltung erlauben. Schwache PCA-Komponenten

werden bei 4-Bit korrekt als „nahe Null“ kodiert und tragen wenig zur Distanz bei. Bei 1-Bit verliert jede Dimension ihre Magnitude, sodass ein schwaches Signal in Dimension 500 dasselbe Gewicht hat wie ein starkes Signal in Dimension 1.

TurboQuant 1-Bit: Auflösung des Paradoxes. TQ 1-Bit zeigt das Paradox nicht. Die Korrelation steigt monoton von $r \approx 0,79$ bei $d = 64$ auf $r \approx 0,94$ bei $d = 768$ und erreicht bei $d = 1024$ nahezu das Niveau der naiven Variante. Die orthogonale Rotation von TurboQuant verteilt die Varianz gleichmäßig über alle Dimensionen. Nach der Rotation trägt jede Dimension einen vergleichbaren Informationsanteil; es gibt keine „nutzlosen“ Dimensionen mehr, deren Bits nur Rauschen beitragen. Damit wird der Verdünnungseffekt aufgelöst. Dass gerade die Gleichverteilung der Varianz durch Rotation das Paradox behebt, stützt die obige Erklärung: Die ungleiche Varianzverteilung über die Dimensionen ist mit hoher Wahrscheinlichkeit die Ursache, auch wenn Phase 2 den Mechanismus nicht durch eine gezielte Ablation einzelner Komponenten isoliert.

4.2.3 Gesamtmatrix und absolute Verzerrung

Abbildung 4.6 fasst die gesamte Versuchsmatrix (Repräsentation $\times$ Dimensionszahl) exemplarisch als Pearson-Heatmap für den SciFact-Datensatz zusammen und macht die zuvor beschriebenen Muster auf einen Blick sichtbar. Bezüglich MAE und RMSE bestätigen sich die Korrelationsergebnisse. Da Float- und komprimierte Distanzen vor dem Vergleich jeweils separat auf $[0, 1]$ normiert werden (vgl. Abschnitt 3.6), sind die absoluten MAE-Werte zwischen den Repräsentationen nur eingeschränkt vergleichbar; sie beschreiben die Abweichung nach getrennter Reskalierung, nicht einen absoluten Distanzfehler. Die Verhältnisse RMSE/MAE liegen für alle Methoden zwischen 1,1 und 1,3, was auf eine relativ gleichmäßige Fehlerverteilung ohne extreme Ausreißer hindeutet. Selbst bei naiver 1-Bit-Quantisierung mit $d = 768$ (dem ungünstigsten Fall) bleibt RMSE/MAE bei 1,25. Die Fehler sind also nicht durch wenige katastrophal falsche Paare dominiert, sondern verteilen sich gleichmäßig. Der in Abschnitt 3.6 beschriebene kritische Fall einer hohen Korrelation bei gleichzeitig hohem absolutem Fehler tritt bei der 2-Bit-Quantisierung auf: Bei $d = 1024$ erreicht naive 2-Bit zwar $r \approx 0,93$, weist aber mit $\text{MAE} \approx 0,086$ eine deutliche absolute Abweichung auf. Die Distanzen bleiben also monoton weitgehend korrekt, das absolute Distanzniveau ist jedoch spürbar verschoben. Für rein rangbasiertes Retrieval ist dies unkritisch, für schwellwertbasierte Systeme mit fester Ähnlichkeitsgrenze wäre es dagegen relevant.

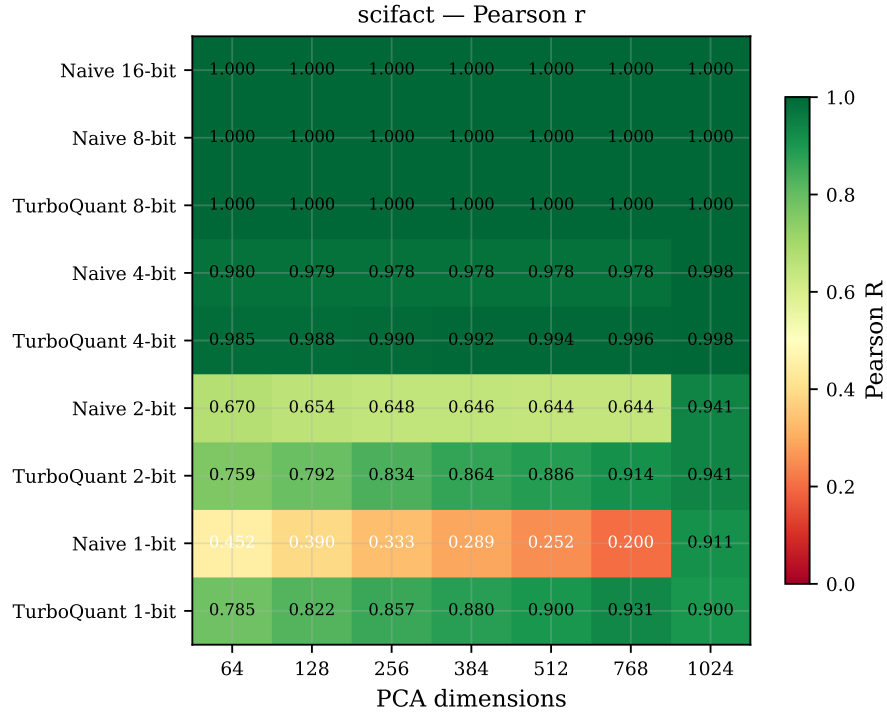


Abbildung 4.6: Heatmap der Pearson-Korrelation über die Versuchsmatrix (SciFact). Die oberen Zeilen (8-Bit, Float16) zeigen durchgehend hohe Werte, während naive 1-Bit bei mittleren Dimensionen deutlich abfällt. Die anderen Datensätze zeigen ein qualitativ identisches Bild.

4.2.4 Pareto-Analyse: Qualität vs. Speicherbedarf

Abbildung 4.7 zeigt die Pareto-Front über alle Kombinationen aus Bittiefe und Dimensionszahl. Eine Konfiguration liegt auf der Pareto-Front, wenn keine andere Konfiguration bei gleichem oder geringerem Speicherbedarf eine bessere Spearman-Korrelation erreicht. Aus der Pareto-Front lassen sich drei Regime ableiten:

- **Hohe Qualität** ($\rho > 0,99$): Naive oder TQ 4-Bit ab $d = 256$ (1024 Bit/Vektor) liefern nahezu perfekte Rangerhaltung. Der Sprung von 4-Bit auf 8-Bit bringt nur noch marginalen Zugewinn bei doppeltem Speicher.
- **Mittlere Qualität** ($\rho \approx 0,88$ bis $0,93$): TQ 1-Bit bei $d = 512$ bis 768 (512 bis 768 Bit/Vektor) erreicht gute Rangerhaltung bei extremer Kompression. Naive 1-Bit ist in diesem Bereich nicht wettbewerbsfähig.

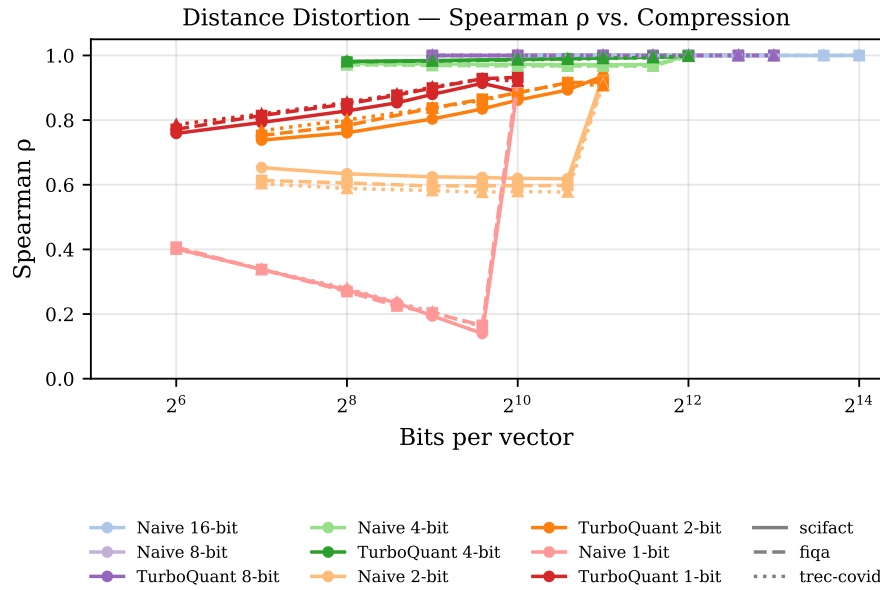


Abbildung 4.7: Pareto-Front der Distanzerhaltung: Spearman ρ als Funktion des Speicherbedarfs (Bit pro Vektor). Konfigurationen auf der Front sind nicht dominiert; es existiert keine andere Konfiguration mit gleichem oder kleinerem Speicher und besserer Rangerhaltung.

- **Minimaler Speicher** (< 256 Bit/Vektor): TQ 1-Bit bei $d = 64$ bis 128 liefert noch $\rho \approx 0,77$ bis 0,80, was für Vorfilterung in zweistufigen Retrieval-Systemen ausreichend sein kann.

4.2.5 Diskussion und Bezug zu Phase 1

Die Ergebnisse von Phase 2 lassen sich in direkten Bezug zu den Vorhersagen aus Phase 1 (Abschnitt 4.1.4) setzen:

Vorhersage 1: Naive Binarisierung zeigt nicht-triviale Distanzerhaltung. Bestätigt. Naive 1-Bit erreicht bei $d = 1024$ einen Pearson $r \approx 0,93$; das bedeutet, dass allein die Vorzeichen der 1024 Dimensionen über 85 % der Distanzvarianz erklären (bestimmt als $r^2 \approx 0,86$). Die in Phase 1 beobachteten günstigen Voraussetzungen (Symmetrie, perfekte Normierung) setzen sich in einem überraschend starken Binarisierungsergebnis um.

Vorhersage 2: Moderater TurboQuant-Vorteil. Teilweise bestätigt, teilweise revidiert. Bei voller Dimensionszahl ist der Vorteil nicht nur moderat, sondern *null* (vgl. den Konvergenzbefund oben). Unter Dimensionsreduktion kehrt sich das Bild jedoch um: Bei $d = 768$ hebt die Rotation den Pearson-Wert für 1-Bit von 0,20 auf 0,94 und für 2-Bit von 0,62 auf 0,92. Die Vorhersage traf die Richtung, unterschätzte aber, dass sich der TQ-Vorteil erst im Zusammenspiel mit Dimensionsreduktion entfaltet.

Vorhersage 3: PCA auf $d \geq 384$ verursacht nur marginale Verluste. Bestätigt für 8-Bit und 4-Bit (sowohl naiv als auch TQ), wo die Korrelation bei $d = 384$ praktisch identisch zu $d = 1024$ bleibt. Für 2-Bit und 1-Bit gilt dies nicht: Bittiefe und Dimensionszahl wirken hier nicht unabhängig voneinander, sondern verstärken sich gegenseitig. Die geringe Stufenzahl macht diese Repräsentationen empfindlich gegenüber Dimensionsreduktion, sodass der Verlust bei $d \geq 384$ deutlich größer ausfällt als bei den hochbitigen Verfahren.

Vorhersage 4: Stärkste Qualitätseinbrüche bei $d \leq 128$. Bestätigt für TQ 1-Bit und TQ 2-Bit, wo $d = 64$ deutlich unter dem Niveau bei $d = 256$ liegt. Für naive 1-Bit trifft sie nicht zu; dort liegt das Minimum überraschenderweise nicht bei der niedrigsten Dimensionszahl, sondern bei mittleren Dimensionen um $d = 768$.

Zusammenfassung. Phase 2 zeigt, dass die Distanzerhaltung durch zwei Faktoren bestimmt wird: Bittiefe und Dimensionszahl. Für hochbitige Verfahren (≥ 4 Bit) ist Dimensionsreduktion weitgehend unproblematisch. Für niedrigbitige Verfahren (1-Bit, 2-Bit) entsteht eine kritische Interaktion, bei der TurboQuant die Rolle eines *Wegbereiters* übernimmt: Ohne Rotation kollabiert naive 1-Bit unter PCA, mit Rotation bleibt die Distanzstruktur intakt. Ob diese Distanzerhaltung sich auch in korrekte *lokale* Nachbarschaften übersetzt, die für Retrieval letztlich entscheidend sind, prüft Phase 3.

4.3 Phase 3: Nachbarschaftserhaltung

Phase 2 bewertet die globale Distanzstruktur über zufällige Paare. Für Retrieval ist jedoch nicht entscheidend, ob weit entfernte Dokumente korrekt als weit entfernt erkannt werden, sondern ob die nächsten Nachbarn eines Dokuments dieselben bleiben. Phase 3 misst daher die Erhaltung der lokalen Nachbarschaftsstruktur über zwei Metriken: Neighborhood Overlap (welcher Anteil der k nächsten Nachbarn bleibt erhalten) und Trustworthiness (werden falsche Nachbarn eingeführt). Alle Werte sind

über die drei Datensätze gemittelt und beziehen sich, sofern nicht anders angegeben, auf $k = 10$ als praxisrelevantesten Wert.

4.3.1 Nachbarschaftserhaltung bei voller Dimensionszahl

Tabelle 4.5 zeigt Overlap und Trustworthiness bei $d = 1024$.

Tabelle 4.5: Neighborhood Overlap und Trustworthiness bei $d = 1024$ und $k = 10$ (keine Dimensionsreduktion). Mittelwerte über drei Datensätze. Die Zufallsbaseline für Overlap liegt bei $k/N \approx 0,002$.

Repräsentation	Overlap@10	Trustworthiness@10
Float16	0,994	1,000
Naive 8-Bit	0,996	1,000
TQ 8-Bit	0,996	1,000
Naive 4-Bit	0,936	1,000
TQ 4-Bit	0,936	1,000
Naive 2-Bit	0,706	0,999
TQ 2-Bit	0,707	0,999
Naive 1-Bit	0,605	0,998
TQ 1-Bit	0,604	0,998

Overlap: strengeres Kriterium als Distanzerhaltung. Der Overlap fällt mit sinkender Bittiefe deutlich steiler ab als die Distanzkorrelation aus Phase 2. Float16 und 8-Bit erhalten die Nachbarschaft nahezu vollständig (Overlap $> 0,99$), 4-Bit bleibt mit 0,936 auf hohem Niveau. Bei 2-Bit verbleiben nur noch etwa 71 % und bei 1-Bit rund 60 % der ursprünglichen Nachbarn. Zum Vergleich erreichten dieselben Verfahren in Phase 2 bei $d = 1024$ noch Distanzkorrelationen um $r \approx 0,92$. Beide Zahlen sind nicht direkt kommensurabel, da Overlap ein linearer Anteil und Pearson r ein Korrelationskoeffizient ist; der Vergleich ist daher illustrativ, nicht als exaktes Verhältnis zu lesen. Inhaltlich ist die lokale Nachbarschaft jedoch das strengere Kriterium: Eine gut erhaltene globale Distanzstruktur garantiert noch keine identische Nachbarschaft, da bei kleinen Distanzunterschieden bereits geringe Verzerrungen die Rangfolge der nächsten Nachbarn verändern.

Trustworthiness: nur rang-nahe Vertauschungen. Das zentrale Ergebnis der Phase 3 ist der Kontrast zwischen Overlap und Trustworthiness. Während der Overlap bei

niedrigen Bittiefen deutlich einbricht, bleibt die Trustworthiness durchgehend nahe 1 und fällt selbst bei 1-Bit nicht unter 0,998. Da Trustworthiness eingeführte Nachbarn proportional zu ihrer Float-Rangdistanz bestraft, bedeutet dieser hohe Wert nicht, dass die Nachbarschaft kaum verändert wird, sondern dass die ausgetauschten Nachbarn fast ausschließlich aus Rängen knapp außerhalb der Nachbarschaft stammen. Im Float-Raum weit entfernte Dokumente erscheinen praktisch nie in der engsten Nachbarschaft. Der sinkende Overlap beschreibt somit die Recall-Perspektive, also den Verlust echter Nachbarn, während die hohe Trustworthiness die Precision-Perspektive abdeckt: Es werden keine grob falschen Nachbarn eingeführt. Für Retrieval ist dies die günstigere Kombination, da die angezeigten Treffer relevant bleiben, auch wenn einzelne echte Nachbarn fehlen.

Konvergenz von TurboQuant und naiver Quantisierung. Wie schon in Phase 2 bei voller Dimensionszahl sind naive und rotierte Varianten nahezu identisch (etwa 0,605 gegenüber 0,604 bei 1-Bit). Ohne Dimensionsreduktion bietet die Rotation auch für die Nachbarschaftserhaltung keinen Vorteil, was mit der bereits gleichmäßigen Raumstruktur aus Phase 1 übereinstimmt.

4.3.2 Einfluss der Dimensionsreduktion

Abbildung 4.8 zeigt den Overlap als Funktion der PCA-Dimension, Abbildung 4.9 die entsprechende Trustworthiness. Die Kurven und die folgenden Werte beziehen sich auf $k = 20$; das Muster ist für $k = 10$ identisch.

Robuste Methoden (4-Bit und höher). TQ 4-Bit erhält die Nachbarschaft über den gesamten Dimensionsbereich robust; der Overlap steigt von 0,867 bei $d = 64$ auf 0,941 bei $d = 1024$ und bleibt damit auch unter starker Dimensionsreduktion hoch. Dies bestätigt die Vorhersage aus Phase 1, dass bei ausreichender Bittiefe eine PCA-Reduktion die lokale Struktur nur marginal beschädigt.

Wiederkehr des Verdünnungsparadoxes. Das in Phase 2 beobachtete Verdünnungsparadox tritt auch bei der Nachbarschaftserhaltung auf. Der Overlap der naiven 1-Bit-Quantisierung steigt mit zunehmender Dimension nicht an, sondern verharrt zwischen $d = 64$ und $d = 768$ bei etwa 0,24–0,31 und springt erst bei $d = 1024$ auf 0,612. Trustworthiness zeigt dasselbe Muster in verschärfter Form: Sie sinkt bei naiver 1-Bit von 0,925 bei $d = 64$ auf 0,799 bei $d = 768$, bevor sie bei $d = 1024$ auf 0,997 zurückspringt. Unter Dimensionsreduktion führt naive 1-Bit also nicht nur mehr Nachbarn-Verluste, sondern auch spürbar mehr falsche Nachbarn ein. Die Ursache ist

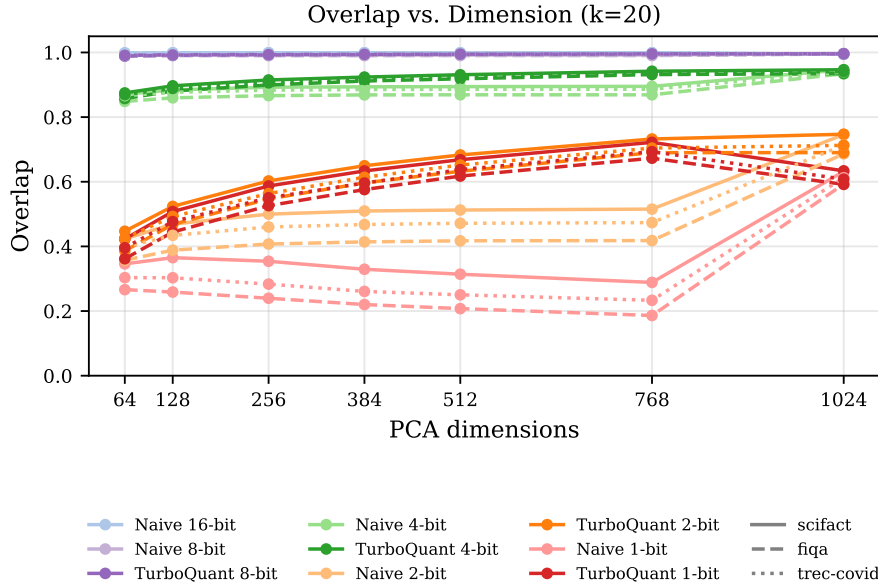


Abbildung 4.8: Neighborhood Overlap als Funktion der PCA-Dimension d (aggregiert über drei Datensätze, $k = 20$). Höhere Bittiefen erhalten die Nachbarschaft über alle Dimensionen robust, während 1-Bit ohne Rotation bei mittleren Dimensionen einbricht.

dieselbe wie in Phase 2: Die hinteren PCA-Komponenten tragen wenig Signal, ihre Binarisierung erzeugt Rauschen, das die Hamming-Distanz dominiert.

TurboQuant als Wegbereiter. TQ 1-Bit zeigt das Paradox erneut nicht. Der Overlap steigt monoton von 0,393 bei $d = 64$ auf 0,696 bei $d = 768$, und die Trustworthiness bleibt über alle Dimensionen bei mindestens 0,985. Die orthogonale Rotation verteilt die Varianz gleichmäßig, sodass keine signalschwachen Dimensionen entstehen, deren Bits die Nachbarschaft verrauschen. Damit bestätigt Phase 3 die Rolle von TurboQuant als Wegbereiter niedrigbitiger Kompression unter Dimensionsreduktion, die bereits in Phase 2 sichtbar wurde.

Einfluss der Nachbarschaftsgröße. Die Wahl von k verändert das Bild nur geringfügig. Bei $d = 1024$ steigt der Overlap der 1-Bit-Varianten von $k = 5$ zu $k = 20$ leicht an (etwa 0,60 auf 0,61), das heißt die engste Nachbarschaft ist nicht überproportional stärker betroffen als die weitere. Abbildung 4.10 zeigt diesen schwachen Zusammenhang über alle Verfahren.

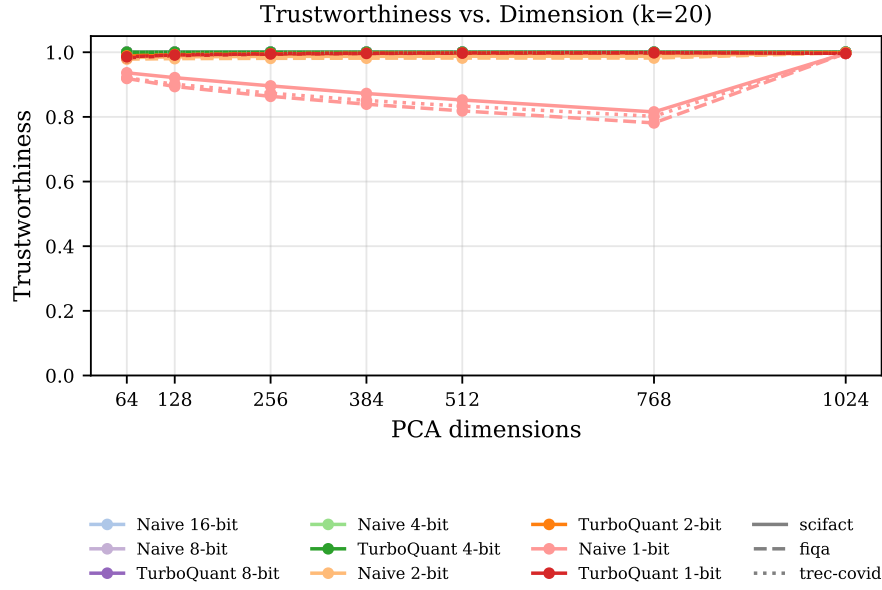


Abbildung 4.9: Trustworthiness als Funktion der PCA-Dimension d (aggregiert über drei Datensätze, $k = 20$). Alle rotierten Verfahren bleiben nahe 1; nur naive 1-Bit fällt unter Dimensionsreduktion sichtbar ab.

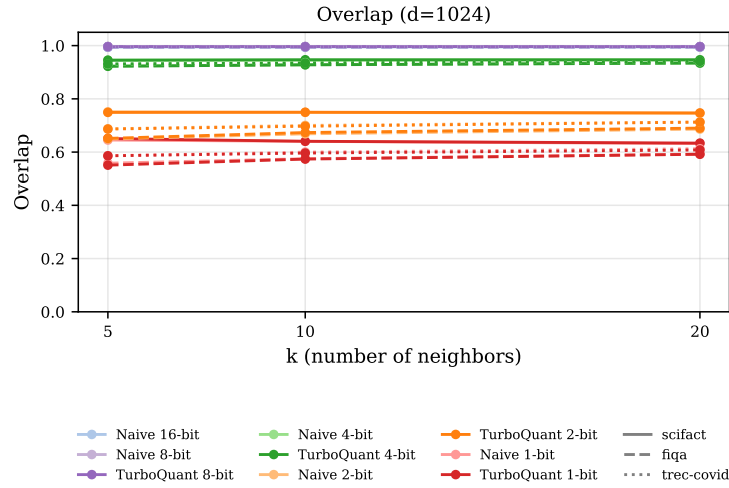


Abbildung 4.10: Neighborhood Overlap als Funktion der Nachbarschaftsgröße k (aggregiert über drei Datensätze, $d = 1024$). Der Overlap ändert sich über $k \in \{5, 10, 20\}$ nur geringfügig.

4.3.3 Pareto-Analyse: Nachbarschaft vs. Speicherbedarf

Abbildung 4.11 zeigt die Pareto-Front für den Overlap in Abhängigkeit vom Speicherbedarf. Die Pareto-Front bestätigt das Bild aus Phase 2. Für hohe Nachbarschaftser-

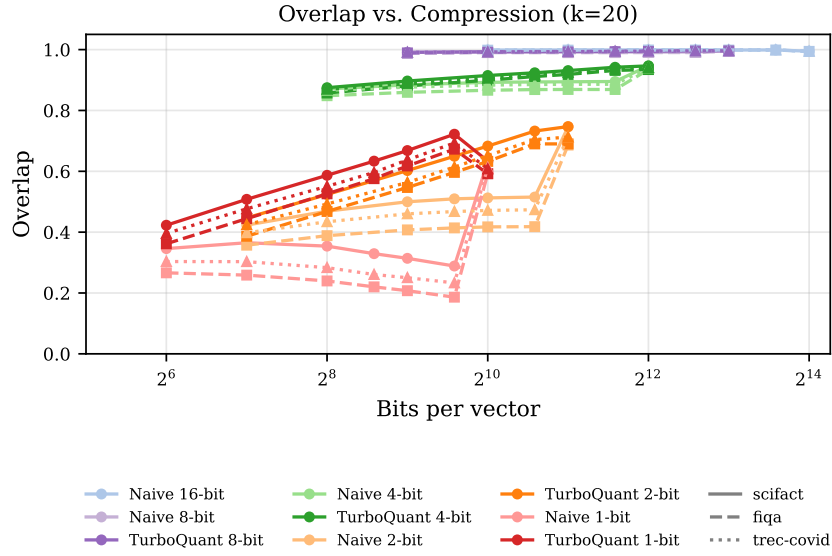


Abbildung 4.11: Pareto-Front der Nachbarschaftserhaltung: Overlap als Funktion des Speicherbedarfs (Bit pro Vektor, $k = 20$). Konfigurationen auf der Front sind nicht dominiert.

haltung ist TQ 4-Bit ab mittleren Dimensionen die effizienteste Wahl; bei extremer Kompression liefert TQ 1-Bit die beste Erhaltung pro Bit, während naive 1-Bit unter Dimensionsreduktion dominiert wird.

4.3.4 Diskussion und Bezug zu den vorherigen Phasen

Bestätigung der Phase-1-Hypothese. Phase 1 hatte aus der niedrigen lokalen Dimensionalität (TwoNN ≈ 11 –22) die Hypothese abgeleitet, dass die lokale Nachbarschaftsstruktur die Quantisierung besser übersteht als die globale Distanzstruktur. Die Ergebnisse bestätigen diese Hypothese differenziert: Die durchweg hohe Trustworthiness zeigt, dass keine grob falschen Nachbarn eingeführt werden, die lokale Struktur in diesem Sinne also stabil bleibt. Der gleichzeitige Rückgang des Overlaps zeigt jedoch, dass die exakte Zusammensetzung der Nachbarschaft bei niedrigen Bittiefen

dennoch verändert wird; erhalten bleibt die grobe lokale Anordnung, nicht die exakte Nachbarmenge.

Overlap und Trustworthiness gemeinsam. Die Ergebnisse fallen durchgehend in das Muster „sinkender Overlap bei hoher Trustworthiness“. Übertragen auf Retrieval bedeutet dies, dass niedrigbitige Verfahren einzelne relevante Nachbarn verlieren, aber die verbleibenden Treffer relevant bleiben. Ob dieser Nachbarverlust die tatsächliche Retrieval-Qualität beeinträchtigt, hängt davon ab, wie viele relevante Dokumente in den Top- k verbleiben, was Phase 4 direkt über Recall und NDCG@10 prüft.

Zusammenfassung. Phase 3 zeigt, dass die Nachbarschaftserhaltung demselben Grundmuster wie die Distanzerhaltung folgt, das Kriterium aber strenger ist. Bittiefen ab 4-Bit erhalten die Nachbarschaft robust, während 1-Bit und 2-Bit einen deutlichen Overlap-Verlust zeigen. Die hohe Trustworthiness über alle Verfahren ist der für Retrieval entscheidende Befund, da eingeführte falsche Nachbarn die schädlichere Fehlerform wären. Unter Dimensionsreduktion wiederholt sich das Verdünnungsparadox, und TurboQuant übernimmt erneut die Rolle des Wegbereiters niedrigbitiger Kompression.

4.4 Phase 4: Retrieval-Evaluation

Die Phasen 2 und 3 messen geometrische Strukturhaltung. Phase 4 prüft, ob sich diese Beobachtungen in tatsächlicher Suchqualität niederschlagen, gemessen an den BEIR-Relevanzurteilen. Für jede Query wird eine exakte Suche über alle zehn Repräsentationen durchgeführt und mit NDCG@10 als Primärmetrik sowie Recall@10, Recall@100 und MRR bewertet. Alle Werte sind über die drei Datensätze gemittelt. Die statistische Signifikanz jedes Vergleichs gegen Float32 wird über einen Wilcoxon-Signed-Rank-Test mit Bonferroni-Korrektur ($\alpha_{\text{korr}} \approx 0,0011$) bestimmt.

4.4.1 Retrieval-Qualität bei voller Dimensionszahl

Tabelle 4.6 zeigt die Retrieval-Metriken bei $d = 1024$.

Tabelle 4.6: Retrieval-Metriken bei $d = 1024$ (keine Dimensionsreduktion). Mittelwerte über drei Datensätze. Die relative Qualität bezieht sich auf NDCG@10 gegenüber Float32.

Repräsentation	NDCG@10	rel.	Recall@10	Recall@100	MRR
Float32	0,6424	100,0 %	0,469	0,616	0,719
Float16	0,6427	100,0 %	0,469	0,617	0,719
Naive 8-Bit	0,6425	100,0 %	0,469	0,617	0,719
TQ 8-Bit	0,6420	99,9 %	0,469	0,617	0,719
Naive 4-Bit	0,6380	99,3 %	0,469	0,617	0,717
TQ 4-Bit	0,6417	99,9 %	0,470	0,615	0,713
Naive 2-Bit	0,6245	97,2 %	0,454	0,609	0,709
TQ 2-Bit	0,6261	97,4 %	0,467	0,608	0,709
Naive 1-Bit	0,5755	89,6 %	0,429	0,580	0,681
TQ 1-Bit	0,5593	87,1 %	0,428	0,580	0,667

Qualitätsverlust erst bei 1-Bit deutlich. Bis hinunter zu 4-Bit ist der Retrieval-Qualitätsverlust vernachlässigbar: Float16, 8-Bit und TQ 4-Bit erreichen mindestens 99,9 % der Float32-NDCG. 2-Bit liegt mit etwa 97 % weiterhin nahe an der Referenz. Erst 1-Bit zeigt einen deutlichen Abfall auf 87–90 %. Bemerkenswert ist, dass dieser Qualitätsverlust milder ausfällt als der Rückgang der Nachbarschaftserhaltung aus Phase 3: Während der Overlap bei 1-Bit auf rund 60 % fiel, bleibt die NDCG@10 bei etwa 88 %. Beide Werte messen allerdings gegen unterschiedliche Referenzen: Der Overlap vergleicht gegen die Float32-Nachbarschaft, die NDCG@10 gegen die menschlichen Relevanzurteile. Da Float32 selbst kein perfektes Relevanzmaß ist, bedeutet eine verschobene Nachbarschaft nicht zwangsläufig einen Relevanzverlust; die durch Quantisierung ausgetauschten Nachbarn sind für die Suchaufgabe oft gleichwertig. Dies bestätigt die zentrale Beobachtung aus Phase 3: Quantisierung verliert einzelne Nachbarn, aber die verbleibenden Treffer sind relevant genug, um die Rankingqualität weitgehend zu erhalten.

Statistische Signifikanz. Der Wilcoxon-Test stützt dieses Bild. Die Unterschiede von Float16, 8-Bit, 4-Bit und in den meisten Fällen auch 2-Bit gegenüber Float32 sind nach Bonferroni-Korrektur nicht signifikant; ein Qualitätsunterschied zur Referenz ist statistisch nicht nachweisbar. Erst die 1-Bit-Varianten weichen deutlich ab: TQ 1-Bit ist über alle drei Datensätze signifikant schlechter als Float32, naive 1-Bit auf zwei der drei Datensätze (auf TREC-COVID wird die korrigierte Schwelle knapp verfehlt). Die Kompression auf 4-Bit oder 2-Bit verursacht damit keinen statistisch belegbaren Qualitätsverlust, während 1-Bit einen weitgehend query-robusten, echten

Verlust erzeugt.

Umkehrung des TurboQuant-Vorteils bei 1-Bit. Bei voller Dimensionszahl ist naive 1-Bit (89,6 %) geringfügig besser als TQ 1-Bit (87,1 %). Dies ist konsistent mit dem Befund aus Phase 2, dass die Rotation bei ohnehin binärer Quantisierung und voller Dimension keine Vorteile bringt und sogar leichte Bit-Korrelationen einführen kann. Für die Multi-Bit-Varianten sind naive und rotierte Version dagegen praktisch gleichwertig.

4.4.2 Einfluss der Dimensionsreduktion

Abbildung 4.12 zeigt die NDCG@10 als Funktion der PCA-Dimension.

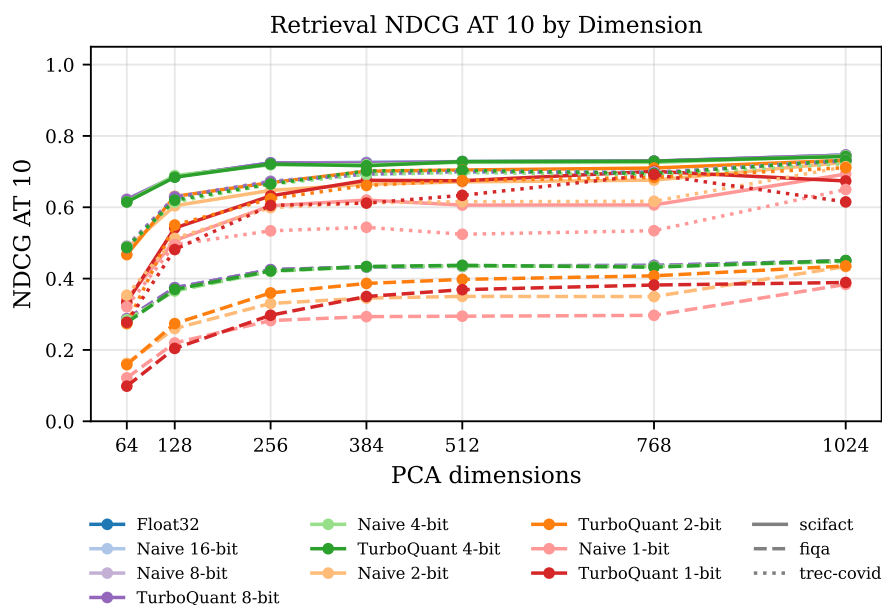


Abbildung 4.12: NDCG@10 als Funktion der PCA-Dimension d (Mittelwert über drei Datensätze). Höhere Bittiefen folgen der Float32-Kurve eng, während 1-Bit deutlich abfällt.

Bittiefe folgt der Float32-Referenz. TQ 4-Bit folgt der Float32-Kurve über den gesamten Dimensionsbereich nahezu deckungsgleich (etwa 0,460 bei $d = 64$ bis 0,642 bei $d = 1024$). Auch die Dimensionsreduktion selbst kostet Qualität: Die NDCG@10

der Float32-Referenz sinkt von 0,642 bei $d = 1024$ auf 0,465 bei $d = 64$. Dies bestätigt die Vorhersage aus Phase 1, dass eine Reduktion auf $d \leq 128$ die stärksten Einbrüche verursacht, während $d \geq 384$ nur moderate Verluste zeigt.

Verdünnungsparadox in abgeschwächter Form. Naive 1-Bit zeigt auch im Retrieval das aus den Phasen 2 und 3 bekannte Verdünnungsparadox, jedoch schwächer ausgeprägt. Die NDCG@10 steigt zunächst bis $d = 384$ auf 0,486, stagniert dann und springt bei $d = 1024$ auf 0,575. TQ 1-Bit dagegen steigt unter Dimensionsreduktion monoton und übertrifft naive 1-Bit bei allen reduzierten Dimensionen deutlich, etwa bei $d = 768$ mit 0,591 gegenüber 0,479. Erst bei voller Dimensionszahl kehrt sich dieses Vorteil um. Damit überträgt sich das Muster der geometrischen Analysen direkt auf die Suchqualität: TurboQuant ist unter Dimensionsreduktion der Wegbereiter niedrig-bitiger Kompression, während naive 1-Bit nur bei voller Dimension konkurrenzfähig ist.

4.4.3 Recall für zweistufige Systeme

Abbildung 4.13 zeigt Recall@100, die für zweistufige Retrieval-Systeme entscheidende Metrik, in denen eine erste Stufe eine Kandidatenmenge für ein späteres Reranking erzeugt. Der Recall@100 fällt unter Kompression deutlich schwächer ab als die NDCG@10. TQ 1-Bit hält bei $d = 768$ einen Recall@100 von 0,587 gegenüber 0,616 bei Float32 ($d = 1024$). Das bedeutet, dass die aggressive Kompression zwar die exakte Reihenfolge der Top-Treffer verändert, aber die relevanten Dokumente weiterhin in der breiteren Kandidatenmenge enthält. Für zweistufige Systeme, in denen ein präzises Reranking auf Float-Basis nachgeschaltet ist, bleibt selbst TQ 1-Bit damit brauchbar.

4.4.4 Pareto-Analyse: Qualität vs. Speicherbedarf

Abbildung 4.14 zeigt die Pareto-Front der NDCG@10 gegen den Speicherbedarf. Die Pareto-Front fasst den praktischen Trade-off zusammen. TQ 4-Bit ab mittleren Dimensionen bildet den qualitativ hochwertigen Arbeitspunkt mit nahezu unverändertem NDCG@10; TQ 2-Bit bietet einen ausgewogenen Kompromiss, und TQ 1-Bit besetzt den Bereich extremer Kompression. Diese Pareto-Front bewertet allerdings nur die Qualitätsseite; den zugehörigen Speicher- und Laufzeitpreis quantifiziert Phase 5, bevor Phase 6 beide Seiten zu konkreten Einsatzszenarien zusammenführt.

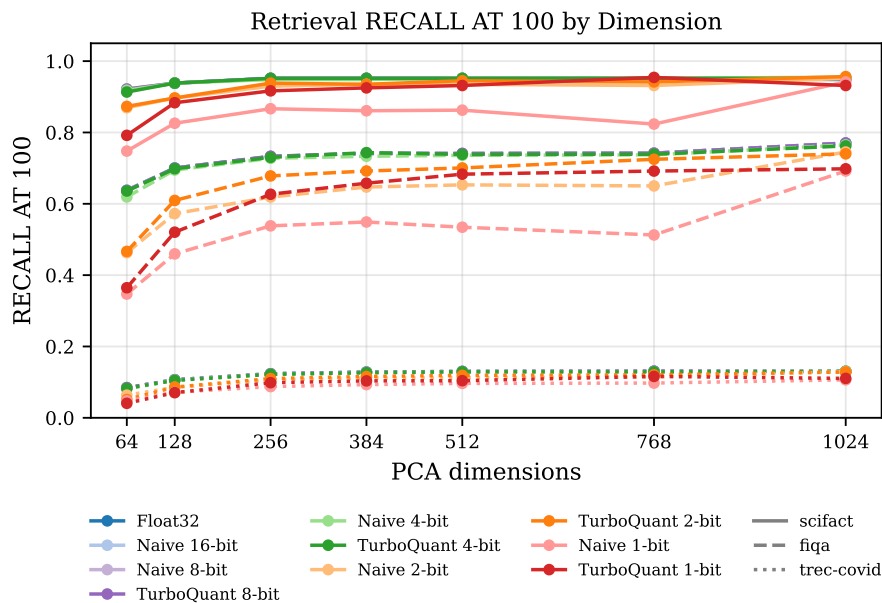


Abbildung 4.13: Recall@100 als Funktion der PCA-Dimension d (Mittelwert über drei Datensätze). Auch bei starker Kompression bleibt ein großer Teil der relevanten Dokumente in der Kandidatenmenge.

4.4.5 Diskussion und Bezug zu den vorherigen Phasen

Geschlossene Erklärungskette. Phase 4 schließt die Argumentationskette von der Raumgeometrie bis zur Suchqualität. Die günstigen Raumeigenschaften aus Phase 1 (Symmetrie, Normierung, niedrige lokale Dimensionalität) führten zu der Vorhersage, dass Quantisierung die Struktur weitgehend erhält. Die Phasen 2 und 3 bestätigten dies für Distanzen und Nachbarschaften, wobei die Nachbarschaftserhaltung das strengere Kriterium war. Phase 4 zeigt nun, dass die tatsächliche Retrieval-Qualität sogar robuster ist als die Nachbarschaftserhaltung vermuten ließ: Der für Retrieval entscheidende Faktor ist nicht, ob alle Nachbarn erhalten bleiben, sondern ob die verbleibenden relevant sind. Die durchgehende Übereinstimmung von Vorhersage und Messung über alle Phasen stützt die Erklärung, auch wenn sie einen kausalen Zusammenhang nicht beweist.

Praktische Konsequenz. Bis 4-Bit ist Quantisierung praktisch verlustfrei, ohne statistisch nachweisbaren Unterschied zu Float32, bei einem Kompressionsfaktor von 8. 2-Bit verursacht einen kleinen, meist nicht signifikanten Verlust bei Kompressionsfak-

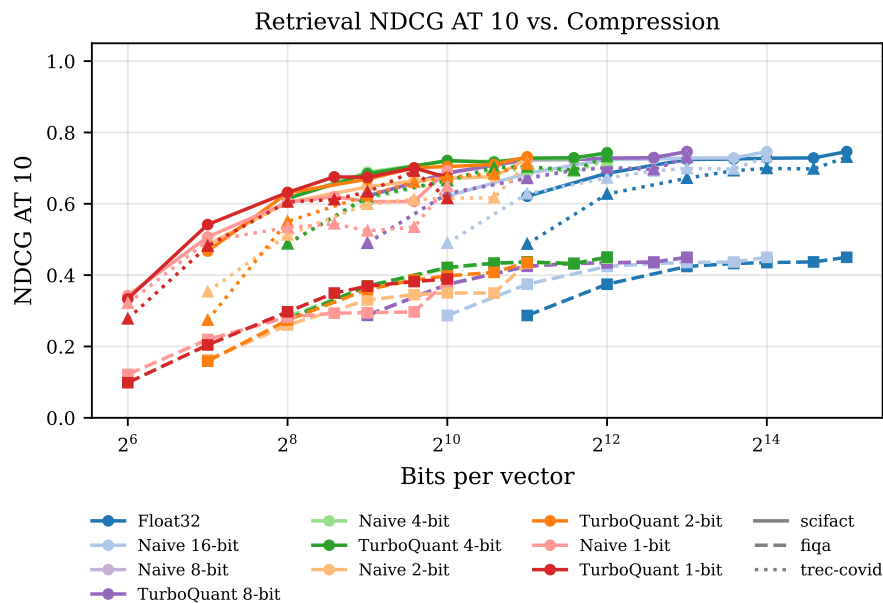


Abbildung 4.14: Pareto-Front der Retrieval-Qualität: NDCG@10 als Funktion des Speicherbedarfs (Bit pro Vektor). Konfigurationen auf der Front sind nicht dominiert.

tor 16. Erst 1-Bit erzeugt einen signifikanten, aber mit rund 88 % relativer Qualität keineswegs katastrophalen Abfall bei Kompressionsfaktor 32. Diese Werte gelten für das untersuchte Modell `bge-large-en-v1.5` und die exakte Brute-Force-Suche; unter anderen Modellen oder approximativer Indexierung können sie abweichen. Diese Qualitätsstufen erhalten ihren Preis jedoch erst durch den zugehörigen Ressourcenbedarf, den Phase 5 misst, bevor Phase 6 beide Seiten zu konkreten Einsatzkontexten verbindet.

Zusammenfassung. Phase 4 bestätigt die geometrischen Befunde auf der Ebene der Suchqualität. Der Qualitätsverlust ist bis 2-Bit gering und statistisch meist nicht belegbar, während 1-Bit einen signifikanten, aber moderaten Abfall zeigt. Unter Dimensionsreduktion bleibt TurboQuant der entscheidende Faktor für niedrigbitige Kompression, und der hohe Recall@100 belegt die Eignung stark komprimierter Repräsentationen als erste Stufe zweistufiger Systeme.

4.5 Phase 5: Speicher- und Laufzeiteffizienz

Die Phasen 2 bis 4 bewerten, wie gut die Kompression die semantische Struktur erhält. Phase 5 wechselt die Perspektive und misst den praktischen Preis der Kompression: Speicherbedarf und Laufzeit im tatsächlich implementierten NumPy-Pfad, zusammengeführt mit der Retrieval-Qualität aus Phase 4 zu Trade-off-Kennzahlen. Damit adressiert Phase 5 die zweite der beiden Hauptzielgrößen der Forschungsfrage, den Speicherverbrauch, und beantwortet insbesondere die Teilfrage nach dem Verhältnis von Speicherersparnis zu Qualitätsverlust. Die Messungen erfolgen auf kontrollierten Korpusschnitten von 1 000 Dokumenten je Datensatz über die gesamte Dimensionsmatrix $d \in \{64, 128, 256, 384, 512, 768, 1024\}$. Sofern nicht anders angegeben, beziehen sich die berichteten Werte auf die volle Dimensionszahl $d = 1024$ und sind über die drei Datensätze gemittelt. Es ist zu betonen, dass die Laufzeitzahlen eine Implementierungsmessung des NumPy-Referenzpfads darstellen (vgl. Abschnitt 3.9) und keine Aussage über einen optimalen nativen Index treffen.

4.5.1 Speicherbedarf

Tabelle 4.7 stellt den theoretischen Speicherbedarf bei vollständiger Bitpackung dem tatsächlich gemessenen NumPy-Layout gegenüber.

Tabelle 4.7: Speicherbedarf pro Vektor bei $d = 1024$ und $N = 1\,000$ Vektoren. Der theoretische Wert beschreibt die ideale Bitpackung, das NumPy-Layout den tatsächlich gemessenen Verbrauch inklusive Dequantisierungsmetadaten. Der Kompressionsfaktor (CR) bezieht sich auf Float32 bei $d = 1024$. Die Werte sind über alle drei Datensätze identisch.

Repräsentation	Byte/Vektor (theor.)	CR (theor.)	Byte/Vektor (NumPy)	CR (NumPy)
Float32	4096,0	1,0	4096,0	1,0
Float16	2048,0	2,0	2048,0	2,0
8-Bit	1024,0	4,0	1040,4	3,9
4-Bit	512,0	8,0	528,4	7,8
2-Bit	256,0	16,0	272,4	15,0
1-Bit	128,0	32,0	128,0	32,0

Metadaten-Overhead als fixe Indexkosten. Der Speicherbedarf skaliert bei allen Verfahren linear mit der Vektoranzahl, sodass die Bittiefe den dominierenden Faktor

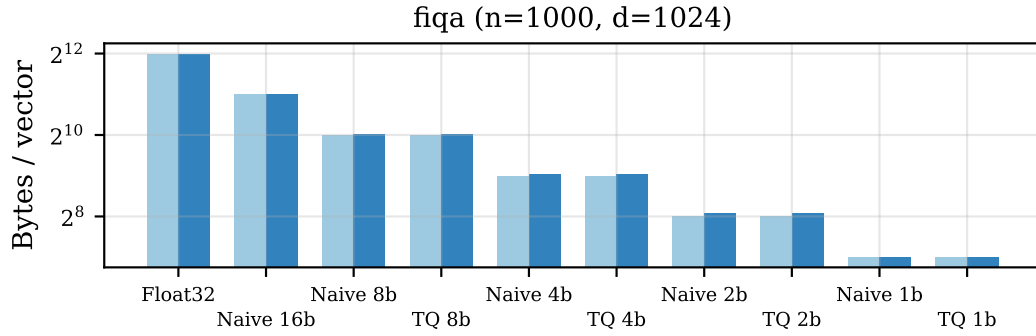
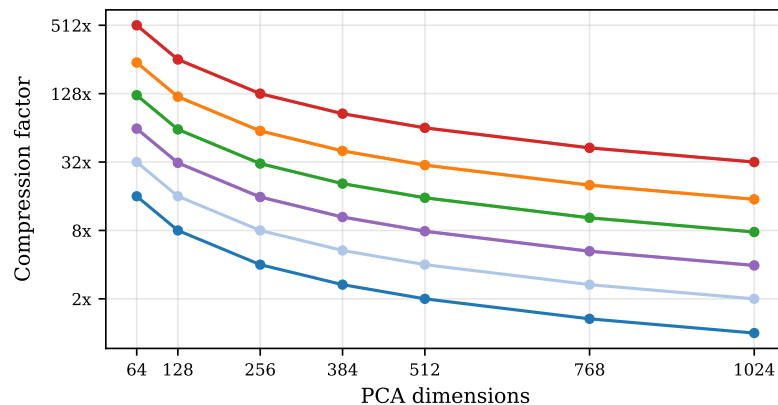


Abbildung 4.15: Theoretischer gegenüber praktisch gemessenem Speicherbedarf pro Repräsentation (exemplarisch für $d = 1024$; die Werte sind über alle Datensätze identisch). Das NumPy-Layout liegt für alle Verfahren nahe an der idealen Bitpackung; die kleine Abweichung bei 2-Bit bis 8-Bit entsteht durch die Dequantisierungsmetadaten.

bildet. Die einzige Abweichung von der idealen Bitpackung stammt aus den Quantisierungsmetadaten `col_min` und `col_max`, die für die 2-Bit- bis 8-Bit-Dequantisierung benötigt werden. Diese Metadaten sind mit $2 \times d \times 8$ Byte (hier 16 384 Byte bei $d = 1024$) eine *feste* Größe pro Index, unabhängig von der Vektoranzahl. Bei den hier gemessenen $N = 1000$ Vektoren verteilen sie sich auf etwa 16,4 Byte pro Vektor und drücken den Kompressionsfaktor von 8-Bit sichtbar von theoretisch 4,0 auf 3,9 sowie von 2-Bit von 16,0 auf 15,0. Da dieser Overhead fix ist, verschwindet er bei größeren Indizes nahezu vollständig; genau diese Amortisierung liegt den in Phase 6 verwendeten Byte-pro-Vektor-Werten (etwa 272,4 für 2-Bit) zugrunde. Die 1-Bit-Repräsentation benötigt keine solchen Metadaten und erreicht daher exakt den theoretischen Kompressionsfaktor von 32.

Zusammenwirken von Bittiefe und Dimensionsreduktion. Abbildung 4.16 zeigt den Kompressionsfaktor als Funktion der Dimensionszahl. Beide Kompressionsachsen multiplizieren sich: Die Reduktion von 1024 auf 64 Dimensionen liefert einen zusätzlichen Faktor 16 über der reinen Bittiefenkompression. 1-Bit bei $d = 64$ erreicht damit rechnerisch einen Kompressionsfaktor von mehreren hundert gegenüber der Float32-Referenz bei $d = 1024$. Ob eine solche aggressive Konfiguration brauchbar bleibt, entscheidet jedoch nicht Phase 5, sondern die Qualitätsanalyse der Phasen 2 bis 4.



* Hinweis: Nur TREC-COVID die anderen TurboQuant 8-bit (QA) Naive 2-bit TurboQuant 2-bit Naive 4-bit TurboQuant 4-bit Naive 8-bit TurboQuant 8-bit Naive 16-bit TurboQuant 16-bit tree-covid (n=1000)

Abbildung 4.16: Kompressionsfaktor gegenüber Float32 bei $d = 1024$ als Funktion der PCA-Dimension. Kleinere Dimensionen und niedrigere Bittiefen wirken multiplikativ; 1-Bit liegt durchgehend oben, Float32 bildet die Baseline.

4.5.2 Laufzeit

Tabelle 4.8 zeigt die Medianlaufzeiten und den Durchsatz für die beiden gemessenen Operationen: paarweise Distanzberechnung über 10 000 Paare und Brute-Force-Top- k -Suche mit $k = 10$ über den Korpusausschnitt.

Paarweise Distanz: 1-Bit dominiert. Bei der paarweisen Distanzberechnung ist die 1-Bit-Repräsentation mit Abstand am schnellsten. Sie erreicht mit rund 3,4 ms einen Durchsatz von etwa 2,9 Millionen Paaren pro Sekunde und liegt damit fast eine Größenordnung über Float32 (506 000 Paare/s). Die Ursache ist die Distanzmetrik selbst: Die Hamming-Distanz zweier Bitvektoren reduziert sich auf ein XOR gefolgt von einem Popcount, beides billige bitweise Operationen. Die Multi-Bit-Verfahren (2-Bit bis 8-Bit) sind hingegen langsamer als Float32, weil vor der Distanzberechnung entpackt und dequantisiert werden muss. Naive und rotierte Varianten sind hier praktisch gleich schnell, da die Rotation nur den Indexaufbau betrifft und nicht zur Suchlaufzeit zählt.

Tabelle 4.8: Laufzeit im NumPy-Pfad bei $d = 1024$ und $N = 1\,000$, gemittelt über drei Datensätze. Pairwise bezieht sich auf 10 000 Paare, Top- k auf die vollständige Nachbarschaftssuche über den Korpusausschnitt.

Repräsentation	Pairwise-Distanz		Top- k ($k = 10$)	
	Median (ms)	Paare/s	Median (ms)	Queries/s
Float32	33,7	506 000	24,7	40 400
Float16	45,7	219 000	4457,8	224
TQ 8-Bit	40,0	250 000	80,4	12 400
TQ 4-Bit	40,7	246 000	82,6	12 100
TQ 2-Bit	40,7	246 000	80,2	12 500
Naive 1-Bit	3,4	2 936 000	481,5	2 386
TQ 1-Bit	3,4	2 943 000	343,4	2 912

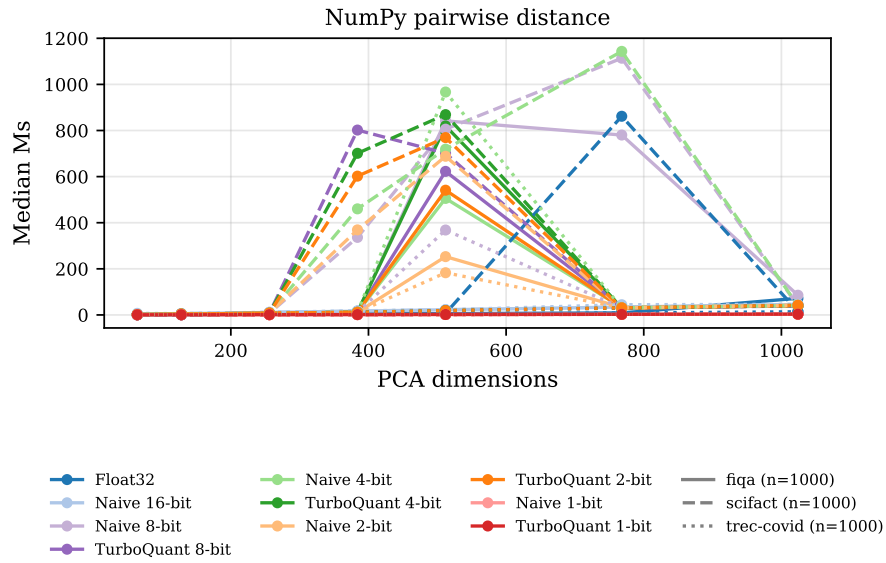


Abbildung 4.17: Laufzeit der paarweisen Distanzberechnung (10 000 Paare) als Funktion der Dimension. Ab mittleren Dimensionen ist 1-Bit klar am schnellsten und zieht mit wachsender Dimension weiter davon, während die Multi-Bit-Verfahren durch Entpacken und Dequantisierung langsamer sind als Float32. Bei sehr kleinen Dimensionen liegen 1-Bit und Float32 nahezu gleichauf.

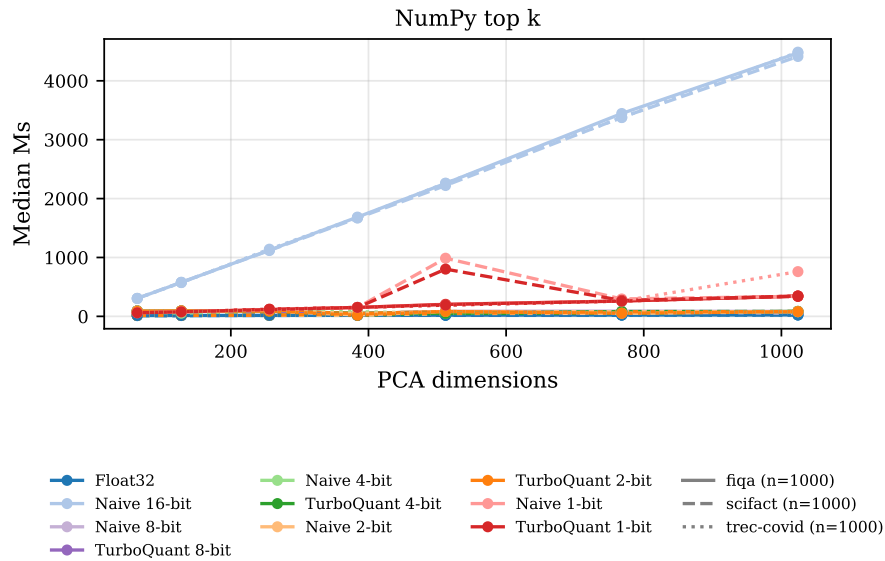


Abbildung 4.18: Laufzeit der Brute-Force-Top- k -Suche als Funktion der Dimension. In diesem NumPy-Pfad ist Float32 durchgehend am schnellsten, da dichte Matrixmultiplikation optimal auf BLAS abgebildet wird.

Top- k -Suche: Float32 dominiert. Bei der Brute-Force-Top- k -Suche kehrt sich das Bild um. Float32 ist hier mit rund 24,7 ms (etwa 40 000 Queries/s) das schnellste Verfahren, während alle komprimierten Varianten deutlich langsamer sind. Der Grund liegt in der Struktur der Operation: Die Top- k -Suche ist im Kern eine dichte Matrixmultiplikation, für die NumPy und BLAS hochgradig optimierte Routinen bereitstellen. Die komprimierten Varianten können diesen Pfad nicht nutzen, sondern müssen entpacken (Multi-Bit) oder über Broadcast-XOR und Lookup-Table-Popcount arbeiten (1-Bit), was den BLAS-Vorteil überkompensiert. Besonders auffällig ist Float16 mit 4458 ms: NumPy besitzt keine schnelle Float16-BLAS-Routine, sodass diese Repräsentation zur reinen Speicheroption wird und für die Suche unbrauchbar ist.

Diskrepanz zwischen den Operationen. Der scheinbare Widerspruch, dass 1-Bit bei paarweiser Distanz dominiert, bei der Top- k -Suche aber zu den langsamsten Verfahren gehört, ist ein Implementierungsartefakt. Die paarweise Messung nutzt die native Effizienz bitweiser Operationen aus, während die Top- k -Messung gegen hochoptimierte dichte Matrixmultiplikation antritt, für die es im NumPy-Pfad keine gleichwertige binäre Entsprechung gibt. Ein nativer Binärindex mit Hardware-Popcount würde hier voraussichtlich anders abschneiden; die vorliegende Messung isoliert bewusst den

reproduzierbaren NumPy-Referenzpfad und trifft daher keine Aussage über theoretisch mögliche Bestwerte niedrigbitiger Verfahren.

4.5.3 Qualitäts-Speicher-Trade-off

Speicher und Laufzeit beschreiben nur die Kostenseite. Um sie mit der Retrieval-Qualität aus Phase 4 zusammenzuführen, werden drei zusammenfassende Trade-off-Metriken berechnet (vgl. Abschnitt 3.9), jeweils relativ zur Float32-Referenz bei $d = 1024$ auf SciFact. Die *Relative Quality-to-Space Ratio* (RQSR) setzt die relative Qualität ins Verhältnis zum relativen Speicher und misst damit die Qualität pro Speichereinheit. Der *Compression-Quality-Score* CQ_1 bildet das harmonische Mittel aus relativer Qualität und relativer Speicherersparnis und ist genau dann hoch, wenn beide Größen gleichzeitig hoch sind. Die *Quality Elasticity of Storage* (QES) gibt den Qualitätsverlust pro Einheit gewonnener Speicherersparnis an; kleine Werte sind günstig. Das zentrale

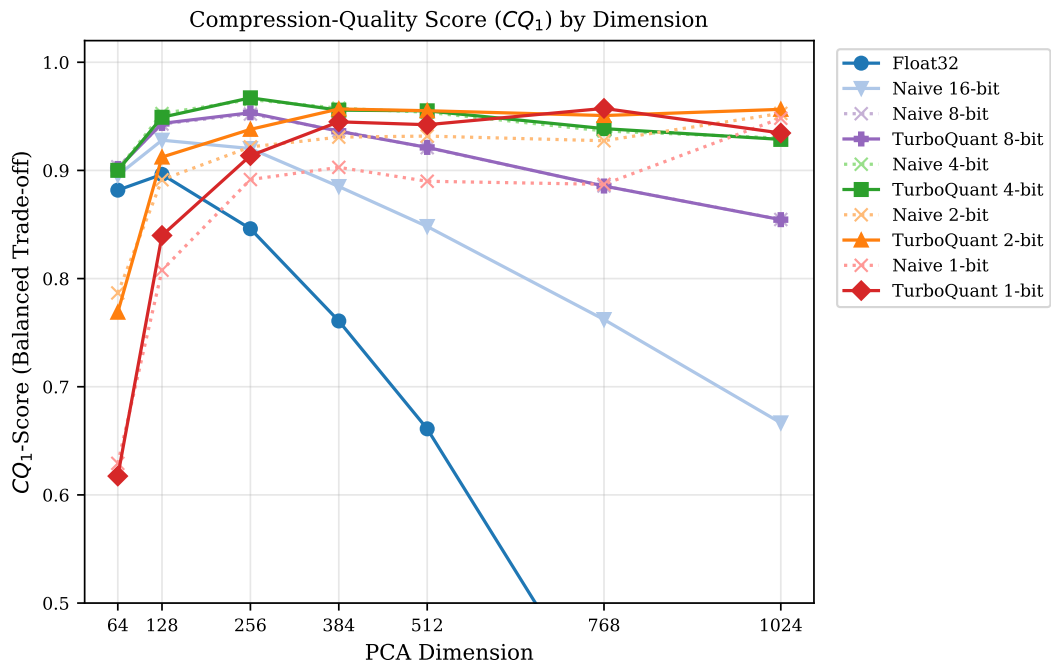


Abbildung 4.19: Compression-Quality-Score CQ_1 als Funktion der PCA-Dimension. Das Maximum liegt nicht bei voller Dimensionszahl, sondern bei mittleren Dimensionen, da dort Qualität und Speicherersparnis gemeinsam hoch sind.

Ergebnis dieser Metriken ist, dass der beste kombinierte Arbeitspunkt *nicht* bei voller

Dimensionszahl liegt. Nach CQ_1 schneiden dimensionsreduzierte Konfigurationen am besten ab, etwa TQ 4-Bit bei $d = 256$ ($CQ_1 \approx 0,97$) sowie TQ 1-Bit bei $d = 768$ und TQ 2-Bit bei $d = 384$ (je $CQ_1 \approx 0,96$). Diese Punkte behalten über 93 % der Float32-Qualität und geben zugleich einen Großteil des Speichers frei. Die RQSR verschiebt den Fokus stärker auf reine Speichereffizienz und favorisiert entsprechend die aggressivsten Konfigurationen, während die QES bestätigt, dass der Qualitätsverlust pro gewonnener Speichereinheit bei den hochbitigen Verfahren am kleinsten bleibt. Diese Metriken ersetzen NDCG@10 nicht, sondern verdichten den Trade-off zu einer einzelnen Kennzahl und dienen als Entscheidungshilfe zwischen ähnlich guten Konfigurationen. Dass CQ_1 mittlere Dimensionen bevorzugt, während Phase 6 für die Cloud-Szenarien die volle Dimensionszahl wählt, ist kein Widerspruch, sondern eine Frage der Gewichtung: CQ_1 und RQSR belohnen jede eingesparte Speichereinheit, während Phase 6 den Speicher als weiches Budget behandelt und innerhalb dieses Budgets die höchste Qualität priorisiert. Die Übersetzung dieser Kennzahlen in konkrete, an Speicher- und Kostenbudgets ausgerichtete Arbeitspunkte erfolgt daher in Phase 6.

4.5.4 Diskussion und Bezug zu den vorherigen Phasen

Entkopplung von Speicher und Laufzeit. Die zentrale Erkenntnis aus Phase 5 ist, dass Speicherersparnis nicht automatisch Laufzeitgewinn bedeutet. Die Kompression senkt den Speicherbedarf robust und vorhersagbar um bis zu Faktor 32 (1-Bit) allein über die Bittiefe, multipliziert mit dem Faktor der Dimensionsreduktion. Die Laufzeit folgt jedoch einer anderen Logik: Sie wird im gemessenen Pfad davon bestimmt, wie gut eine Operation auf optimierte NumPy- und BLAS-Routinen abgebildet werden kann, nicht von der reinen Bitanzahl. Der praktische Nutzen der Kompression liegt damit primär im Speicher, nicht in der Suchgeschwindigkeit des Referenzpfads.

Einordnung des Trade-offs. Phase 5 bewertet für sich genommen keine Repräsentation abschließend, sondern liefert die Kostenseite der Abwägung. Die Qualitätsseite stammt aus den Phasen 2 bis 4, die zeigen, dass die Kompression bis 4-Bit praktisch verlustfrei bleibt. Zusammengeführt ergibt sich das Bild, das Phase 6 in konkrete Arbeitspunkte überführt: Die Kompression ist als Speicheroptimierung uneingeschränkt attraktiv, während die Laufzeitwahl vom konkreten Zugriffsmuster und einer möglichen nativen Implementierung abhängt.

Zusammenfassung. Phase 5 zeigt einen klaren, aber differenzierten Trade-off. Der Speicherbedarf sinkt vorhersagbar mit Bittiefe und Dimensionszahl, wobei nur die 1-Bit-Repräsentation den idealen Kompressionsfaktor ohne Metadaten-Overhead erreicht. Bei der Laufzeit gewinnt keine Repräsentation über alle Operationen: 1-Bit ist bei

paarweiser Distanz führend, Float32 bei der Brute-Force-Top- k -Suche. Die zusammenfassenden Trade-off-Metriken zeigen zudem, dass der kombiniert beste Arbeitspunkt bei mittleren Dimensionen liegt, nicht bei voller Dimensionszahl. Die gemessenen Werte sind als Implementierungsmessung des NumPy-Pfads zu lesen und liefern die Speichergrundlage für die praktische Skalierungsanalyse in Phase 6.

4.6 Phase 6: Praktische Anwendungsfall- und Skalierungsanalyse

Phase 6 führt die Ergebnisse der vorherigen Phasen in eine praktische Entscheidungsperspektive über. Während Phase 4 die Retrieval-Qualität und Phase 5 den Ressourcenbedarf isoliert messen, fragt Phase 6, welche TurboQuant-Konfigurationen für konkrete Einsatzkontexte sinnvoll sind. Die zentrale Beobachtung ist, dass die stärksten Speichergewinne nicht erst bei sehr großen Cloud-Indizes relevant werden, sondern bereits im Bereich mobiler und lokaler Umgebungen.

4.6.1 Szenarioüberblick

Tabelle 4.9 fasst die drei ausgewählten Anwendungsszenarien zusammen. Die relative Qualität bezieht sich auf NDCG@10 auf SciFact im Vergleich zur Float32-Referenz bei $d = 1024$. Anders als die über drei Datensätze gemittelten Werte der Phase 4 (Float32-NDCG@10 $\approx 0,64$) beziehen sich die folgenden Absolutwerte allein auf SciFact (Float32-NDCG@10 = 0,7463); die höheren Zahlen entstehen also durch die datensatzspezifische Auswertung, nicht durch einen methodischen Widerspruch. Die

Tabelle 4.9: Praktische TurboQuant-Szenarien in Phase 6. Die Speicherwerte beschreiben den reinen Vektorspeicher ohne Metadaten, Textchunks oder ANN-Zusatzstrukturen.

Szenario	Konfiguration	NDCG@10	Qualität	Byte/Vektor	1M	10M
Float32-Referenz	float32 1024d	0,7463	100,0 %	4096,0	3,81 GiB	38,15 GiB
Edge/Mobile	tq_1bit 768d	0,7007	93,9 %	96,0	91,6 MiB	0,89 GiB
Business	tq_2bit 1024d	0,7318	98,1 %	272,4	259,8 MiB	2,54 GiB
Enterprise	tq_4bit 1024d	0,7422	99,5 %	528,4	503,9 MiB	4,92 GiB

Float32-Referenz benötigt bei 10 Millionen Vektoren 38,15 GiB reinen Vektorspeicher. Schon die konservative Enterprise-Konfiguration **tq_4bit 1024d** reduziert diesen Wert auf 4,92 GiB, ohne die gemessene Retrieval-Qualität nennenswert zu verändern. Der

Qualitätsverlust beträgt nur etwa 0,5 Prozentpunkte relativ zur Float32-Referenz. Das Business-Szenario verschiebt den Arbeitspunkt aggressiver: **tq_2bit 1024d** benötigt bei 10 Millionen Vektoren nur 2,54 GiB und hält dennoch 98,1 % der Float32-Qualität. Für kostenempfindliche Cloud-Suche ist dies der plausibelste Kompromiss, weil der Speicherbedarf um den Faktor 15 sinkt, während der Qualitätsverlust mit 1,9 % klein bleibt. Das Edge/Mobile-Szenario verwendet **tq_1bit 768d**. Diese Konfiguration ist nicht für maximale Qualität optimiert, sondern für minimale Speichernutzung bei noch brauchbarer Retrieval-Leistung. Mit 96 Byte pro Vektor erreicht sie einen Kompressionsfaktor von 42,7 gegenüber Float32 und erhält 93,9 % der Float32-Qualität.

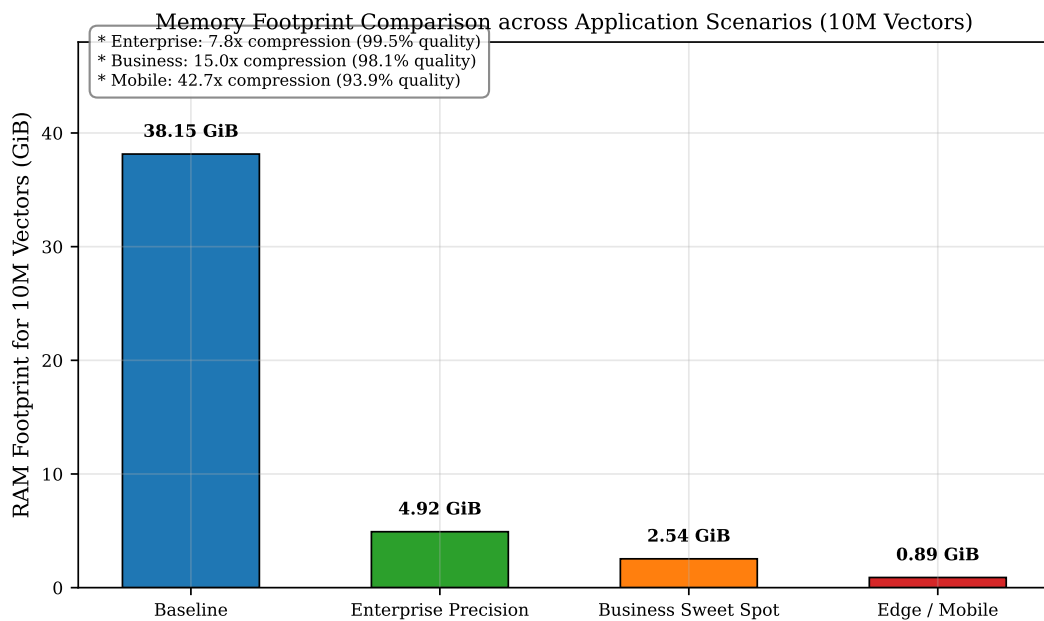


Abbildung 4.20: Speicherbedarf der Phase-6-Szenarien bei 10 Millionen Vektoren. Die Abbildung zeigt, dass alle TurboQuant-Arbeitspunkte den Float32-Index um eine Größenordnung oder mehr reduzieren.

4.6.2 Mobile Skalierung unterhalb von 10 Millionen Vektoren

Für lokale Anwendungen ist ein 10-Millionen-Vektorindex eher eine Obergrenze als ein typischer Fall. Deshalb wird das Edge/Mobile-Szenario zusätzlich unterhalb dieser Grenze betrachtet. Tabelle 4.10 zeigt den Einfluss der Dimensionszahl für **tq_1bit**. Die Tabelle zeigt zwei unterschiedliche Regime. Bei sehr harten Speichergrenzen sind

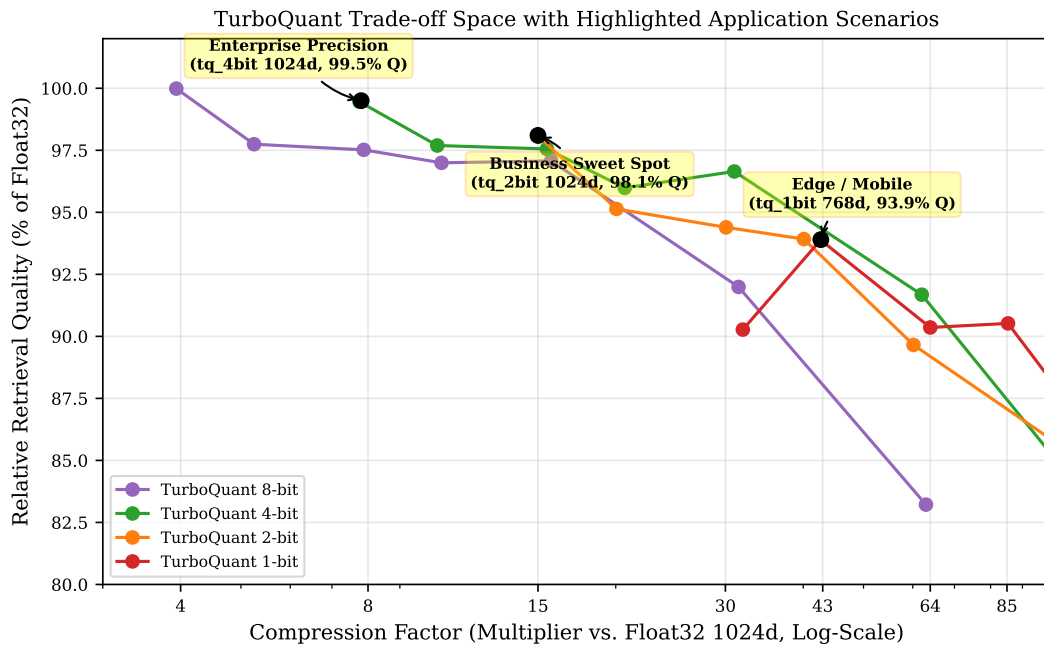


Abbildung 4.21: Qualitäts-Speicher-Trade-off der TurboQuant-Konfigurationen mit den drei Anwendungsszenarien. Die markierten Punkte zeigen, dass die praktische Auswahl nicht entlang einer einzelnen optimalen Konfiguration erfolgt, sondern vom Anwendungskontext abhängt.

Tabelle 4.10: Dimensionsvergleich für tq_1bit im mobilen Szenario. Die Qualität bezieht sich auf NDCG@10 relativ zur Float32-1024d-Referenz.

Dimension	Byte/Vektor	RAM bei 100k	RAM bei 500k	Qualität
64d	8	0,76 MiB	3,8 MiB	44,7 %
128d	16	1,53 MiB	7,6 MiB	72,6 %
256d	32	3,05 MiB	15,3 MiB	84,7 %
384d	48	4,58 MiB	22,9 MiB	90,5 %
512d	64	6,10 MiB	30,5 MiB	90,4 %
768d	96	9,16 MiB	45,8 MiB	93,9 %
1024d	128	12,21 MiB	61,0 MiB	90,3 %

384d und 512d attraktiv: Selbst 500 000 Vektoren bleiben deutlich unter 64 MiB, während die relative Qualität noch bei etwa 90 % liegt. Der beste gemessene mobile Arbeitspunkt ist jedoch 768d. Er benötigt bei 1 Million Vektoren 91,6 MiB und erreicht 93,9 % der Float32-Qualität. Die Erhöhung auf 1024d bringt im SciFact-Ergebnis keinen zusätzlichen Qualitätsgewinn, erhöht aber den Speicherbedarf auf 122,1 MiB pro 1 Million Vektoren. Auffällig ist, dass die Qualität nicht streng monoton mit der Dimension steigt: 512d liegt mit 90,4 % minimal unter 384d (90,5 %), und 1024d liegt mit 90,3 % unter 768d (93,9 %). Diese kleinen Nichtmonotonien sind für eine einzelne Bittiefe auf einem einzelnen Datensatz zu erwarten, da die 1-Bit-Quantisierung nach PCA-Reduktion je nach beibehaltenem Varianzanteil unterschiedlich stark von Vorzeichenrauschen betroffen ist; die Abweichungen liegen im Bereich weniger Zehntel Prozentpunkte und sind daher als Messrauschen und nicht als systematischer Effekt zu werten. Abbildung 4.22 erweitert diese Sicht auf alle TurboQuant-Bittiefen bis 8 Bit. Für jede Bittiefe werden die Dimensionen als gruppierte Balken über den betrachteten Indexgrößen dargestellt. Die Balkenhöhen wachsen linear mit der Vektoranzahl und mit der Dimension; die Legende ergänzt den Speicherblick um die jeweilige Retrieval-Qualität. Für das mobile Szenario ist die wichtigste Erkenntnis, dass der reine Vektorspeicher unterhalb von 1 Million Embeddings nicht der dominante Engpass sein muss. Bei `tq_1bit 768d` belegt ein Index mit 100 000 Vektoren nur 9,16 MiB, 500 000 Vektoren belegen 45,8 MiB und 1 Million Vektoren 91,6 MiB. Praktische Grenzen entstehen daher eher durch Zusatzstrukturen: ANN-Indizes, Text- und Metadatenhaltung, App-Runtime und Speicherlimits des Betriebssystems. Der Bereich von 100 000 bis 500 000 Embeddings erscheint als realistischer On-Device-Korridor. Diese Aussage beschreibt jedoch nur den reinen Vektorspeicher; sie ersetzt keine gerätespezifische Speicherbudget-Analyse für ein konkretes mobiles Betriebssystem oder eine konkrete App.

4.6.3 Cloud-Kosten und betriebliche Relevanz

Für Cloud-Szenarien erlaubt die Speicherreduktion in der modellierten Rechnung den Wechsel in kleinere Instanzklassen. Tabelle 4.11 zeigt die modellierte Kostenwirkung für einen 10-Millionen-Vektorindex. Die konkreten Stundenpreise stammen aus der regionalen AWS-Preislisten-CSV [2]: `r6i.2xlarge` mit 0,6080 USD/h, `m6g.large` mit 0,0920 USD/h und `t4g.medium` mit 0,0384 USD/h. Die Monatsbeträge sind keine separaten AWS-Monatsangebote, sondern diese Stundenpreise multipliziert mit 730 Stunden; die RAM-Ausstattung der Instanzen stammt aus den AWS-Instanzspezifikationen [5, 6]. Die Kostenrechnung ist nicht als universelle Preisprognose zu verstehen, sondern als regions- und zeitabhängige Größenordnung. Sie zeigt dennoch eine klare betriebliche Konsequenz: `tq_2bit 1024d` reduziert den modellierten Monatsbetrag von 443,84 USD auf 28,03 USD. Das entspricht einer Ersparnis von 415,81 USD pro

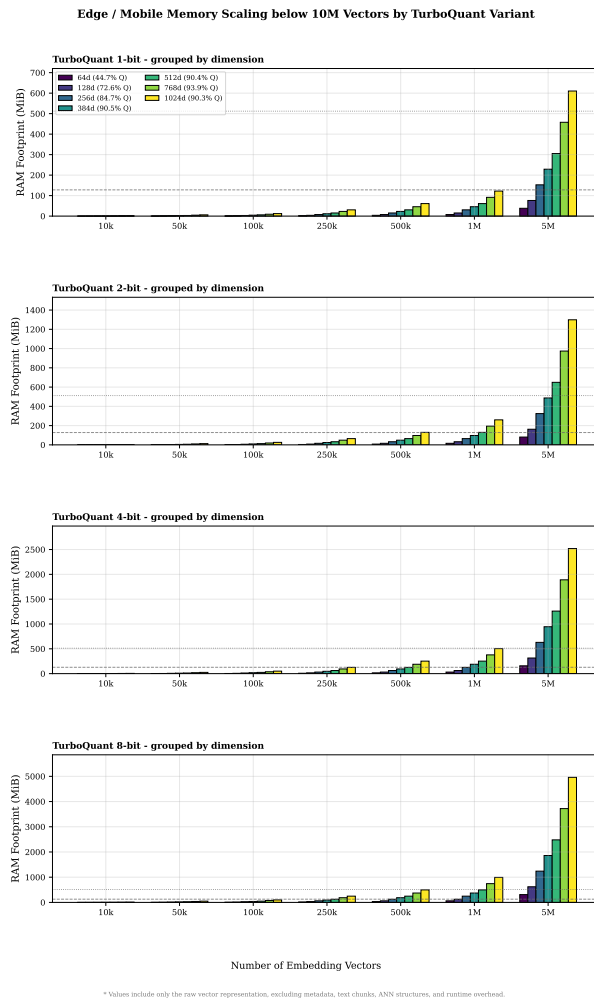
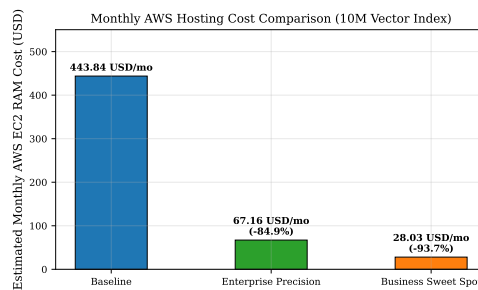


Abbildung 4.22: Mobile Speicherskalierung für TurboQuant unterhalb von 10 Millionen Vektoren. Jede Teilgrafik entspricht einer Bittiefe; innerhalb einer Vektoranzahl stehen die Balken für die evaluierten Dimensionszahlen.

Monat beziehungsweise rund 4990 USD pro Jahr für einen einzelnen 10-Millionen-Vektorindex. Bei mehreren Indizes oder mandantengetrennten Deployments skaliert dieser Effekt entsprechend. Zwei Einschränkungen sind bei der Interpretation dieser Zahlen zu beachten. Erstens entsteht die Ersparnis nicht allein durch den geringeren Speicherbedarf, sondern zusätzlich durch einen Wechsel der Instanzfamilie: Die Float32-Referenz wird einer speicheroptimierten `r6i`-Instanz zugeordnet, während die komprimierten Arbeitspunkte in günstigere Allzweck- beziehungsweise Graviton-

Tabelle 4.11: Grobe AWS-Kostenrechnung für einen 10-Millionen-Vektorindex. Grundlage sind EC2-On-Demand-Listenpreise (Linux) aus der AWS Price List Bulk API für die Region eu-central-1, abgerufen am 25.08.2026 und veröffentlicht am 24.08.2026, umgerechnet mit 730 Stunden pro Monat gemäß AWS-Pricing-Calculator-Annahme [2, 3]. Die Werte sind als On-Demand-Größenordnung zu interpretieren.

Szenario	Instanz	RAM	Kosten/Monat	Ersparnis
Float32	r6i.2xlarge	64 GiB	443,84 USD	–
Enterprise	m6g.large	8 GiB	67,16 USD	84,9 %
Business	t4g.medium	4 GiB	28,03 USD	93,7 %



* AWS EC2 Estimates for Cloud Hosting (10M Vector Index): Baseline (r6i.2xlarge, 443.84 USD/mo), Enterprise (m6g.large, 67.16 USD/mo), Business (t4g.medium, 28.03 USD/mo). Linux On-Demand list prices, eu-central-1, retrieved 2026-08-25 (730 h/month).

Abbildung 4.23: Geschätzte monatliche AWS-Kosten für einen 10-Millionen-Vektorindex. Die Kostenreduktion entsteht nicht durch schnellere Berechnung, sondern dadurch, dass der geringere Speicherbedarf sowohl eine kleinere Instanz als auch einen Wechsel von einer speicher-optimierten zu einer günstigeren Allzweck- beziehungsweise Graviton-Instanz erlaubt.

Instanzen (**t4g**, **m6g**) passen. Ein Teil der prozentualen Ersparnis geht damit auf die andere Preisstruktur der Instanzfamilie zurück und nicht ausschließlich auf die reine Speicherreduktion. Zweitens ist die Zuordnung der Float32-Referenz zur speicherstarken **r6i.2xlarge** eine bewusst konservative Annahme, die das obere Ende der darstellbaren Ersparnis markiert; eine andere Baseline-Wahl würde die prozentuale Differenz verringern. Die berichteten Prozentwerte sind daher als plausibles Best-Case-Szenario der Betriebskostenreduktion zu lesen, nicht als garantierter Mindestwert. Auf eine Laufzeit- oder Latenzbetrachtung verzichtet Phase 6 bewusst. Phase 5 hat gezeigt, dass die Kompression im gemessenen NumPy-Pfad primär einen Speicher- und keinen Geschwindigkeitsvorteil bietet; die betriebliche Relevanz von Phase 6 liegt

daher konsequent auf der Speicher- und der daraus abgeleiteten Kostenseite.

4.6.4 Einordnung

Phase 6 bestätigt die praktische Relevanz der in den vorherigen Phasen gemessenen Kompressionseffekte. Für hohe Qualitätsanforderungen ist **tq_4bit 1024d** die konservative Wahl: Der Speicherbedarf sinkt um den Faktor 7,8, während die gemessene Qualität praktisch erhalten bleibt. Für kostenoptimierte Cloud-Suche ist **tq_2bit 1024d** der stärkere Arbeitspunkt, weil der Speicher um den Faktor 15 sinkt und die Qualität bei 98,1 % bleibt. Für lokale Suche ist **tq_1bit 768d** besonders interessant, weil es unterhalb von 1 Million Vektoren in einem Speicherbereich liegt, der für mobile und lokale Anwendungen plausibel erscheint. Die naive Quantisierung wird in dieser Phase nicht weiter ausgewertet, weil sie bei gleicher Bittiefe und Dimensionszahl keine andere Speicherrechnung erzeugt. Die Abwägung zwischen naiver Quantisierung und TurboQuant ist damit keine Frage des Speichers, sondern der Qualitätsrobustheit. Für die praktische Empfehlung ist TurboQuant deshalb der relevantere Kandidat: Es bietet denselben Speicherbedarf wie die naive Variante, vermeidet aber die in Phase 2 beobachteten Instabilitäten bei niedrigen Bittiefen und Dimensionsreduktion.

4.7 Synthese: Beantwortung der Forschungsfrage

Die sechs Phasen ergeben zusammen ein kohärentes Bild. Dieser Abschnitt bündelt die verteilten Befunde zunächst entlang der vier in der Methodik formulierten Leitfragen (Was bleibt erhalten? Was geht verloren? Was bedeutet das für die Praxis? Welche Konfiguration ist sinnvoll?) und bindet sie anschließend explizit an die vier in Abschnitt 1.4 gestellten Teilfragen der Forschungsfrage zurück. Die Leitfragen dienen dabei als interpretatives Raster, die Teilfragen als der operative Nachweis, dass die eingangs gestellte Forschungsfrage vollständig beantwortet ist; damit schließt sich die Argumentationskette von der Raumgeometrie bis zur praktischen Empfehlung.

Was bleibt erhalten? Bis zu einer Bittiefe von 4-Bit bleibt die semantische Struktur nahezu vollständig erhalten: Distanzkorrelation ($r > 0,99$), Nachbarschaftserhaltung (Overlap $> 0,93$) und Retrieval-Qualität (mindestens 99,9 % der Float32-NDCG) sind praktisch ununterscheidbar von der Referenz. Selbst unter aggressiver Kompression bleibt die *Precision*-Perspektive stabil: Die durchgehend hohe Trustworthiness zeigt, dass auch bei 1-Bit kaum grob falsche Nachbarn eingeführt werden. Für zweistufige Systeme bleibt zudem der Recall@100 robust, sodass stark komprimierte Repräsentationen als Vorfilter brauchbar sind.

Was geht verloren? Der Verlust konzentriert sich auf die *Recall*-Perspektive niedrig-bitiger Verfahren: Bei 1-Bit und 2-Bit sinkt der Anteil erhaltener nächster Nachbarn deutlich, und die Retrieval-Qualität fällt bei 1-Bit auf rund 88 %. Diese Verluste lassen sich unmittelbar auf die in Phase 1 identifizierten Raumeigenschaften zurückführen und beantworten damit zugleich die Frage nach den quantisierbarkeitsbestimmenden Eigenschaften (Teilfrage 1): Die perfekte L2-Normierung macht die Binarisierung verlustfrei bezüglich der Norm, die symmetrischen Dimensionen legen den Vorzeichenschwellwert optimal, und die hohe Redundanz erlaubt Dimensionsreduktion. Der einzige verbleibende Verlustkanal ist die aufgegebene Magnitude pro Dimension, deren Wirkung sich im Verdünnungsparadox der naiven 1-Bit-Quantisierung unter Dimensionsreduktion zeigt und durch die varianzausgleichende Rotation von TurboQuant aufgehoben wird.

Was bedeutet das für die Praxis? Der Speicher- und Kostenvorteil wird erst bei 1-Bit durch einen signifikanten, aber mit etwa 88 % relativer Qualität nicht katastrophalen Qualitätsverlust erkaufte; bis 4-Bit ist der Vorteil quasi kostenlos. Auf der Laufzeitseite ist der Nutzen der Kompression im gemessenen NumPy-Pfad primär ein Speicher-, kein Geschwindigkeitsvorteil, da dichte Float-Matrixmultiplikation hochoptimiert ist. Der praktische Gewinn liegt somit in der Speicherreduktion, die kleinere Instanzklassen und lokale Deployments ermöglicht.

Welche Konfiguration ist praktisch sinnvoll? Es gibt keine universell optimale Konfiguration, sondern budgetabhängige Arbeitspunkte: `tq_4bit 1024d` für hohe Qualitätsanforderungen, `tq_2bit 1024d` als kostenoptimierter Kompromiss und `tq_1bit 768d` für speicherkritische lokale Szenarien. Über alle Betriebspunkte hinweg ist TurboQuant der naiven Quantisierung vorzuziehen, da es bei identischem Speicherbedarf die Instabilitäten unter Dimensionsreduktion vermeidet.

Fazit. Die Quantisierung von `bge-large-en-v1.5`-Embeddings erhält die für Retrieval entscheidende Struktur bis 4-Bit nahezu vollständig und bleibt selbst bei 1-Bit brauchbar. Der beobachtete Trade-off zwischen Kompressionsgrad und Suchqualität ist damit für das untersuchte Modell und die exakte Suche klar quantifiziert und auf die geometrischen Eigenschaften des Ausgangsraums zurückführbar.

Rückbindung an die Teilfragen der Forschungsfrage. Die vier eingangs gestellten Teilfragen sind damit vollständig beantwortet. *Teilfrage 1* (quantisierbarkeitsbestimmende Eigenschaften) beantwortet Phase 1: Perfekte L2-Normierung, symmetrische

und ausreißerarme Dimensionen sowie hohe Redundanz bestimmen die gute Quantisierbarkeit und erklären die späteren Verlustmuster kausal. *Teilfrage 2* (Erhaltung von Distanzen und Nachbarschaften) beantworten die Phasen 2 und 3: Die paarweise Distanzstruktur bleibt bis 4-Bit mit $r > 0,99$ erhalten, und die lokale Nachbarschaft bleibt in der Precision-Perspektive (Trustworthiness) selbst bei 1-Bit stabil, während die Recall-Perspektive (Overlap) bei niedrigen Bittiefen erwartungsgemäß nachgibt. *Teilfrage 3* (Schwelle signifikanter Retrieval-Verluste) beantwortet Phase 4: Bis 4-Bit ist der Qualitätsverlust vernachlässigbar; ein signifikanter, aber nicht katastrophaler Einbruch auf rund 88 % der Float32-Qualität tritt erst bei 1-Bit auf. *Teilfrage 4* (Verhältnis von Speicherersparnis zu Qualitätsverlust) beantworten die Phasen 5 und 6: Bis 4-Bit ist die Speicherersparnis quasi kostenlos, und die budgetabhängigen Arbeitspunkte übersetzen dieses Verhältnis in konkrete Einsatzszenarien.

Literatur

- [1] Nir Ailon und Bernard Chazelle. „The Fast Johnson-Lindenstrauss Transform and Approximate Nearest Neighbors“. In: *SIAM Journal on Computing* 39.1 (2009), S. 302–322. DOI: 10.1137/060673096.
- [2] Amazon Web Services. *Amazon EC2 Regional Price List: eu-central-1*. <https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonEC2/current/eu-central-1/index.csv>. CSV-Preisliste; abgerufen am 25.08.2026; Publication Date 2026-08-24T19:31:47Z. 2026.
- [3] Amazon Web Services. *AWS Pricing Calculator: Calculator Assumptions*. <https://aws.amazon.com/calculator/calculator-assumptions/>. Abgerufen am 25.08.2026. 2026.
- [4] Amazon Web Services. *Getting Price List Files Manually*. <https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/using-the-aws-price-list-bulk-api-fetching-price-list-files-manually.html>. Abgerufen am 25.08.2026; regionale AmazonEC2-Preisliste eu-central-1, Publication Date 2026-08-24T19:31:47Z. 2026.
- [5] Amazon Web Services. *Specifications for Amazon EC2 General Purpose Instances*. <https://docs.aws.amazon.com/ec2/latest/instancetypes/gp.html>. Abgerufen am 25.08.2026. 2026.
- [6] Amazon Web Services. *Specifications for Amazon EC2 Memory Optimized Instances*. <https://docs.aws.amazon.com/ec2/latest/instancetypes/mo.html>. Abgerufen am 25.08.2026. 2026.
- [7] Alexandr Andoni, Piotr Indyk, Thijs Laarhoven, Ilya Razenshteyn und Ludwig Schmidt. „Practical and Optimal LSH for Angular Distance“. In: *Advances in Neural Information Processing Systems 28 (NIPS 2015)*. 2015. arXiv: 1509.02897 [cs.DS].
- [8] Martin Aumüller, Erik Bernhardsson und Alexander Faithfull. „ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms“. In: *Information Systems* 87 (2020), S. 101374. DOI: 10.1016/j.is.2019.02.006.
- [9] M. G. Bulmer. *Principles of Statistics*. New York: Dover Publications, 1979. ISBN: 978-0-486-63760-0.

-
- [10] Paola Campadelli, Elena Casiraghi, Claudio Ceruti und Alessandro Rozza. „Intrinsic Dimension Estimation: Relevant Techniques and a Benchmark Framework“. In: *Mathematical Problems in Engineering* 2015 (2015), S. 759567. DOI: 10.1155/2015/759567.
- [11] Moses S. Charikar. „Similarity Estimation Techniques from Rounding Algorithms“. In: *Proceedings of the 34th Annual ACM Symposium on Theory of Computing (STOC)*. 2002, S. 380–388. DOI: 10.1145/509907.509965.
- [12] Ryan Codrai. *turbovec*. <https://github.com/RyanCodrai/turbovec>. GitHub Repository, abgerufen am 10.06.2026. 2024.
- [13] Nick Craswell. „Mean Reciprocal Rank“. In: *Encyclopedia of Database Systems*. Hrsg. von Ling Liu und M. Tamer Özsu. Boston, MA: Springer, 2009, S. 1703. DOI: 10.1007/978-0-387-39940-9_488.
- [14] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini und Hervé Jégou. „The Faiss Library“. In: (2024). arXiv: 2401.08281 [cs.LG].
- [15] Elena Facco, Maria d’Errico, Alex Rodriguez und Alessandro Laio. „Estimating the intrinsic dimension of datasets by a minimal neighborhood information“. In: *Scientific Reports* 7.1 (2017), S. 12140. DOI: 10.1038/s41598-017-11873-y.
- [16] Keinosuke Fukunaga und David R. Olsen. „An Algorithm for Finding Intrinsic Dimensionality of Data“. In: *IEEE Transactions on Computers* C-20.2 (1971), S. 176–183. DOI: 10.1109/T-C.1971.223208.
- [17] Tiezheng Ge, Kaiming He, Qifa Ke und Jian Sun. „Optimized Product Quantization“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36.4 (2014), S. 744–755. DOI: 10.1109/TPAMI.2013.240.
- [18] Yunchao Gong, Svetlana Lazebnik, Albert Gordo und Florent Perronnin. „Iterative Quantization: A Procrustean Approach to Learning Binary Codes for Large-Scale Image Retrieval“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.12 (2013), S. 2916–2929. DOI: 10.1109/TPAMI.2012.193.
- [19] Robert M. Gray und David L. Neuhoff. „Quantization“. In: *IEEE Transactions on Information Theory* 44.6 (1998), S. 2325–2383. DOI: 10.1109/18.720541.
- [20] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern und Sanjiv Kumar. „Accelerating Large-Scale Inference with Anisotropic Vector Quantization“. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*. Bd. 119. Proceedings of Machine Learning Research. 2020, S. 3887–3896. arXiv: 1908.10396 [cs.LG].
- [21] Richard W. Hamming. „Error Detecting and Error Correcting Codes“. In: *Bell System Technical Journal* 29.2 (1950), S. 147–160. DOI: 10.1002/j.1538-7305.1950.tb00463.x.

-
- [22] Harold Hotelling. „Analysis of a Complex of Statistical Variables into Principal Components“. In: *Journal of Educational Psychology* 24.6 (1933), S. 417–441. DOI: 10.1037/h0071325.
- [23] Kalervo Järvelin und Jaana Kekäläinen. „Cumulated Gain-Based Evaluation of IR Techniques“. In: *ACM Transactions on Information Systems* 20.4 (2002), S. 422–446. DOI: 10.1145/582415.582418.
- [24] Hervé Jégou, Matthijs Douze und Cordelia Schmid. „Product Quantization for Nearest Neighbor Search“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), S. 117–128. DOI: 10.1109/TPAMI.2010.57.
- [25] D. N. Joanes und C. A. Gill. „Comparing Measures of Sample Skewness and Kurtosis“. In: *Journal of the Royal Statistical Society: Series D (The Statistician)* 47.1 (1998), S. 183–189. DOI: 10.1111/1467-9884.00122.
- [26] Jeff Johnson, Matthijs Douze und Hervé Jégou. „Billion-Scale Similarity Search with GPUs“. In: *IEEE Transactions on Big Data* 7.3 (2021), S. 535–547. DOI: 10.1109/TBDATA.2019.2921572.
- [27] Ian T. Jolliffe. *Principal Component Analysis*. 2. Aufl. Springer Series in Statistics. New York: Springer, 2002. DOI: 10.1007/b98835.
- [28] Ian T. Jolliffe und Jorge Cadima. „Principal Component Analysis: A Review and Recent Developments“. In: *Philosophical Transactions of the Royal Society A* 374.2065 (2016), S. 20150202. DOI: 10.1098/rsta.2015.0202.
- [29] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen und Wen-tau Yih. „Dense Passage Retrieval for Open-Domain Question Answering“. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, S. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550.
- [30] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain und Ali Farhadi. „Matryoshka Representation Learning“. In: (2022). arXiv: 2205.13147 [cs.LG].
- [31] Elizaveta Levina und Peter J. Bickel. „Maximum Likelihood Estimation of Intrinsic Dimension“. In: *Advances in Neural Information Processing Systems 17 (NIPS 2004)*. 2004. URL: <https://proceedings.neurips.cc/paper/2004/hash/74934548253bcab8490ebd74afed7031-Abstract.html>.
- [32] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel und Douwe Kiela. „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks“. In: *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. 2020. arXiv: 2005.11401 [cs.CL].

-
- [33] Yu. A. Malkov und D. A. Yashunin. „Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), S. 824–836. DOI: 10.1109/TPAMI.2018.2889473.
- [34] Christopher D. Manning, Prabhakar Raghavan und Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge: Cambridge University Press, 2008. DOI: 10.1017/CB09780511809071.
- [35] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado und Jeffrey Dean. „Distributed Representations of Words and Phrases and their Compositionality“. In: *Advances in Neural Information Processing Systems 26 (NIPS 2013)*. 2013. arXiv: 1310.4546 [cs.CL].
- [36] Niklas Muennighoff, Nouamane Tazi, Loïc Magne und Nils Reimers. „MTEB: Massive Text Embedding Benchmark“. In: *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2023, S. 2014–2037. arXiv: 2210.07316 [cs.CL].
- [37] Rodrigo Nogueira und Kyunghyun Cho. „Passage Re-ranking with BERT“. In: (2019). arXiv: 1901.04085 [cs.IR].
- [38] Mohammad Norouzi, Ali Punjani und David J. Fleet. „Fast Search in Hamming Space with Multi-Index Hashing“. In: *2012 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2012, S. 3108–3115. DOI: 10.1109/CVPR.2012.6248043.
- [39] Zach Nussbaum, John X. Morris, Brandon Duderstadt und Andriy Mulyar. „Nomic Embed: Training a Reproducible Long Context Text Embedder“. In: (2024). arXiv: 2402.01613 [cs.CL].
- [40] James Jie Pan, Jianguo Wang und Guoliang Li. „Survey of Vector Database Management Systems“. In: *The VLDB Journal* 33.5 (2024), S. 1591–1615. DOI: 10.1007/s00778-024-00864-x.
- [41] Karl Pearson. „On Lines and Planes of Closest Fit to Systems of Points in Space“. In: *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 2.11 (1901), S. 559–572. DOI: 10.1080/14786440109462720.
- [42] Nils Reimers und Iryna Gurevych. „Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks“. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019, S. 3980–3990. DOI: 10.18653/v1/D19-1410.
- [43] Gerard Salton, Anita Wong und Chung-Shu Yang. „A Vector Space Model for Automatic Indexing“. In: *Communications of the ACM* 18.11 (1975), S. 613–620. DOI: 10.1145/361219.361220.

-
- [44] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda und Thomas Scialom. „Toolformer: Language Models Can Teach Themselves to Use Tools“. In: *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. 2023. arXiv: 2302.04761 [cs.CL].
- [45] Aamir Shakir, Tom Aarsen und Sean Lee. *Binary and Scalar Embedding Quantization for Significantly Faster & Cheaper Retrieval*. <https://huggingface.co/blog/embedding-quantization>. Hugging Face Blog, veröffentlicht am 22.03.2024, abgerufen am 26.09.2026. 2024.
- [46] Claude E. Shannon. „Coding Theorems for a Discrete Source with a Fidelity Criterion“. In: *IRE National Convention Record* 7.4 (1959), S. 142–163.
- [47] Mark D. Smucker, James Allan und Ben Carterette. „A Comparison of Statistical Significance Tests for Information Retrieval Evaluation“. In: *Proceedings of the 16th ACM Conference on Information and Knowledge Management (CIKM)*. 2007, S. 623–632. DOI: 10.1145/1321440.1321528.
- [48] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava und Iryna Gurevych. „BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models“. In: *NeurIPS Datasets and Benchmarks Track*. 2021. arXiv: 2104.08663 [cs.IR].
- [49] Jarkko Venna und Samuel Kaski. „Local multidimensional scaling“. In: *Neural Networks* 19.6–7 (2006), S. 889–899. DOI: 10.1016/j.neunet.2006.05.014.
- [50] Jarkko Venna und Samuel Kaski. „Neighborhood Preservation in Nonlinear Projection Methods: An Experimental Study“. In: *Artificial Neural Networks – ICANN 2001*. Bd. 2130. Lecture Notes in Computer Science. Springer, 2001, S. 485–491. DOI: 10.1007/3-540-44668-0_68.
- [51] Ellen M. Voorhees. „The TREC-8 Question Answering Track Report“. In: *Proceedings of the Eighth Text REtrieval Conference (TREC-8)*. Bd. 500-246. NIST Special Publication. 1999, S. 77–82. URL: https://trec.nist.gov/pubs/trec8/papers/qa_report.pdf.
- [52] Peter H. Westfall. „Kurtosis as Peakedness, 1905–2014. R.I.P.“ In: *The American Statistician* 68.3 (2014), S. 191–195. DOI: 10.1080/00031305.2014.917055.
- [53] Frank Wilcoxon. „Individual Comparisons by Ranking Methods“. In: *Biometrics Bulletin* 1.6 (1945), S. 80–83. DOI: 10.2307/3001968.
- [54] Shitao Xiao, Zheng Liu, Peitian Zhang und Niklas Muennighoff. „C-Pack: Packed Resources For General Chinese Embeddings“. In: (2023). arXiv: 2309.07597 [cs.CL].

- [55] Ikuya Yamada, Akari Asai und Hannaneh Hajishirzi. „Efficient Passage Retrieval with Hashing for Open-domain Question Answering“. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*. 2021, S. 979–986. DOI: 10.18653/v1/2021.acl-short.123.
- [56] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan und Yuan Cao. „ReAct: Synergizing Reasoning and Acting in Language Models“. In: *International Conference on Learning Representations (ICLR)*. 2023. arXiv: 2210.03629 [cs.CL].
- [57] Amir Zandieh, Majid Daliri, Majid Hadian und Vahab Mirrokni. „TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate“. In: (2025). arXiv: 2504.19874 [cs.LG].

Anhang

Eigenständigkeitserklärung

Wir versichern, die von uns vorgelegte Arbeit selbstständig verfasst zu haben. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Arbeiten anderer entnommen sind, haben wir als entnommen kenntlich gemacht. Sämtliche Quellen und Hilfsmittel, die wir für die Arbeit benutzt haben, sind angegeben. Die Arbeit hat mit gleichem Inhalt bzw. in wesentlichen Teilen noch keiner anderen Prüfungsbehörde vorgelegen.

Verwendete Hilfsmittel

Bei der Erstellung dieser Arbeit wurden die folgenden KI-gestützten Werkzeuge als technische Hilfsmittel eingesetzt:

- **Google Gemini** (Google DeepMind, 2026)
- **Google Antigravity CLI** (Google, 2026)
- **Kiro CLI** (Amazon, 2026)
- **OpenAI Codex / ChatGPT** (OpenAI, 2026)

Die genannten Werkzeuge wurden gleichermaßen für Recherche, Formulierung und Strukturierung der Projektarbeit, für die Codeentwicklung sowie für die Literaturrecherche eingesetzt. Die inhaltliche Verantwortung, die wissenschaftliche Argumentation, die Konzeption des Experiments sowie die Interpretation der Ergebnisse liegen vollständig bei den Verfassern. Alle von KI-Werkzeugen generierten Inhalte wurden kritisch geprüft, überarbeitet und in eigener Verantwortung in die Arbeit integriert.

Ort, Datum

Unterschrift

Ort, Datum

Unterschrift

Ort, Datum

Unterschrift